

## СОДЕРЖАНИЕ

|                                                                                                                                         |     |
|-----------------------------------------------------------------------------------------------------------------------------------------|-----|
| Введение.....                                                                                                                           | 7   |
| 1 Обзор литературы .....                                                                                                                | 8   |
| 1.1 Исследование предметной области .....                                                                                               | 8   |
| 1.2 Обзор аналогов .....                                                                                                                | 10  |
| 1.3 Обзор средств разработки .....                                                                                                      | 18  |
| 1.4 База данных.....                                                                                                                    | 24  |
| 1.5 Концепция разрабатываемой игровой системы .....                                                                                     | 26  |
| 1.6 Выводы.....                                                                                                                         | 27  |
| 2 Системное проектирование.....                                                                                                         | 29  |
| 2.1 Обоснование выбора средств разработки.....                                                                                          | 30  |
| 3 Функциональное проектирование .....                                                                                                   | 41  |
| 3.1 Описание структуры классов.....                                                                                                     | 41  |
| 3.2 Описание модели данных.....                                                                                                         | 68  |
| 4 Разработка программных модулей .....                                                                                                  | 70  |
| 4.1 Модуль конечного автомата .....                                                                                                     | 70  |
| 4.2 Модуль оценки контекста и угрозы .....                                                                                              | 72  |
| 4.3 Модуль памяти и распознавания паттернов поведения.....                                                                              | 77  |
| 4.4 Модуль адаптации весов .....                                                                                                        | 80  |
| 4.5 Модуль выбора состояния через функции полезности .....                                                                              | 82  |
| 4.6 Модуль процедурной генерации комнат .....                                                                                           | 88  |
| 5 Программа и методика испытаний.....                                                                                                   | 91  |
| 5.1 Главное меню и переход в хаб.....                                                                                                   | 91  |
| 5.2 Покупка улучшений в хабе .....                                                                                                      | 92  |
| 5.3 Запуск забега и активация комнат.....                                                                                               | 93  |
| 5.4 Открытие сундука и получение награды.....                                                                                           | 94  |
| 5.5 Модификатор рикошета стрелы .....                                                                                                   | 95  |
| 5.6 Модификатор мультивыстрела.....                                                                                                     | 95  |
| 5.7 Реакция адаптивной системы на изменение здоровья .....                                                                              | 97  |
| 5.8 Реакция адаптивной системы на изменение дистанции .....                                                                             | 97  |
| 5.9 Реакция адаптивной системы на число союзников .....                                                                                 | 98  |
| 5.10 Распознавание дальнобойного паттерна.....                                                                                          | 99  |
| 5.11 Распознавание агрессивного паттерна.....                                                                                           | 100 |
| 5.12 Эволюция весов за десятисекундное окно .....                                                                                       | 101 |
| 5.13 А/В-тест адаптивного и неадаптивного FSM.....                                                                                      | 102 |
| 5.14 Экран победы.....                                                                                                                  | 104 |
| 5.15 Экран поражения.....                                                                                                               | 105 |
| 5.16 Сохранение и восстановление метапрогрессии .....                                                                                   | 106 |
| 6 Руководство пользователя.....                                                                                                         | 107 |
| 6.1 Системные требования .....                                                                                                          | 107 |
| 6.2 Краткое руководство пользователя .....                                                                                              | 107 |
| 7 Техничко-экономическое обоснование разработки программного модуля<br>адаптивного поведения персонажей 2D-игры в жанре ActionRPG ..... | 114 |
| 7.1 Характеристика программного средства, разрабатываемого по                                                                           |     |

|                                                                                             |     |
|---------------------------------------------------------------------------------------------|-----|
| индивидуальному заказу .....                                                                | 114 |
| 7.2 Расчет инвестиций в разработку программного средства .....                              | 115 |
| 7.3 Расчет разработки и использования программного средства.....                            | 118 |
| 7.4 Расчет показателей экономической эффективности разработки<br>программного средства..... | 119 |
| 7.5 Вывод об экономической целесообразности реализации проектного<br>решения .....          | 119 |
| Заключение .....                                                                            | 121 |
| Список использованных источников .....                                                      | 122 |
| Приложение А .....                                                                          | 124 |
| Приложение Б .....                                                                          | 154 |
| Приложение В.....                                                                           | 155 |
| Приложение Г .....                                                                          | 156 |

## ВВЕДЕНИЕ

Игровые программные системы с процедурной организацией пространства и повторяемым циклом прохождения предъявляют требования к поведению противников. При фиксированных шаблонах реакции игровой процесс становится предсказуемым, поскольку последовательность действий противника повторяется при одинаковых условиях и в одинаковой конфигурации игрового пространства. Для проектов жанра ActionRPG это снижает вариативность боевых сценариев уже на ранних этапах взаимодействия игрока с системой и сокращает различие между отдельными игровыми столкновениями.

В рамках проектирования игровых систем данного типа задача управления противниками обычно решается через заранее заданные сценарии поведения, основанные на простых проверках расстояния до цели и факта обнаружения игрока. Такой подход обеспечивает предсказуемость реализации, однако не учитывает изменение состояния противника, плотность окружения и последовательность действий игрока. Для сохранения различий между боевыми ситуациями требуется отдельный программный модуль выбора поведения по совокупности входных параметров, способный изменять реакцию противника в одинаковых игровых условиях при различной последовательности действий игрока.

Игровая система используется как среда функционирования разработанного алгоритма и обеспечивает проверку его работы в условиях игрового цикла, включающего перемещение между комнатами, боевые взаимодействия, повторные прохождения и накопление прогресса между попытками. Основная часть проектирования сосредоточена на построении собственной схемы выбора поведения противников без применения встроенных средств описания поведения игрового движка.

Целью дипломного проекта является разработка программного модуля адаптивного поведения противников и создание двумерной игровой среды в жанре ActionRPG для его функционирования, обеспечивающих изменение состояний поведения противников на основе параметров игровой ситуации и последних действий игрока.

В соответствии с поставленной целью определены следующие задачи:

- разработать игровую среду, базовые игровые механики и модуль процедурной генерации комнат для функционирования программного модуля;
- спроектировать конечный автомат состояний поведения противников;
- реализовать модуль адаптивного выбора поведения на основе оценки угрозы, динамической адаптации весов факторов, памяти действий игрока и функций полезности состояний.

Данный дипломный проект выполнен мной лично, проверен на заимствования, процент оригинальности составляет 97.75% (отчет о проверке на заимствования прилагается).

# 1 ОБЗОР ЛИТЕРАТУРЫ

## 1.1 Исследование предметной области

Жанр ActionRPG представляет собой направление игровых систем, в котором механика непосредственного управления персонажем сочетается с элементами постепенного развития игровых характеристик в ходе прохождения. Для данного жанра характерно выполнение действий в реальном времени, когда игрок самостоятельно определяет траекторию движения, момент атаки, дистанцию взаимодействия с противником и порядок прохождения игровых участков. В отличие от проектов с заранее фиксированной последовательностью событий, структура прохождения в ActionRPG часто строится вокруг серии локальных боевых эпизодов, между которыми происходит изменение параметров персонажа, получение новых усилений или доступ к следующим игровым зонам. Отдельное развитие внутри жанра получили проекты с элементами roguelite, где после завершения попытки прохождения игровой цикл начинается заново, однако часть накопленного результата сохраняется и используется в последующих попытках. Такой подход позволяет соединить повторяемость базовых механик с постепенным усложнением игровых условий и формирует устойчивую модель многократного прохождения, при которой пользователь постепенно осваивает закономерности игрового пространства и поведения противников.

Игровые программные системы жанра ActionRPG ориентированы на непрерывное взаимодействие пользователя с игровым пространством в реальном времени. Основу игрового процесса составляют перемещение, выбор направления атаки, уклонение от угроз и последовательное прохождение игровых зон. В отличие от проектов, где действия разделены на отдельные фазы, здесь изменение игровой ситуации происходит непрерывно, а результат каждого действия сразу влияет на последующие игровые события. Для двумерных игровых проектов с видом сверху характерно ограниченное игровое пространство, внутри которого игрок одновременно контролирует собственное положение, направление атаки и расстояние до окружающих объектов. При такой организации уровня значительную роль приобретает плотность размещения игровых элементов внутри комнаты, поскольку даже небольшое изменение взаимного расположения объектов влияет на характер столкновения.

Комнатная структура используется как один из устойчивых способов организации игрового процесса в проектах данного жанра. Каждая отдельная зона представляет собой завершённый игровой эпизод с собственным набором препятствий, противников и допустимых направлений движения. После завершения взаимодействия внутри одной комнаты происходит переход к следующему игровому участку, где формируется новая комбинация условий. Такое построение позволяет дозированно изменять интенсивность прохождения и последовательно увеличивать сложность.

Повторяемость игровых действий в пределах одинаковых по структуре

столкновений быстро приводит к формированию устойчивых пользовательских сценариев. Если игровой противник действует по заранее фиксированному набору реакций, игрок после нескольких повторений определяет закономерность поведения и начинает использовать одинаковую последовательность действий в разных игровых ситуациях.

При многократных прохождении это особенно заметно в системах, где игровой цикл строится на последовательном повторении боевых эпизодов. Даже при изменении состава игровых объектов общий характер взаимодействия остается узнаваемым, если отсутствует изменение логики поведения противников. В результате различие между отдельными игровыми ситуациями начинает определяться преимущественно числом объектов на игровом поле.

Отдельное направление развития игровых систем связано с использованием повторных игровых циклов, при которых после завершения попытки пользователь начинает прохождение заново, сохраняя часть накопленного результата. Подобный принцип применяется для постепенного освоения механик и усложнения прохождения без изменения базовой структуры проекта. В таких условиях возрастает значение внутренних игровых алгоритмов, способных обеспечивать различие между повторяющимися игровыми эпизодами.

Дополнительный уровень вариативности достигается за счет изменения конфигурации игрового пространства. Перестроение порядка игровых зон, изменение состава объектов и последовательности переходов создают новые условия взаимодействия даже при ограниченном количестве исходных элементов. Однако изменение расположения объектов не устраняет предсказуемость поведения, если сами игровые реакции остаются неизменными.

В рамках предметной области поведение противников рассматривается как самостоятельный компонент игровой системы, определяющий характер локального столкновения. Противник ограничивает траекторию движения игрока, формирует временные интервалы для атаки и влияет на выбор безопасной позиции в пределах комнаты. При этом поведение должно сохранять понятную внутреннюю логику, чтобы действия игрового объекта воспринимались как закономерные.

Сложность проектирования заключается в том, что повышение интенсивности игрового процесса за счет увеличения числа противников или скорости их движения не приводит к изменению структуры взаимодействия. Игрок продолжает применять знакомую модель действий, адаптируя ее лишь к количеству угроз. По этой причине в исследовании игровых систем жанра ActionRPG внимание уделяется подходам, при которых различие игровых ситуаций достигается изменением выбора поведения внутри уже существующего набора игровых действий.

Такой подход изменяет характер взаимодействия без постоянного расширения количества игровых сущностей и сохраняет устойчивую архитектуру игрового процесса.

## 1.2 Обзор аналогов

При разработке игровых систем жанра ActionRPG анализ существующих проектов позволяет определить прагматичные решения, применяемые для организации игрового пространства, построения боевых сценариев и формирования повторяемого игрового цикла. В рамках обзора рассматриваются проекты, в которых используются элементы, близкие по структуре к разрабатываемой системе: комнатная организация уровня, последовательное прохождение игровых зон, высокая плотность боевого взаимодействия, повторный игровой цикл и постепенное усложнение поведения противников.

Выбор аналогов обусловлен тем, что в современных игровых проектах данного направления именно сочетание ограниченного пространства, интенсивного взаимодействия с противниками и вариативности игровых ситуаций определяет устойчивость игрового процесса при многократном прохождении. Отдельное значение имеет способ формирования сложности: в ряде проектов она достигается увеличением количества угроз, в других случаях за счет изменения структуры комнаты, темпа боя или набора действий противников.

Анализ аналогов позволяет определить, какие игровые механики оказывают наибольшее влияние на длительность удержания интереса пользователя и какие элементы могут быть использованы при построении собственной игровой модели. Особое внимание уделяется проектам, где повторяемость прохождения сочетается с изменением состава игровых ситуаций, а также присутствует выраженная зависимость между действиями игрока и характером ответной реакции игровых объектов.

### 1.2.1 The Binding of Isaac

The Binding of Isaac [1] является одним из наиболее показательных примеров построения двумерной игровой системы с комнатной организацией пространства и повторяемым циклом прохождения. Игровой процесс основан на последовательном прохождении замкнутых комнат, соединенных переходами, внутри которых игрок сталкивается с различными типами противников, препятствиями и объектами усиления. Каждая игровая зона представляет собой отдельный боевой эпизод с ограниченным пространством перемещения, где требуется одновременно контролировать направление движения, траекторию атаки и положение угроз.

Структура прохождения строится по принципу последовательного продвижения через этажи подземелья, составленные из случайным образом сформированных комнат. При каждом новом запуске изменяется порядок соединения игровых зон, расположение противников, типы доступных предметов и состав вспомогательных игровых событий. Это обеспечивает значительное число комбинаций даже при ограниченном наборе базовых элементов игрового пространства.

Существенной особенностью проекта является большое количество

игровых предметов, влияющих на характеристики персонажа и характер прохождения. На протяжении длительного периода развития проекта количество доступных предметов было расширено до нескольких сотен, включая пассивные усиления, модификаторы атаки, изменение скорости движения, дополнительные эффекты поражения и специальные игровые свойства. Комбинация нескольких предметов внутри одного прохождения формирует новые варианты поведения игрока и существенно изменяет характер взаимодействия с противниками. Такой объем контента сформировался в результате длительной разработки и многолетней поддержки проекта.

С точки зрения поведения противников игра использует большое количество заранее определенных схем движения и атак. Каждый тип противника действует в пределах собственного набора реакций, связанных с расстоянием до игрока, направлением движения и текущим положением внутри комнаты. Несмотря на разнообразие типов противников, внутренняя логика поведения каждого отдельного объекта остается фиксированной, что позволяет игроку постепенно распознавать повторяющиеся модели.

На рисунке 1.1 представлен игровой процесс The Binding of Isaac.

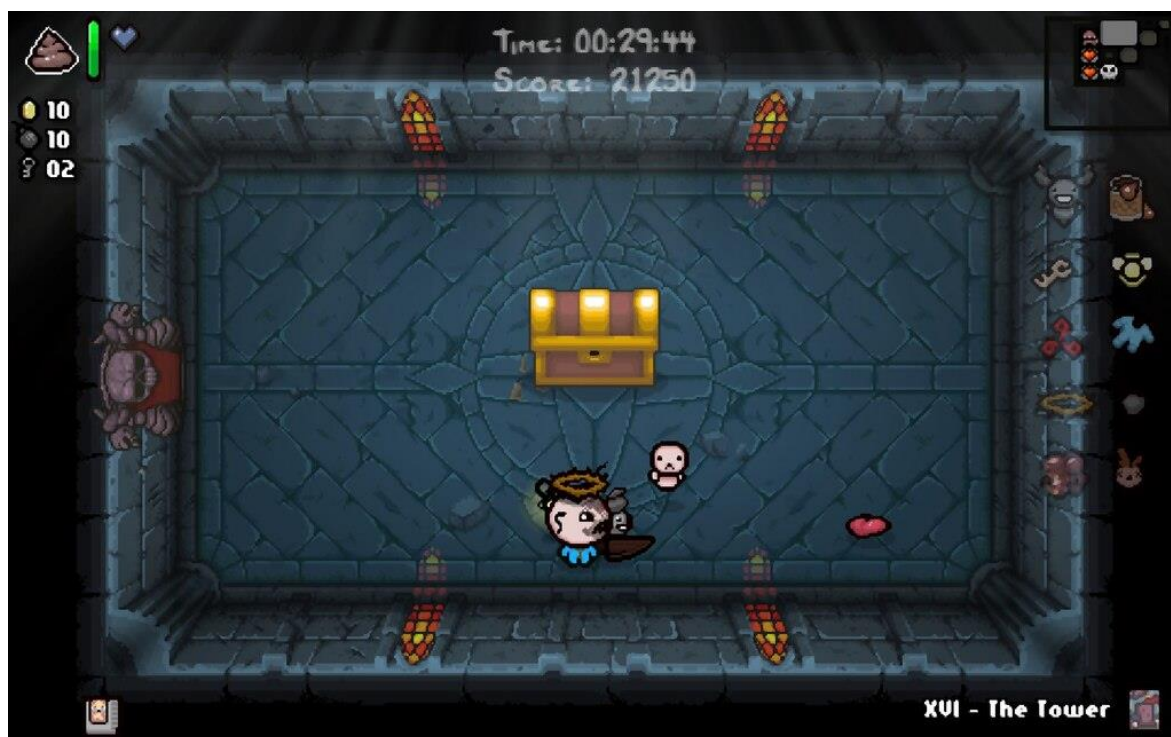


Рисунок 1.1 – Игровой процесс The Binding of Isaac

К преимуществам проекта относятся:

1 Высокая реиграбельность за счет процедурного формирования маршрута прохождения.

2 Большое количество предметов, формирующих различные игровые комбинации.

3 Устойчивая структура коротких боевых эпизодов внутри отдельных

комнат.

4 Значительное разнообразие противников и типов атак.

К недостаткам проекта относятся:

1 Поведение отдельных противников после многократного прохождения становится предсказуемым.

2 Существенная часть сложности формируется через увеличение числа активных угроз.

3 Отдельные игровые комбинации предметов создают резкий дисбаланс сложности.

4 При длительном взаимодействии пользователь начинает использовать заранее сформированные стратегии прохождения.

Данный проект представляет практический интерес для анализа, поскольку демонстрирует устойчивую модель построения игрового цикла, в которой процедурная организация пространства сочетается с большим объемом контента и фиксированными поведенческими шаблонами противников.

### **1.2.2 Enter the Gungeon**

Enter the Gungeon [2] представляет собой двумерный игровой проект, построенный на сочетании комнатной структуры прохождения, высокой плотности боевого взаимодействия и большого количества одновременно активных атакующих объектов на игровом поле. Игровой процесс организован в виде последовательного продвижения по этажам подземелья, где каждая отдельная комната содержит собственный набор противников, препятствий и элементов укрытия.

Структура прохождения построена на случайной генерации маршрута внутри каждого игрового цикла. В игре реализовано несколько сотен единиц вооружения, отличающихся скоростью стрельбы, формой траектории, дополнительными эффектами и взаимодействием с окружающими объектами. Значительная часть сложности формируется не количеством противников, а плотностью траекторий атак, которые одновременно присутствуют в пределах ограниченного пространства комнаты.

Проект развивался в течение продолжительного времени и получил крупные обновления после основного выпуска. Дополнительный контент включал новые игровые зоны, расширенный набор оружия, новых противников и дополнительные игровые события. Последовательное развитие системы позволило существенно увеличить число игровых комбинаций без изменения базовой структуры игрового цикла.

Поведение противников строится на заранее заданных схемах движения и атаки, однако за счет высокой скорости боя и различной геометрии комнат одни и те же типы противников воспринимаются по-разному в разных игровых ситуациях. Многие игровые объекты используют комбинированные траектории движения, резкие смены направления и интервальные атаки, что требует постоянной адаптации действий игрока.

На рисунке 1.2 представлен игровой процесс Enter the Gungeon.





Рисунок 1.2 – Игровой процесс Enter the Gungeon

К преимуществам проекта относятся:

1 Высокая динамика боевых эпизодов в пределах ограниченного пространства.

2 Большое разнообразие оружия и способов взаимодействия с противниками.

3 Использование геометрии комнаты как элемента боевой механики.

4 Процедурное построение игровых маршрутов.

К недостаткам проекта относятся:

1 Основная сложность часто определяется плотностью атак, а не изменением логики поведения противников.

2 При длительном прохождении часть боевых ситуаций начинает восприниматься через узнаваемые схемы движения.

3 Отдельные комбинации вооружения существенно снижают уровень сложности.

4 Высокая интенсивность визуальных эффектов затрудняет восприятие ситуации при большом количестве объектов.

Проект представляет интерес для анализа как пример игровой системы, где вариативность достигается через сочетание процедурного построения пространства, большого числа игровых комбинаций и высокой плотности боевого взаимодействия.

### 1.2.3 Hades

Hades [3] представляет собой игровой проект, в котором элементы ActionRPG сочетаются с повторяемым циклом прохождения, последовательным усложнением игровых зон и постоянным накоплением игровых улучшений между попытками. Игровой процесс построен вокруг последовательного прохождения отдельных боевых помещений,

объединенных в более крупные этапы, где каждая новая зона отличается составом противников, структурой пространства и характером боевых столкновений.

Система прохождения организована таким образом, что после завершения каждой комнаты игрок получает выбор дальнейшего маршрута и возможных усилений. Это формирует зависимость между краткосрочными решениями внутри текущего прохождения и накоплением преимуществ для последующих этапов.

В отличие от проектов, где основной акцент сделан на количестве предметов, здесь вариативность во многом достигается через управляемую систему выбора улучшений и постепенное изменение набора доступных игровых эффектов.

Поведение противников в игре строится на заранее определенных схемах, однако высокая скорость смены игровых ситуаций и различие геометрии комнат создают ощущение постоянного изменения условий взаимодействия. Многие типы противников используют короткие циклы подготовки атаки, быстрое перемещение и изменение дистанции относительно игрока, что требует постоянного контроля пространства.

Дополнительную роль играет постепенное усложнение игровых зон. На более поздних этапах прохождения возрастает плотность угроз, увеличивается число одновременно активных объектов и изменяется ритм столкновений, что усиливает требования к точности действий игрока.

На рисунке 1.3 представлен игровой процесс Hades.

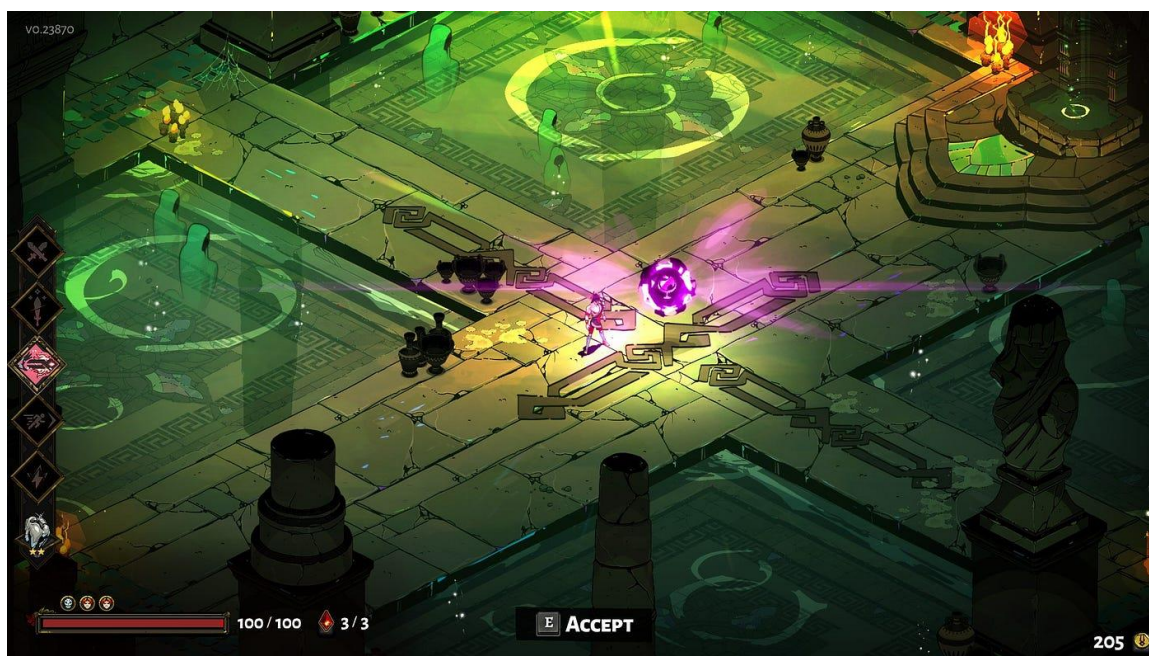


Рисунок 1.3 – Игровой процесс Hades

К преимуществам проекта относятся:

1 Устойчивая система повторного прохождения с накоплением прогресса между игровыми циклами.

2 Большое количество комбинаций временных усиления в пределах одного прохождения.

3 Высокая плотность боевого взаимодействия при сохранении читаемости игрового пространства.

4 Постепенное изменение сложности между игровыми этапами.

К недостаткам проекта относятся:

1 Поведение противников со временем становится узнаваемым.

2 Значительная часть сложности достигается через увеличение интенсивности столкновений.

3 При повторных прохождениях часть игровых решений переходит в устойчивые пользовательские шаблоны.

4 Высокая насыщенность эффектов затрудняет анализ отдельных игровых событий при плотном бою.

Проект представляет интерес для анализа как пример системы, в которой повторяемый игровой цикл сочетается с метапрогрессией, управляемой вариативностью усиления и устойчивой структурой боевых эпизодов.

#### **1.2.4 Dead Cells**

Dead Cells [4] представляет собой игровой проект, в котором механика динамичного боевого взаимодействия сочетается с повторяемым циклом прохождения, случайной перестройкой игровых зон и постепенным накоплением постоянных улучшений между игровыми попытками. Несмотря на то что проект выполнен в формате двумерного платформенного пространства, его структура по организации игрового цикла и управлению сложностью близка к системам жанра roguelite.

Игровое прохождение строится вокруг последовательного перемещения через взаимосвязанные игровые зоны, содержащие противников, боевые участки, скрытые маршруты и элементы усиления.

Особенность проекта связана с тем, что скорость игрового взаимодействия сохраняется высокой на протяжении всего прохождения. Большая часть боевых эпизодов требует постоянного перемещения, точного выбора момента атаки и контроля безопасной дистанции. При этом система движения и боя построена так, что игрок постоянно находится в режиме активного принятия решений.

Поведение противников строится на заранее заданных паттернах движения и атак, однако большое значение имеет различие скоростей отдельных игровых объектов и необходимость учитывать особенности окружающего пространства.

Многие противники используют резкие смены положения, быстрый выход на дистанцию удара и короткие интервалы между атаками, что требует от игрока постоянного анализа ближайшей игровой ситуации. Игрок самостоятельно выбирает направление дальнейшего продвижения, что влияет на состав противников, типы игровых зон и интенсивность последующих столкновений.

На рисунке 1.4 представлен игровой процесс Dead Cells.



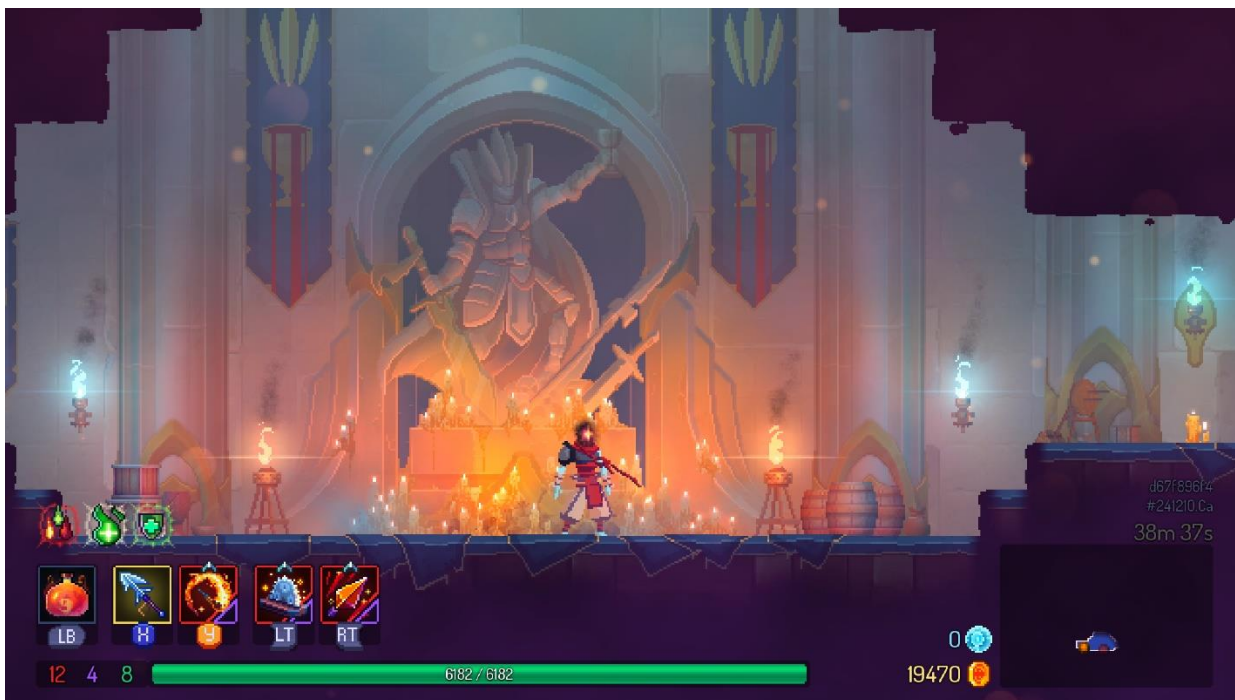


Рисунок 1.4 – Игровой процесс Dead Cells

К преимуществам проекта относятся:

1 Высокая скорость игрового процесса и устойчивый темп боевых эпизодов.

2 Большое количество комбинаций оружия и активных игровых навыков.

3 Процедурное изменение маршрутов прохождения.

К недостаткам проекта относятся:

1 Поведение противников после длительного прохождения становится предсказуемым внутри отдельных игровых зон.

2 Отдельные игровые комбинации вооружения снижают необходимость адаптации к поведению противников.

3 При повторных прохождениях пользователь начинает использовать устойчивые маршруты и заранее сформированные решения.

Проект представляет интерес для анализа как пример игровой системы, где высокая динамика прохождения сочетается с перестройкой игровых условий и накоплением игровых преимуществ между попытками.

### 1.2.5 Moonlighter

Moonlighter [5] представляет собой игровой проект, сочетающий элементы двумерного боевого прохождения подземелий и систему развития между игровыми циклами через внешнюю экономическую модель. Игровой процесс разделен на две взаимосвязанные части: прохождение боевых зон с последовательным исследованием подземелья и управление накопленными ресурсами вне активного игрового цикла.

Основная часть прохождения строится вокруг исследования подземелий, состоящих из отдельных комнат, соединенных переходами.

Внутри каждой комнаты размещаются противники, объекты взаимодействия и награды, получаемые после завершения боевого эпизода. После очистки зоны игрок получает возможность продолжить продвижение вглубь уровня или завершить текущую попытку с сохранением найденных ресурсов.

Особенностью проекта является сохранение результатов между игровыми циклами через систему развития внешней игровой среды. Полученные в подземелье ресурсы используются для расширения возможностей персонажа, улучшения доступного снаряжения и постепенного увеличения эффективности следующих прохождений. Такое построение связывает отдельные игровые попытки в единую последовательность развития.

Поведение противников построено на фиксированных схемах движения и атак, связанных с конкретным типом игрового объекта. Каждый тип противника использует ограниченный набор реакций, который определяется дистанцией до игрока и текущим положением внутри комнаты. При длительном взаимодействии пользователь постепенно формирует устойчивое понимание этих схем и заранее подбирает безопасную модель прохождения.

На рисунке 1.5 представлен игровой процесс Moonlighter.

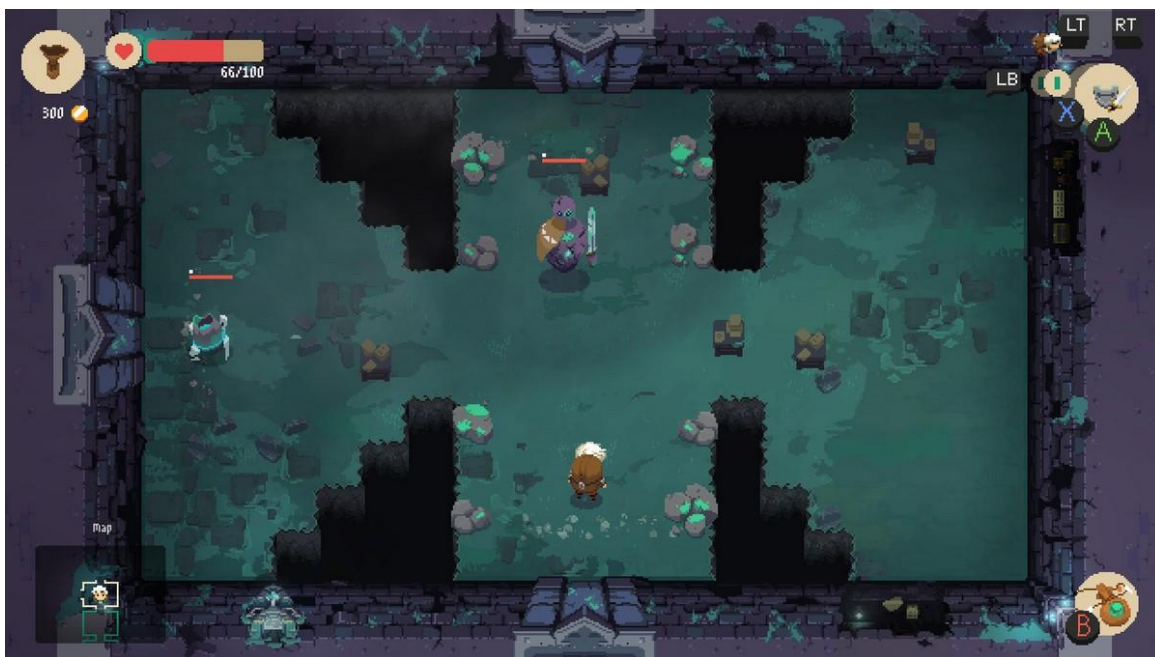


Рисунок 1.5 – Игровой процесс Moonlighter

К преимуществам проекта относятся:

- 1 Сочетание боевого прохождения и системы накопления прогресса между игровыми циклами.
- 2 Процедурное изменение структуры подземелий.
- 3 Разнообразие экипировки и способов ведения боя.
- 4 Устойчивая связь между отдельными игровыми попытками через систему развития.

К недостаткам проекта относятся:

1 Поведение противников внутри отдельных типов быстро становится узнаваемым.

2 Основная сложность часто определяется характеристиками экипировки.

3 При повторных прохождении часть игровых ситуаций начинает восприниматься как заранее известная последовательность действий.

4 Ограниченное число базовых поведенческих моделей снижает различие между поздними игровыми этапами.

Проект представляет интерес для анализа как пример игровой системы, где процедурная организация пространства сочетается с метапрогрессией и длительным накоплением игровых преимуществ между повторными прохождениями.

### **1.3 Обзор средств разработки**

Разработка игровых программных систем требует выбора инструментов, обеспечивающих одновременно построение игровой логики, управление визуальными объектами, организацию взаимодействия игровых сущностей и поддержку последующего расширения проекта. При выборе средств разработки учитываются особенности жанра, предполагаемый объем программной части, требования к производительности и доступность инструментов для интеграции различных компонентов в единую систему.

Для игровых проектов жанра ActionRPG существенное значение имеет возможность разделения исполнительской части и логических модулей. Игровая система включает обработку пользовательских действий, управление игровыми объектами, построение боевых сценариев, работу с визуальными эффектами и сохранение данных между игровыми циклами. Это требует использования средств, позволяющих сочетать высокоуровневое управление сценой с возможностью реализации отдельных алгоритмических компонентов на уровне исходного кода.

В современной игровой разработке используется широкий спектр языков программирования и специализированных программных платформ. Их выбор определяется требованиями конкретного проекта, типом игрового движка, моделью хранения данных и способом построения пользовательского взаимодействия. Для двумерных игровых систем с внутренней модульной логикой особое значение приобретают инструменты, позволяющие реализовывать собственные алгоритмы поведения без ограничения встроенными шаблонами.

#### **1.3.1 Языки программирования**

C++ [6] относится к числу основных языков, применяемых при разработке компьютерных игр и высокопроизводительных интерактивных систем. Язык сочетает объектно-ориентированный подход, возможность работы с низкоуровневыми структурами памяти и высокую скорость выполнения программного кода после компиляции. За счет прямого

управления ресурсами и широких возможностей оптимизации C++ используется в игровых движках, системах обработки графики, физическом моделировании и построении внутренних игровых алгоритмов.

В игровой разработке C++ широко применяется для создания модулей, работающих в условиях постоянного обновления игровых состояний, где каждая вычислительная операция выполняется многократно в течение игрового цикла. Это относится к системам обработки столкновений, расчета поведения игровых объектов, обновления координат, контроля состояний и внутреннего обмена данными между компонентами проекта. При построении игровых модулей такой подход позволяет сохранять предсказуемость структуры программы и гибкость в проектировании архитектуры.

Дополнительным преимуществом языка является возможность построения собственной иерархии классов и логического разделения игровых сущностей. В игровых проектах это позволяет выделять отдельные программные компоненты для управления персонажами, противниками, обработкой игровых событий и вычислительными алгоритмами. При использовании C++ разработчик получает доступ к внутренней структуре игрового движка и может реализовывать логику, не ограничиваясь только встроенными инструментами визуального программирования.

C# [7] широко применяется в игровой разработке благодаря сочетанию объектно-ориентированной модели, автоматического управления памятью и развитой стандартной библиотеки. Наиболее активно язык используется в проектах, создаваемых на движке Unity, где он является основным средством реализации игровой логики. C# обеспечивает удобное построение игровых сценариев, управление объектами сцены, обработку пользовательского ввода и создание интерфейсных компонентов.

Особенность языка заключается в более высоком уровне абстракции по сравнению с C++. Это упрощает создание игровых систем среднего масштаба и сокращает количество низкоуровневых операций. При этом часть вычислительных возможностей ограничивается внутренней архитектурой среды выполнения, что в крупных проектах требует более внимательного подхода к производительности.

Java [8] сохраняет устойчивое применение в проектах, где требуется кроссплатформенная серверная часть, распределенная обработка данных или интеграция игровых сервисов с внешними программными системами. Язык обладает развитой системой стандартных библиотек, строгой типизацией и стабильной моделью выполнения через виртуальную машину.

В игровой индустрии Java используется значительно реже для клиентской части игровых проектов, однако сохраняет значение в многопользовательских сервисах, серверных приложениях и системах обработки пользовательских данных. Его применение оправдано в случаях, когда основная нагрузка сосредоточена вне графической части проекта.

### **1.3.2 Среда разработки**

Unreal Engine [9] относится к числу наиболее развитых игровых

движков, применяемых при создании интерактивных систем различной сложности. Среда объединяет инструменты построения игровых сцен, систему работы с объектами, управление коллизиями, анимацией, визуальными эффектами и внутренней логикой проекта. Разработка внутри движка строится вокруг единой структуры проекта, где игровые объекты, уровни, визуальные ресурсы и программные классы связаны общей архитектурой.

Существенным преимуществом Unreal Engine является компонентный принцип построения игровых объектов. Каждый игровой объект может включать набор независимых компонентов, отвечающих за отображение, движение, обработку столкновений, получение событий и взаимодействие с другими объектами сцены. Такой подход позволяет формировать сложные игровые сущности без нарушения общей структуры проекта и обеспечивает удобное разделение отдельных функциональных частей.

Для двумерных игровых проектов в составе движка используется подсистема Paper2D, предназначенная для работы со спрайтами, покадровой анимацией, тайловыми картами и двумерными игровыми сценами. Подсистема позволяет формировать игровое пространство на основе тайловых элементов, управлять анимационными состояниями игровых объектов и создавать уровни с фиксированной геометрией или комнатной структурой.

Существует встроенная система визуальных эффектов Niagara, позволяющая формировать графические события, связанные с игровыми столкновениями, уничтожением объектов и реакцией на действия игрока. Визуальные эффекты внутри движка интегрируются непосредственно в игровую логику и могут запускаться как реакция на изменение состояния игровых объектов.

Для крупных игровых проектов использование Unreal Engine связано с высокими требованиями к вычислительным ресурсам рабочей станции и значительным объемом самого проекта. Даже при разработке двумерных игровых систем среда использует внутреннюю инфраструктуру, рассчитанную на сложные трехмерные сцены, что увеличивает объем служебных файлов и время компиляции. Полноценное применение движка наиболее оправдано в проектах, где требуется построение собственной программной архитектуры, работа с внутренними игровыми подсистемами и возможность расширения базовых механизмов. Наиболее активно Unreal Engine применяется при разработке ActionRPG, шутеров, проектов с развитой системой состояний игровых объектов и сложной внутренней логикой.

Unity [10] представляет собой широко распространенную игровую среду, используемую для разработки двухмерных и трехмерных приложений различного масштаба. В основе работы среды лежит компонентная модель построения игровых объектов, при которой каждый объект сцены получает набор подключаемых функциональных элементов, отвечающих за визуальное представление, физическое поведение, обработку событий и пользовательское взаимодействие.

Среда включает визуальный редактор сцен, систему анимации, встроенный физический движок, средства работы со звуком, интерфейсом и



ресурсами проекта. Основная логика реализуется на языке C#, который тесно интегрирован с внутренней структурой движка и используется для описания поведения игровых объектов, управления сценой и обработки пользовательских действий.

При этом в проектах, где требуется глубокая реализация собственных алгоритмов и высокая степень контроля над внутренними вычислениями, возможности среды ограничиваются архитектурой управляемого выполнения и структурой встроенных компонентов.

Unity сохраняет высокую эффективность при разработке мобильных игр, независимых проектов, двумерных платформеров, аркадных приложений и игровых систем среднего масштаба. Ограничения среды становятся заметны в проектах, где требуется глубокое изменение внутренних механизмов работы движка или высокая степень контроля над вычислительной частью. При увеличении числа одновременно активных игровых объектов часть задач требует дополнительной оптимизации. В крупных проектах архитектура управляемого выполнения может создавать ограничения при построении сложных вычислительных модулей.

Godot [11] представляет собой свободную игровую среду с открытым исходным кодом, предназначенную для разработки двумерных и трехмерных интерактивных систем. В основе движка используется собственная архитектура узлов, где каждый игровой объект строится как иерархия взаимосвязанных элементов, отвечающих за отдельные части поведения и отображения.

Среда включает редактор сцен, систему анимации, обработку коллизий, встроенные средства работы с тайловыми картами и визуальными эффектами. Для программирования используется язык GDScript, синтаксически близкий к Python, а также поддерживаются C# и дополнительные механизмы расширения через нативные модули.

Открытая архитектура дает возможность изменять внутренние части движка и адаптировать отдельные механизмы под задачи конкретного проекта. Однако при разработке крупных игровых систем количество готовых встроенных инструментов уступает более крупным коммерческим решениям.

Godot наиболее часто используется при создании небольших двумерных проектов, обучающих игровых приложений и независимых игровых прототипов. Ограничением среды остается сравнительно меньшее количество готовых профессиональных инструментов и библиотек по сравнению с коммерческими игровыми платформами. При построении крупных игровых проектов часть специализированных задач требует самостоятельной доработки или подключения дополнительных решений. На практике движок чаще применяется в проектах с ограниченной командой разработки и относительно компактной архитектурой.

Visual Studio [12] представляет собой интегрированную среду разработки, ориентированную на создание программных систем на языках C++, C#, Python и ряде других языков. Среда включает редактор исходного кода, систему интеллектуального автодополнения, средства навигации по

проекту, встроенный отладчик и инструменты анализа структуры программного кода.

При разработке игровых проектов Visual Studio используется для построения программной части игровых систем, включая создание игровых классов, модулей логики поведения, обработку игровых состояний и реализацию внутренних вычислительных алгоритмов. Среда обеспечивает прямую работу с программными файлами игровых проектов и поддерживает крупные кодовые базы.

Существует развитая система отладки, позволяющая пошагово анализировать выполнение программы, отслеживать значения переменных, состояние объектов и последовательность вызова функций.

Visual Studio активно используется при разработке игровых проектов, в которых основная часть логики реализуется на C++ или C#. Среда особенно востребована в проектах, создаваемых на Unreal Engine, а также при разработке собственных игровых модулей, серверных частей и вспомогательных инструментов сопровождения. Ограничением среды является значительное количество внутренних настроек и высокая зависимость удобства работы от корректной конфигурации проекта. При крупных игровых решениях время индексирования и анализа проекта возрастает вместе с объемом исходного кода.

### **1.3.3 Система контроля версий и хостинг**

Git [13] представляет собой распределенную систему контроля версий, предназначенную для фиксации изменений в исходном коде, отслеживания последовательности разработки и управления состоянием проекта на различных этапах его формирования. Основной принцип работы Git заключается в сохранении набора изменений в виде последовательных фиксаций, каждая из которых содержит информацию о состоянии проекта в конкретный момент времени.

Использование системы контроля версий имеет особое значение при разработке программных проектов, содержащих большое количество взаимосвязанных файлов, поскольку позволяет безопасно вносить изменения в архитектуру проекта, возвращаться к предыдущим состояниям и анализировать историю изменений. В игровых системах это особенно важно при параллельной работе с программной логикой, ресурсами проекта и внутренней структурой игровых модулей.

Одним из базовых механизмов Git является работа с ветвями разработки. Ветвление позволяет создавать отдельные направления изменения проекта, тестировать новые решения без влияния на основную рабочую версию и объединять завершенные изменения после проверки результата. Такой подход позволяет изолировать экспериментальные изменения и сохранять стабильность основной версии проекта.

В программной разработке Git применяется практически во всех типах проектов: от небольших учебных решений до крупных игровых и промышленных систем, где требуется постоянный контроль изменения

исходного кода и возможность возврата к стабильным состояниям проекта.

GitHub [14] представляет собой веб-платформу для размещения репозитория Git и удаленного хранения исходного кода. Платформа обеспечивает централизованное хранение проекта, доступ к истории изменений, управление версиями и возможность резервного копирования исходных файлов.

Использование GitHub позволяет хранить проект вне локальной среды разработки, синхронизировать изменения между рабочими устройствами и сохранять историю разработки независимо от состояния локального компьютера. Платформа также предоставляет инструменты просмотра истории изменений, анализа различий между версиями и управления отдельными ветвями проекта.

В программных проектах GitHub часто используется как средство подтверждения авторского вклада, поскольку каждая операция фиксации сопровождается временной отметкой и сохраняет последовательность внесенных изменений.

GitHub широко используется в индивидуальной и командной разработке программных систем, включая игровые проекты, веб-приложения, библиотеки и исследовательские программные решения. Ограничением платформы является зависимость от внешней облачной инфраструктуры, поскольку доступ к репозиторию и синхронизация изменений требуют подключения к сети. При работе с крупными игровыми проектами значительный объем бинарных файлов усложняет хранение ресурсов и требует дополнительной организации структуры репозитория. В практической разработке сервис распространен в независимых проектах, учебных работах, открытых программных решениях и распределенных командах.

GitLab [15] представляет собой платформу управления репозиториями, основанную на системе Git и дополненную средствами организации процессов сопровождения программного проекта. Помимо хранения исходного кода платформа включает инструменты контроля задач, отслеживания изменений, автоматической сборки и проверки состояния проекта.

Одной из особенностей GitLab является расширенная система управления этапами разработки, позволяющая связывать изменения в коде с конкретными задачами проекта. Это обеспечивает более детальную организацию крупных программных решений, где отдельные изменения должны фиксироваться как часть более общей структуры разработки.

Платформа применяется в проектах, где требуется более детальное управление процессом разработки и объединение хранения исходного кода с внутренней организацией задач. Ограничением является более сложная настройка отдельных механизмов сопровождения по сравнению с более простыми сервисами удаленного хранения. При локальном развертывании возрастает нагрузка на сопровождение собственной серверной инфраструктуры. В реальной разработке часто используется в корпоративных программных системах, внутренних инженерных проектах и командах, где требуется контроль полного цикла изменения программного продукта.

## 1.4 База данных

При проектировании программных систем выбор способа хранения данных определяется характером информации, частотой обращения к ней, требованиями к скорости доступа и объемом сохраняемых состояний. В игровых проектах база данных используется преимущественно для хранения параметров, которые должны сохраняться между игровыми циклами: пользовательские настройки, накопленный прогресс, статистические показатели, открытые улучшения и дополнительные игровые параметры.

Для игровых систем с локальным выполнением основной логики большая часть игровых состояний обрабатывается в оперативной памяти, поскольку игровые объекты, столкновения, вычисление состояний и поведение противников должны обновляться в реальном времени без обращения к внешнему хранилищу данных. База данных используется только для тех компонентов, которые сохраняются между отдельными игровыми запусками и должны быть доступны после завершения текущего игрового цикла.

SQLite [16] представляет собой встроенную реляционную базу данных, не требующую отдельного серверного процесса для работы. Вся структура данных хранится в одном локальном файле, который подключается непосредственно программой во время выполнения. Работа с данными осуществляется через стандартные SQL-запросы, что позволяет использовать привычную реляционную модель хранения без развертывания дополнительной серверной инфраструктуры.

База данных отличается минимальными требованиями к ресурсам и простотой интеграции в программный проект. Отсутствие отдельного сервера упрощает сопровождение базы данных и позволяет использовать ее как часть локальной структуры приложения. Для программных решений, где объем сохраняемых данных ограничен и отсутствует необходимость одновременного доступа большого числа пользователей, такая модель хранения является практичной.

В игровых системах широко применяется для хранения пользовательских настроек, параметров сохранения, статистики прохождений, данных метапрогрессии и локальных таблиц параметров. При этом текущая игровая логика обычно не взаимодействует с базой данных в режиме постоянного обновления, а обращение выполняется только в моменты сохранения или загрузки.

SQLite широко используется в игровых приложениях, где требуется локальное хранение ограниченного объема информации без построения отдельной серверной инфраструктуры. На практике база данных применяется в одиночных играх, мобильных приложениях, локальных системах сохранения, хранении параметров прохождения, игровых настроек и таблиц метапрогрессии.

Ограничением SQLite является отсутствие эффективной работы при большом количестве одновременных операций записи и невозможность

масштабирования под многопользовательскую нагрузку. Для игровых проектов, где основная часть логики выполняется локально, такие ограничения остаются допустимыми.

PostgreSQL [17] представляет собой объектно-реляционную систему управления базами данных, ориентированную на сложные структуры хранения и расширенные возможности обработки информации. Система поддерживает стандартные SQL-операции, транзакции, сложные типы данных, вложенные структуры и механизмы расширения через дополнительные модули.

База данных используется в проектах, где требуется работа со сложными взаимосвязанными данными, высокий уровень согласованности и возможность масштабирования под значительную вычислительную нагрузку. База данных обеспечивает устойчивую обработку больших массивов информации и поддерживает расширенные механизмы анализа данных.

При разработке игровых систем применяется преимущественно в серверной части многопользовательских проектов, системах аналитики, хранении больших объемов пользовательских данных и централизованной обработке статистики.

PostgreSQL используется в игровых системах, где требуется хранение сложных взаимосвязанных данных и работа с большими объемами серверной информации. В реальной практике база данных применяется в многопользовательских игровых платформах, аналитических игровых сервисах, системах обработки телеметрии и хранении больших игровых журналов.

Ограничением является более высокая сложность администрирования по сравнению с более легкими решениями. Для локальных игровых проектов использование такой базы данных требует ресурсов, не оправданных объемом сохраняемой информации.

MongoDB [18] представляет собой нереляционную базу данных, в которой информация хранится в виде документов с вложенной структурой. Данные организуются в JSON-подобном формате, что позволяет сохранять объекты без жесткой схемы таблиц и легко изменять внутреннюю структуру записей.

База данных используется в системах, где структура данных изменяется в процессе развития проекта или заранее не определяется полностью. Гибкость хранения позволяет работать с различными типами информации без предварительного формирования жесткой реляционной модели.

В программной разработке MongoDB применяется в высоконагруженных веб-сервисах, облачных системах, приложениях с динамическими пользовательскими данными и распределенных сервисах.

Для игровых проектов база данных может использоваться при хранении телеметрии, событий больших игровых сессий, пользовательских журналов и данных, не требующих строгих реляционных связей.

Ограничением MongoDB является отсутствие жесткой реляционной структуры, что усложняет контроль целостности данных при хранении параметров,

связанных строгими зависимостями. Для игровых проектов с небольшой локальной системой сохранения применение документной модели хранения часто оказывается избыточным. Когда требуется точная структура хранения и устойчивые связи между наборами параметров, требуется дополнительный контроль схемы данных.

База данных применяется в игровых проектах, связанных с обработкой больших объемов событийных данных, хранением телеметрии, пользовательской активности и динамических игровых журналов.

Ограничением MongoDB является отсутствие жесткой реляционной структуры, что усложняет контроль целостности данных при хранении параметров, связанных строгими зависимостями. Для игровых проектов с небольшой локальной системой сохранения применение документной модели хранения часто оказывается избыточным.

## **1.5 Концепция разрабатываемой игровой системы**

Разрабатываемая игровая система относится к двумерным проектам жанра ActionRPG с последовательным прохождением игровых зон, повторяемым циклом попыток и постепенным накоплением игровых преимуществ между отдельными прохождениями. В основе игровой структуры используется модель продвижения через отдельные комнаты подземелья, внутри которых формируются локальные боевые эпизоды с ограниченным пространством взаимодействия.

Игровое пространство организовано как последовательность взаимосвязанных комнат, соединенных переходами. Каждая игровая зона содержит собственный набор противников, препятствий и доступных направлений перемещения. После завершения взаимодействия внутри одной комнаты открывается возможность перехода к следующему участку подземелья.

Сюжетная основа проекта связана с подземельем, расположенным под руинами древней крепости, где скрывается маг-лич, ранее занимавший положение придворного колдуна и изгнанный за проведение экспериментов с магией времени и иллюзий. После изгнания он продолжил исследования в изолированном пространстве, где сформировал магическую область с нестабильной внутренней структурой

В пределах созданного пространства комнаты подземелья постоянно изменяют расположение, переходы между зонами перестраиваются, а внутренняя структура игрового маршрута не сохраняется между отдельными попытками прохождения. Это создает сюжетное объяснение процедурного формирования игрового пространства и позволяет связать техническую организацию уровня с внутренней логикой игрового мира

Игровой персонаж представлен рыцарем, направленным в подземелье по приказу правителя королевства для устранения источника угрозы. На раннем этапе прохождения становится ясно, что обычное продвижение внутрь подземелья не приводит к завершению задачи, поскольку после поражения



персонаж возвращается к начальному моменту игрового цикла.

Повторяемость игровых попыток объясняется действием временной аномалии, возникшей как следствие магических экспериментов внутри подземелья. После каждой неудачной попытки прохождения персонаж сохраняет часть накопленного опыта, полученных усилений и знаний о внутренней структуре игровых ситуаций. Это формирует устойчивую модель повторного игрового цикла, в которой каждая новая попытка использует результат предыдущего прохождения

Такое построение игровой концепции позволяет связать сюжетную основу, организацию игрового пространства и внутренние механизмы изменения сложности в пределах единой модели игрового процесса.

## **1.6 Выводы**

На основании проведенного обзора предметной области, анализа существующих игровых решений и рассмотрения используемых средств разработки сформированы основные положения, определяющие дальнейшее проектирование дипломного проекта.

Исследование предметной области показало, что для проектов жанра ActionRPG устойчивый интерес представляет сочетание ограниченного игрового пространства, повторяемого цикла прохождения и постепенного усложнения игровых ситуаций. Комнатная организация уровня позволяет формировать локальные боевые эпизоды с контролируемым набором условий. При повторных прохождениях значение приобретает не только изменение структуры пространства, но и сохранение различий между отдельными столкновениями за счет изменения логики игровых объектов.

Анализ существующих проектов показал, что современные решения жанра ActionRPG обеспечивают высокую реиграбельность за счет процедурного изменения маршрута прохождения, расширенного набора предметов и вариативности вооружения. При этом даже в проектах с большим количеством игровых комбинаций поведение противников внутри отдельных типов часто строится на фиксированных паттернах, которые после многократного взаимодействия становятся узнаваемыми.

Рассмотренные аналоги показывают, что в игровых системах с повторяемой структурой прохождения усложнение нередко достигается увеличением числа противников, ростом плотности атак или расширением набора игровых объектов. В большинстве решений вариативность достигается за счет расширения набора предметов, усложнения конфигурации игровых зон, увеличения числа противников и введения дополнительных механик усиления персонажа. При этом внутренняя логика поведения обычно ограничивается заранее заданными схемами движения и фиксированными условиями атаки. В разрабатываемом проекте основное отличие заключается в выделении отдельного программного модуля адаптивного поведения как самостоятельного объекта разработки, где изменение реакции противника определяется совокупностью параметров игровой ситуации, включая

дистанцию до игрока, текущее состояние противника, плотность игрового пространства и сохраненные данные о последних действиях игрока.

Проведенный обзор языков программирования подтвердил, что для реализации алгоритмической части игровых систем, требующей постоянного обновления состояний и управления внутренней логикой игровых объектов, наибольшую практическую применимость сохраняет язык C++. Его использование обеспечивает построение собственной архитектуры классов, реализацию конечного автомата состояний и вычисление параметров выбора поведения в пределах игрового цикла.

В качестве основной среды разработки выбран Unreal Engine, поскольку движок предоставляет готовую инфраструктуру для организации игровых сцен, управления объектами, обработки столкновений и построения двумерной игровой среды. Для программной реализации модулей поведения применяется среда Visual Studio, обеспечивающая работу с исходным кодом проекта и отладку игровых классов.

Для сопровождения проекта и фиксации этапов разработки используется Git как система контроля версий и GitHub как средство удаленного хранения исходного кода. Применение системы контроля версий позволяет последовательно фиксировать этапы формирования программного модуля, сохранять историю изменений и подтверждать самостоятельность разработки.

Для хранения данных между игровыми циклами выбрана SQLite, поскольку структура сохраняемой информации в проекте ограничивается параметрами метапрогрессии и статистикой прохождений. Выбор соответствует архитектуре одиночной игровой системы, где основная обработка игровых состояний выполняется в оперативной памяти, а обращение к базе данных требуется только при сохранении и загрузке данных.

Принятая игровая концепция позволяет использовать игровую систему как среду функционирования программного модуля адаптивного поведения.

В качестве средств разработки были выбраны:

- для реализации программного модуля поведения и основной игровой логики: язык программирования C++ и среда разработки Visual Studio;
- для формирования игровой среды и организации взаимодействия игровых объектов: Unreal Engine;
- для хранения параметров метапрогрессии и статистики между игровыми циклами: SQLite;
- для контроля этапов разработки и фиксации изменений исходного кода Git и GitHub.

Выбранные технологические решения соответствуют поставленной цели дипломного проекта и обеспечивают переход к этапу проектирования архитектуры программного модуля и реализации игровой системы.

## 2 СИСТЕМНОЕ ПРОЕКТИРОВАНИЕ

Требуется определить общую структуру программной системы и выделить функциональные компоненты, обеспечивающие работу игрового приложения и программного модуля адаптивного поведения противников. Поскольку объектом разработки является не только отдельный программный модуль адаптивного поведения, но и игровая среда, архитектура проекта должна учитывать разделение между исполнительной частью игрового процесса и логикой принятия решений внутри игровых объектов.

Разрабатываемая система представляет собой двумерное игровое приложение жанра ActionRPG, в котором игровая среда используется как пространство функционирования программного модуля адаптивного поведения. Игровой цикл включает обработку пользовательского ввода, управление игровым персонажем, формирование боевых столкновений, переход между комнатами, изменение игровых состояний и сохранение части данных между отдельными прохождениями.

Для построения архитектуры проекта выделяются основные функциональные блоки:

- блок обработки пользовательского ввода;
- блок управления персонажем;
- блок генерации уровня;
- блок боевой системы;
- блок адаптивного поведения противников;
- блок управления игровым состоянием;
- блок пользовательского интерфейса;
- блок сохранения и метапрогрессии;
- блок отображения игры.

Работа программной системы организуется следующим образом:

- пользовательские действия поступают в блок обработки ввода;
- команды движения и игровых действий передаются в блок управления персонажем;
- команды системного характера передаются в блок управления игровым состоянием;
- перемещение и атака игрока изменяют состояние боевой системы;
- блок боевой системы взаимодействует с блоком поведения противников;
- блок адаптивного поведения получает игровые параметры и определяет текущее состояние противника;
- блок управления игровым состоянием контролирует переход между комнатами, завершение игровых эпизодов и изменение стадий прохождения;
- блок генерации уровня формирует структуру игровых комнат;
- блок пользовательского интерфейса отображает параметры состояния игрока и текущие игровые показатели;
- блок сохранения фиксирует накопленные параметры между игровыми циклами;

– блок отображения обеспечивает визуальное представление игровой сцены.

Взаимодействие между функциональными блоками отражается в общей структурной схеме ГУИР.400201.112 С1, где центральное положение, на ряду с блоком управления игровым состоянием, занимает блок адаптивного поведения противников как основной объект программной разработки.

## **2.1 Обоснование выбора средств разработки**

Выбор средств разработки определяется требованиями к архитектуре программной системы, характером вычислительной нагрузки и необходимостью разделения игровой среды и программного модуля принятия решений. Поскольку в рамках дипломного проекта разрабатывается не только игровое приложение, но и отдельный алгоритм адаптивного поведения противников, используемые инструменты должны обеспечивать возможность реализации собственной логики состояний, управления игровыми объектами и сопровождения исходного кода на уровне программных классов.

При выборе технологической основы учитывались несколько критериев: поддержка двумерного игрового пространства, возможность построения модульной структуры проекта, работа с покадровой анимацией, доступ к внутренней программной архитектуре и наличие инструментов отладки. Отдельное внимание уделялось возможности разделения вычислительной логики и исполнительской части игровых объектов, поскольку основной объект разработки связан с построением самостоятельного программного модуля поведения.

Дополнительным критерием являлась возможность локального хранения данных между игровыми циклами без усложнения общей структуры проекта серверной инфраструктурой. Для сопровождения разработки также учитывалась необходимость фиксации изменений исходного кода и сохранения последовательности формирования программной части проекта.

## **2.2 Язык программирования C++**

C++ является одним из базовых языков программирования, применяемых при разработке высокопроизводительных программных систем, где требуется постоянная обработка большого количества состояний и контроль над внутренней структурой выполнения. Язык сочетает объектно-ориентированные механизмы проектирования, работу с памятью и высокую скорость выполнения после компиляции, благодаря чему сохраняет устойчивое применение в игровой индустрии и инженерных приложениях.

В игровой разработке C++ используется при построении игровых движков, реализации игровых сущностей, обработке столкновений и вычислительных модулей, работающих в пределах каждого кадра игрового цикла. В отличие от языков, использующих управляемую среду выполнения, C++ позволяет напрямую контролировать жизненный цикл игровых объектов

и порядок обновления логики.

В рамках данного дипломного проекта применение C++ связано с разработкой отдельного модуля адаптивного поведения противников. Модуль должен функционировать независимо от встроенных шаблонов движка и определять состояние противника на основе параметров игровой ситуации: расстояния до игрока, состояния противника, ближайших объектов, недавних действий игрока и условий игровой комнаты.

Состояния поведения противника, включая ожидание, преследование, атаку, отход и обходное перемещение, оформляются как система переходов между состояниями. Каждый переход определяется набором условий и вычисляемых параметров. Использование C++ позволяет организовать такую структуру через отдельные классы и методы обработки состояний.

Основные преимущества использования C++ в рамках проекта:

1 Высокая скорость выполнения вычислений, необходимая для многократного выполнения алгоритма поведения в игровом цикле.

2 Возможность построения собственной объектной архитектуры для противников, вычислительного блока угрозы и FSM.

3 Прямая работа с игровыми классами Unreal Engine.

4 Точное управление внутренним состоянием игровых объектов.

Еще одним преимуществом является возможность отделить вычислительную часть алгоритма от исполнительской логики игрового объекта. Игровой противник в такой структуре получает готовое решение из программного модуля и реализует соответствующее действие внутри игровой сцены.

### **2.3 Среда разработки и игровой движок**

Unreal Engine представляет собой интегрированную среду разработки, предназначенную для создания интерактивных приложений и игровых систем. Движок объединяет инструменты построения сцен, управления объектами, обработки коллизий, анимации, визуальных эффектов и программной логики в пределах единой архитектуры.

Внутренняя архитектура Unreal Engine строится на компонентном принципе, при котором игровой объект формируется как совокупность функциональных элементов. Отдельные компоненты отвечают за отображение, столкновения, движение, взаимодействие и выполнение логики. Такой подход позволяет разделять визуальную часть объекта и его внутреннее поведение.

В рамках дипломного проекта движок используется как среда функционирования программного модуля адаптивного поведения противников. Игровая система обеспечивает двумерное пространство, обработку столкновений, переходы между комнатами и обновление состояний игровых объектов.

Для двумерной части проекта используется Paper2D, обеспечивающий работу со спрайтами, покадровой анимацией и тайловыми картами. С его

помощью формируется игровое пространство, состоящее из комнат, переходов и статических элементов уровня.

Визуальные эффекты столкновений и кратковременных игровых событий реализуются средствами Niagara, что позволяет связывать игровые события с эффектами попадания и уничтожения объектов.

Blueprints используются для вспомогательных настроек игровых объектов, тогда как основная логика программного модуля реализуется средствами C++.

Преимущества использования Unreal Engine:

- 1 Прямая интеграция с C++.
- 2 Компонентная организация игровых объектов.
- 3 Наличие Paper2D для двумерной разработки.
- 4 Поддержка визуальных эффектов через Niagara.

Использование Unreal Engine позволяет сохранить инженерный акцент на программной части, поскольку игровая среда предоставляет инфраструктуру для реализации основного объекта разработки.

## **2.4 Среда разработки Visual Studio**

Visual Studio представляет собой интегрированную среду разработки, предназначенную для создания, сопровождения и отладки программных проектов различной сложности. Среда поддерживает C++, C#, Python и предоставляет инструменты анализа кода, отладки и профилирования.

В рамках дипломного проекта Visual Studio используется как основная среда программной реализации игровых классов и вычислительных модулей. Поскольку основная часть разработки связана с построением собственного модуля адаптивного поведения, требуется инструмент, позволяющий сопровождать исходный код и контролировать выполнение логики.

Среда обеспечивает прямую интеграцию с Unreal Engine, что позволяет автоматически подключать игровые классы к структуре проекта движка. Visual Studio используется для генерации исходных файлов классов, реализации методов и сопровождения программной архитектуры.

Одним из преимуществ является развитая система анализа кода: редактор определяет типы данных, связи между классами и доступные методы, что снижает вероятность синтаксических ошибок.

Отдельное значение имеет встроенный отладчик, позволяющий пошагово анализировать выполнение программы, значения переменных и переходы между состояниями, что особенно важно для FSM.

Преимущества использования Visual Studio:

- 1 Полная интеграция с Unreal Engine.
- 2 Удобная работа с иерархией игровых классов.
- 3 Инструменты навигации по проекту.
- 4 Поддержка Git.

Visual Studio используется как основной инструмент реализации вычислительной части проекта.

## **2.5 База данных SQLite**

SQLite представляет собой встроенную реляционную базу данных, предназначенную для локального хранения информации внутри приложения без отдельного серверного процесса и подключения к сети. Вся структура данных сохраняется в одном локальном файле, а работа выполняется через SQL-запросы.

В рамках дипломного проекта база данных используется для хранения параметров метапрогрессии и статистики прохождений в двух разных таблицах.

Основная игровая логика, включая обработку состояний объектов, вычисление поведения противников и столкновения, выполняется в оперативной памяти. По этой причине база данных не участвует в обработке активного игрового процесса и используется только при сохранении и загрузке.

Выбор SQLite обусловлен тем, что структура данных проекта имеет ограниченный объем и не требует серверной архитектуры.

Преимущества использования SQLite:

- 1 Отсутствие отдельного сервера.
- 2 Простая интеграция с программной частью проекта.
- 3 Низкие требования к ресурсам.
- 4 Удобное локальное хранение данных.
- 5 Поддержка стандартных SQL-запросов.

SQLite соответствует архитектуре одиночного игрового приложения, где база данных выполняет вспомогательную функцию долговременного хранения.

## **2.6 Система контроля версий GitHub**

Git представляет собой распределенную систему контроля версий, предназначенную для фиксации изменений исходного кода и управления состоянием проекта на различных этапах разработки.

В рамках дипломного проекта Git используется для сопровождения разработки программного модуля адаптивного поведения и игровой среды. Поскольку проект включает множество взаимосвязанных файлов, требуется инструмент, позволяющий безопасно вносить изменения и возвращаться к предыдущим состояниям.

Одной из ключевых возможностей Git является работа с ветвями разработки, позволяющая изолировать изменение отдельных частей проекта без нарушения основной версии.

Дополнительное значение имеет история изменений: каждая фиксация содержит временную отметку и описание этапа разработки.

GitHub используется как удаленная платформа хранения репозитория. Платформа обеспечивает резервное хранение кода, доступ к различным версиям проекта и сохранение истории разработки.



Преимущества использования Git и GitHub:

- 1 Последовательная фиксация этапов разработки.
- 2 Возврат к предыдущим версиям.
- 3 Работа с ветвями разработки.
- 4 Удаленное хранение исходного кода.
- 5 Подтверждение авторской последовательности разработки.

Использование Git и GitHub позволяет сопровождать проект как последовательный инженерный процесс.

## **2.7 Блок обработки пользовательского ввода**

Блок обработки пользовательского ввода предназначен для получения управляющих сигналов от пользователя и их преобразования в команды, используемые внутренними подсистемами игрового приложения. В пределах игрового цикла данный блок является начальной точкой формирования действий игрока, поскольку каждое изменение положения персонажа, запуск атаки или вызов игрового меню начинается с регистрации пользовательского воздействия.

В рамках проекта управление строится на использовании клавиатурного ввода. Перемещение игрового персонажа осуществляется через набор направлений, а действия атаки формируются через отдельные команды, определяющие момент запуска боевого взаимодействия. При обработке пользовательского ввода фиксируется не только факт нажатия, но и удержание клавиши, поскольку движение персонажа должно сохраняться до завершения действия пользователя.

Дополнительно блок выполняет фильтрацию команд, которые не должны передаваться в игровые подсистемы при определенных состояниях приложения, например при открытом меню, завершении игрового цикла или переходе между комнатами.

Функции блока обработки пользовательского ввода:

- регистрация нажатий управляющих клавиш;
- определение направления перемещения персонажа;
- передача команд запуска атаки;
- ограничение передачи команд в неактивных игровых состояниях;
- формирование управляющих параметров для последующей обработки.

Связи блока с другими функциональными компонентами:

– блок управления персонажем: управляющие сигналы движения и действий передаются для изменения положения игрового объекта и запуска игровых действий;

– блок управления игровым состоянием: отдельные команды используются для изменения режима работы приложения, включая паузу и системные переходы.

Блок обработки пользовательского ввода формирует исходный поток команд, от которого зависит изменение как игровых действий, так и системных состояний внутри программной структуры проекта.

## **2.8 Блок управления персонажем**

Блок управления персонажем предназначен для обработки команд, поступающих из блока пользовательского ввода, и преобразования их в изменение состояния игрового объекта внутри игровой сцены. В рамках проекта данный блок отвечает за движение игрового персонажа, изменение направления, запуск атакующих действий и обновление параметров состояния игрока.

Основой работы блока является постоянное обновление координат персонажа в пределах игрового пространства. После получения управляющих сигналов определяется направление перемещения, рассчитывается новое положение игрового объекта и выполняется проверка допустимости движения с учетом границ комнаты, препятствий и статических элементов уровня.

Дополнительно блок обеспечивает управление направлением атаки. В проекте реализуются атаки в фиксированных направлениях, соответствующих ориентации персонажа. При запуске атаки фиксируется текущее направление, после чего создается соответствующее боевое действие.

Функции блока управления персонажем:

- обработка параметров движения;
- обновление положения персонажа;
- определение текущего направления ориентации;
- запуск атакующих действий;
- обновление внутренних параметров состояния игрока;
- передача игровых параметров в боевые подсистемы.

Связи блока с другими функциональными компонентами:

- блок обработки пользовательского ввода: получает управляющие команды движения и действий;
- блок боевой системы: передает параметры атаки и состояние оружия;
- блок адаптивного поведения противников: предоставляет данные о положении игрока и характере действий.

Блок управления персонажем формирует игровые параметры, которые используются в вычислении поведения противников

Блок обработки пользовательского ввода формирует исходный поток команд, от которого зависит изменение игровых действий и системных состояний проекта.

## **2.9 Блок генерации уровня**

Блок генерации уровня предназначен для формирования структуры игрового пространства, в пределах которого осуществляется последовательное прохождение игровых эпизодов. В рамках проекта игровая среда строится по комнатному принципу, где каждая зона представляет собой отдельный участок подземелья с набором препятствий.

Основой работы блока является создание взаимосвязанных комнат, соединенных переходами в пределах одного игрового цикла. При каждом

новом запуске изменяется порядок расположения комнат и направление дальнейшего продвижения внутри уровня.

Каждая комната формируется как участок тайловой карты, содержащий напольные элементы, стены, проходы и допустимые области размещения игровых объектов. После создания структуры помещения определяется расположение входных и выходных зон.

Функции блока генерации уровня:

- формирование набора игровых комнат;
- определение взаимного расположения комнат;
- построение структуры проходов;
- размещение тайлов пола и стен;
- выделение позиций появления игровых объектов;
- контроль связности игрового маршрута.

Связь блока с другим функциональным компонентом – блоком управления игровым состоянием, получает сигналы о начале нового игрового цикла, переходе между комнатами и необходимости формирования следующего уровня.

Блок генерации уровня формирует пространственную основу игровой системы и условия функционирования остальных компонентов.

## **2.10 Блок боевой системы**

Блок боевой системы предназначен для обработки игровых столкновений, расчета атакующих действий и изменения параметров игровых объектов в результате боевого взаимодействия. В рамках проекта данный блок обеспечивает выполнение атак игрока, регистрацию попаданий и изменение состояния игровых сущностей.

Основой работы блока является контроль взаимодействия атакующих объектов с противниками и игровым персонажем. После запуска атаки определяется зона поражения или траектория движения атакующего объекта, затем выполняется проверка пересечения с игровыми объектами.

В проекте предусмотрены два типа атакующих действий: удар ближнего действия и атака движущимся объектом. Для ближнего действия используется ограниченная зона поражения, для дальнего действия создается отдельный объект с направленным движением.

Функции блока боевой системы:

- обработка атакующих действий игрока;
- расчет зоны поражения и траектории атакующих объектов;
- проверка столкновений;
- изменение параметров здоровья;
- запуск игровых событий при попадании;
- фиксация уничтожения игровых объектов.

Связи блока с другими функциональными компонентами:

- блок управления персонажем: получает параметры атаки и направление действия;

- блок адаптивного поведения противников: передает данные о столкновениях и состоянии противников;
- блок управления игровым состоянием: сообщает о завершении боевых эпизодов и уничтожении противников.

Работа блока обеспечивает формирование боевого взаимодействия как основной части игрового цикла.

## **2.11 Блок адаптивного поведения противников**

Блок адаптивного поведения противников является центральным программным компонентом дипломного проекта и выполняет функцию выбора текущего состояния игрового противника в зависимости от параметров игровой ситуации. В отличие от стандартных схем поведения, основанных только на расстоянии до цели или факте обнаружения игрока, данный блок использует совокупность нескольких входных параметров, которые анализируются перед каждым изменением состояния.

Внутренняя логика блока построена на конечном автомате состояний, где каждому состоянию соответствует определенный тип поведения игрового объекта. В рамках проекта используются состояния ожидания, преследования, атаки, отхода и обходного перемещения. Каждое состояние задает отдельный режим движения и взаимодействия противника внутри игровой комнаты.

Состояние ожидания используется в случаях, когда противник не имеет достаточных оснований для активного взаимодействия и сохраняет исходное положение либо выполняет ограниченное движение в пределах небольшой области. При появлении игрока в зоне контроля выполняется переход к анализу расстояния и текущих игровых параметров.

Состояние преследования активируется при обнаружении игрока на допустимой дистанции и используется для сокращения расстояния до цели. В этом режиме противник изменяет свое положение в направлении игрока, учитывая препятствия и текущую геометрию комнаты.

Состояние атаки используется при достижении допустимой дистанции взаимодействия. В этом режиме выполняется запуск атакующего действия, после чего блок повторно оценивает игровую ситуацию для определения следующего состояния.

Состояние отхода применяется при снижении уровня здоровья противника, неблагоприятном положении внутри комнаты или наличии угрозы со стороны игрока. В таком режиме противник увеличивает дистанцию до цели и изменяет траекторию движения.

Состояние обходного перемещения используется для смены позиции относительно игрока без прямого приближения или отступления. Это позволяет формировать менее предсказуемую модель движения внутри комнаты.

Поверх конечного автомата в блоке реализуется дополнительный слой оценки угрозы. Перед выбором состояния рассчитывается коэффициент угрозы, учитывающий текущие параметры игровой ситуации. В расчет

включаются расстояние до игрока, используемый тип оружия, текущее состояние здоровья противника, наличие ближайших союзных объектов, плотность заполнения комнаты и информация о последних действиях игрока.

Дополнительно используется слой памяти действий игрока. Блок фиксирует недавние действия, включая запуск дальних атак, резкое сокращение дистанции и повторяющиеся действия в пределах короткого временного интервала. Полученные данные используются при последующем выборе состояния, что позволяет изменять реакцию противника в пределах одинаковых внешних условий.

Функции блока адаптивного поведения противников:

- анализ текущих параметров игровой ситуации;
- выбор состояния конечного автомата;
- выполнение переходов между состояниями;
- учет последних действий игрока;
- формирование управляющего решения для игрового противника.

Связи блока с другими функциональными компонентами:

- блок управления персонажем: получает данные о положении игрока, направлении движения и характере текущих действий;
- блок боевой системы: получает информацию о столкновениях, уровне здоровья и результатах атак;
- блок управления игровым состоянием: передает данные о завершении активности противников, изменении состояния комнаты и переходе между игровыми этапами.

Блок адаптивного поведения отделен от исполнительной логики игрового объекта и функционирует как самостоятельный вычислительный модуль, формирующий решение о поведении противника внутри общей структуры игрового приложения.

## **2.12 Блок управления игровым состоянием**

Блок управления игровым состоянием предназначен для координации основных этапов игрового цикла и контроля переходов между различными режимами работы приложения. В пределах архитектуры проекта данный блок объединяет информацию от игровых подсистем и определяет текущее состояние игровой сессии.

Основная задача блока заключается в фиксации начала игрового цикла, контроле прохождения комнаты, определении завершения боевого эпизода и запуске перехода к следующему участку уровня.

После уничтожения всех противников в пределах комнаты блок изменяет состояние текущего эпизода и разрешает переход к следующей зоне.

Функции блока управления игровым состоянием:

- контроль начала и завершения игрового цикла;
- фиксация завершения боевых эпизодов;
- управление переходами между комнатами;
- изменение режимов работы приложения;

- координация последовательности игровых событий;
  - передача управляющих сигналов другим подсистемам.
- Связи блока с другими функциональными компонентами:
- блок генерации уровня;
  - блок боевой системы;
  - блок адаптивного поведения противников;
  - блок пользовательского интерфейса;
  - блок сохранения и метапрогрессии;
  - блок отображения игры.

Работа данного блока обеспечивает согласованность между игровыми подсистемами и поддерживает единый порядок изменения состояний внутри всего игрового приложения.

### **2.13 Блок пользовательского интерфейса**

Блок пользовательского интерфейса предназначен для отображения игровой информации, необходимой пользователю в процессе прохождения, и обеспечивает визуальную связь между внутренним состоянием системы и действиями игрока.

Основной интерфейсный слой активен во время прохождения и содержит информацию о текущем уровне здоровья персонажа, накопленных игровых параметрах и доступных элементах взаимодействия.

При изменении состояния персонажа блок интерфейса обновляет соответствующие визуальные элементы без задержки, чтобы игрок получал актуальную информацию о последствиях боевых столкновений.

Функции блока пользовательского интерфейса:

- отображение текущего уровня здоровья персонажа;
- вывод игровых параметров;
- отображение экранов паузы и завершения прохождения;
- вывод информации о доступных улучшениях;
- обновление интерфейсных элементов.

Связь блока с другим функциональным компонентом – блоком управления игровым состоянием, получает данные о текущем режиме приложения, завершении игрового цикла и переходах между комнатами.

Работа блока пользовательского интерфейса обеспечивает постоянное отображение состояния игровой системы и делает внутренние изменения приложения доступными для восприятия пользователем в процессе прохождения.

### **2.14 Блок сохранения и метапрогрессии**

Блок сохранения и метапрогрессии предназначен для хранения параметров, которые должны сохраняться между отдельными игровыми циклами и использоваться при последующих прохождениях.

В рамках проекта данный блок обеспечивает долговременное

накопление результатов, включая игровые улучшения, статистику прохождений и пользовательские параметры.

Сохранение выполняется в моменты завершения прохождения, получения нового улучшения или изменения постоянных параметров персонажа.

Функции блока сохранения и метапрогрессии:

- сохранение накопленных параметров;
- загрузка данных при запуске приложения;
- хранение игровых улучшений;
- фиксация статистики прохождений;
- обновление постоянных параметров персонажа.

Связь блока с другим функциональным компонентом – блоком управления игровым состоянием, получает сигналы о завершении игрового цикла и необходимости сохранения данных.

Работа данного блока обеспечивает связь между отдельными игровыми циклами и формирует накопительный слой развития внутри общей структуры проекта.

## **2.15 Блок отображения игры**

Блок отображения игры предназначен для формирования визуального представления игровой сцены и вывода текущего состояния игровых объектов в пределах экрана пользователя.

Основой работы блока является постоянное обновление визуального состояния сцены в соответствии с положением игровых объектов. На каждом этапе игрового цикла отображаются персонаж, противники, элементы уровня и другие игровые объекты.

Визуальная часть проекта построена на использовании двумерных спрайтов, размещаемых в игровой сцене в соответствии с координатами игровых сущностей.

Функции блока отображения игры:

- отображение игрового пространства и структуры комнаты;
- вывод персонажа, противников и игровых объектов;
- обновление визуального положения сущностей;
- отображение покадровой анимации;
- запуск визуальных эффектов игровых событий.

Связь блока с другим функциональным компонентом – блоком управления игровым состоянием, получает сигналы о переходе между комнатами, завершении игровых эпизодов и изменении состояния сцены.

Блок отображения игры обеспечивает визуальное представление работы всех игровых компонентов.

## 3 ФУНКЦИОНАЛЬНОЕ ПРОЕКТИРОВАНИЕ

Раздел функционального проектирования посвящен описанию внутренней структуры программного модуля адаптивного поведения противников и основных программных компонентов игровой системы, обеспечивающих его функционирование в рамках разрабатываемого проекта. Основное внимание в разделе уделено построению иерархии классов, реализующих конечный автомат состояний, модуль оценки угрозы и слой памяти игровых действий, влияющих на выбор поведения противника в пределах текущей игровой ситуации.

Поскольку объектом разработки является не только отдельный программный модуль принятия решений, но и игровая среда, в разделе также рассматриваются программные компоненты, через которые обеспечивается взаимодействие модуля поведения с игровым персонажем, боевой системой, обработкой состояний противников и изменением параметров игрового пространства.

Отдельно описываются свойства и методы ключевых классов, отвечающих за смену состояний противника, расчет коэффициента угрозы, анализ входных параметров и передачу управляющих решений в исполнительную игровую логику.

Диаграмма классов представлена на чертеже ГУИР.400201.112 РР.1.

### 3.1 Описание структуры классов

#### 3.1.1 Класс `MusicSubsystem`

Класс `MusicSubsystem` является подсистемой игрового экземпляра, управляющей воспроизведением адаптивной музыки. Класс обеспечивает плавные переходы между треками (`hub`, `explore`, `combat`) посредством затухания и нарастания громкости. Является наследником `UGameInstanceSubsystem` и сохраняется между переходами между уровнями.

Атрибуты класса:

- `MasterVolume` – общий множитель громкости;
- `TransitionDuckMultiplier` – множитель снижения громкости во время перехода;
- `FadeInTime` – длительность нарастания громкости нового трека в секундах;
- `FadeOutTime` – длительность затухания предыдущего трека в секундах;
- `FadeProgress` – текущий прогресс перехода в диапазоне  $[0, 1]$ ;
- `bIsCrossfading` – флаг активного перехода между треками;
- `TickHandle` – дескриптор делегата тика для обработки интерполяции громкости.

Методы класса:

- `Initialize(Collection)` – инициализирует подсистему и



подключает делегат тика;

- `Deinitialize()` – останавливает воспроизведение и освобождает ресурсы;

- `PlayMusic(Track, FadeIn, FadeOut)` – запускает указанный трек с заданными параметрами перехода;

- `StopMusic(FadeOut)` – останавливает текущую музыку с затуханием;

- `GetCurrentTrack()` – возвращает идентификатор активного трека;

- `GetSoundForTrack(Track)` – возвращает аудиоресурс для указанного трека;

- `GetOrCreateComponent(Track)` – возвращает или создает аудиокомпонент для трека;

- `OnTick(DeltaTime)` – обрабатывает интерполяцию громкости на каждом шаге тика.

### 3.1.2 Класс `CombatMusicTrigger`

Класс `CombatMusicTrigger` является компонентом, прикрепленным к персонажу и управляющим автоматическим переключением музыки в зависимости от боевой активности. С интервалом 0.5 с компонент проверяет наличие живых противников в текущей комнате и инициирует переход к боевому треку или треку исследования с задержкой подавления дребезга.

Атрибуты класса:

- `CombatEndDebounce` – время задержки после окончания боя перед переходом к треку исследования в секундах;

- `PollInterval` – интервал проверки боевого состояния в секундах;

- `CombatEndTime` – момент времени, когда боевое состояние последний раз завершилось;

- `bWasInCombat` – флаг, указывающий на активное боевое состояние в предыдущем цикле;

- `EvalTimerHandle` – дескриптор таймера периодической проверки.

Методы класса:

- `BeginPlay()` – запускает таймер периодической проверки боевого состояния;

- `EndPlay(Reason)` – останавливает таймер и освобождает дескриптор;

- `EvaluateCombatState()` – определяет наличие живых противников и переключает трек с учетом задержки подавления дребезга.

### 3.1.3 Класс `DepthrunCharacter`

Класс `DepthrunCharacter` является центральным компонентом представления пользователя в игровой среде. Класс реализует обработку пользовательского ввода через систему `Enhanced Input`, управление перемещением, переключение слотов вооружения и взаимодействие с подсистемами метапрогрессии, экономики и интерфейса. Являясь наследником `APaperCharacter` и использует режим полета компонента движения для перемещения по плоскости без воздействия гравитации.

Класс выступает единым информационным узлом, через который противники получают координаты персонажа для расчета контекстных факторов в модуле оценки угрозы. Компонент `PlayerActionTracker` регистрирует действия пользователя и передает события в `PatternRecognizer` для анализа тактических паттернов.

**Атрибуты класса:**

- `MaxHP` – максимальное значение очков здоровья персонажа;
- `CurrentHP` – текущее значение очков здоровья;
- `FacingDirection` – текущее направление взгляда, значение перечисления `EPlayerFacingDirection`;
- `ActiveWeaponSlot` – индекс активного слота вооружения (1 – меч, 2 – лук);
- `WeaponSlotClass1` – класс оружия ближнего боя для создания экземпляра при инициализации;
- `WeaponSlotClass2` – класс дистанционного оружия для создания экземпляра при инициализации;
- `DashImpulse` – величина импульса рывка в единицах движка;
- `DashCooldown` – время перезарядки рывка в секундах;
- `MuzzleOffset` – смещение точки появления снаряда от центра персонажа;
- `DamageMultiplier` – множитель урона из профиля метапрогрессии;
- `MeleeRangeMultiplier` – множитель дальности удара ближнего боя из профиля;
- `BaseProjectileCount` – базовое количество стрел за выстрел из профиля;
- `bGodMode` – флаг режима отладки, при котором персонаж не получает урона;
- `bSuperAttack` – флаг режима отладки усиленной атаки.

**Методы класса:**

- `BeginPlay()` – инициализирует компоненты, применяет профиль улучшений и регистрирует входные привязки;
- `Tick(DeltaSeconds)` – обновляет анимацию и эффект хроматической аберрации при каждом кадре;
- `SetupPlayerInputComponent(PlayerInputComponent)` – регистрирует действия `Enhanced Input` для движения, атаки и переключения оружия;
- `Heal(Amount)` – восстанавливает очки здоровья, ограничивая значение максимумом;
- `ApplyProfileUpgrades()` – читает уровни улучшений из подсистемы сохранения и применяет множители;
- `SwitchToWeaponSlot(SlotIndex)` – активирует указанный слот вооружения и применяет модификаторы предметов;
- `TogglePause()` – открывает или закрывает виджет паузы и останавливает игровое время;
- `ShowVictoryScreen()` – отображает экран победы после завершения

финального уровня;

- `GetWeaponSlot(Slot)` – возвращает указатель на экземпляр оружия в заданном слоте;

- `IsDead()` – возвращает истину, если персонаж находится в состоянии поражения;

- `TakeDamage(DamageAmount)` – обрабатывает входящий урон, запускает анимацию попадания и проверяет условие поражения;

- `Die()` – фиксирует заработанные алмазы, отключает ввод и отображает экран поражения.

### 3.1.4 Класс `RunItemInventory`

Класс `RunItemInventory` является компонентом, управляющим инвентарем предметов текущего прохождения. Компонент хранит набор структур `FRunItemData` и обеспечивает применение их эффектов к оружию и характеристикам персонажа при смене слота.

Атрибуты класса:

- `MaxItems` – максимальное количество предметов, допустимых за одно прохождение;

- `Items` – массив активных предметов текущего прохождения.

Методы класса:

- `AddItem(Data)` – добавляет предмет в инвентарь; возвращает ложь, если инвентарь заполнен;

- `ClearItems()` – удаляет все предметы при начале нового прохождения;

- `ApplyToWeapon(Weapon)` – применяет эффекты предметов к указанному оружию при его активации;

- `ApplyToCharacter(Character)` – применяет статовые усилители (здоровье, скорость, количество стрел) к персонажу;

- `HasEffect(Effect)` – возвращает истину, если хотя бы один предмет с указанным эффектом присутствует в инвентаре;

- `GetItems()` – возвращает массив всех предметов инвентаря.

### 3.1.5 Класс `BaseWeapon`

Класс `BaseWeapon` является абстрактным базовым классом для всего вооружения персонажа. Класс задает единый интерфейс выстрела, хранит базовый урон и перезарядку, а также управляет таймером охлаждения. Направление выстрела передается извне через `SetFireDirection()` до вызова `Fire()`.

Атрибуты класса:

- `BaseDamage` – базовый урон за одну атаку;

- `AttackCooldown` – интервал между атаками в секундах;

- `bCanFire` – флаг готовности к следующей атаке;

- `FireDirection` – нормализованный вектор направления выстрела в мировых координатах;

- CooldownTimer – дескриптор таймера перезарядки;
- InitialBaseDamage – исходное значение урона для сброса после применения модификаторов.

Методы класса:

- Fire() – чисто виртуальный метод; реализуется в производных классах;
- GetDamage() – возвращает текущее значение базового урона;
- GetWeaponType() – чисто виртуальный метод; возвращает тип оружия;
- SetFireDirection(Direction) – устанавливает направление выстрела перед вызовом атаки;
- ResetBaseDamage() – возвращает урон к исходному значению после сброса предметных модификаторов;
- BeginPlay() – сохраняет исходное значение урона для последующего сброса;
- StartCooldown() – запускает таймер перезарядки; вызывается в конце реализации Fire();
- OnCooldownEnd() – сбрасывает флаг готовности по истечении перезарядки.

### 3.1.6 Класс **MeleeWeapon**

Класс **MeleeWeapon** реализует оружие ближнего боя персонажа. При вызове **Fire()** активирует зону поражения на 0.2 с, любой противник в зоне получает урон. Поддерживает модификаторы предметов: увеличенный радиус, двойной взмах и всенаправленный удар.

Атрибуты класса:

- HitZoneHalfExtent – дальность зоны поражения вперед в единицах движка;
- HitZoneWidth – ширина зоны поражения в единицах движка;
- HitZoneThickness – толщина зоны поражения по оси Z;
- RangeMultiplier – множитель дальности из эффекта предмета **MeleeExtendedRange**;
- WidthMultiplier – множитель ширины зоны поражения;
- ThicknessMultiplier – множитель толщины зоны поражения;
- bDoubleSwing – флаг двойного взмаха из эффекта предмета **MeleeDoubleSwing**;
- bIsSecondSwing – флаг активного второго взмаха;
- bOmniSwing – флаг всенаправленного удара;
- HitZoneTimer – дескриптор таймера активной зоны поражения;
- DoubleSwingTimer – дескриптор таймера второго взмаха.

Методы класса:

- Fire() – активирует зону поражения и при необходимости запускает второй взмах;
- GetWeaponType() – возвращает тип **Melee**;

- `ResetEffects()` – сбрасывает все предметные модификаторы к значениям по умолчанию;
- `SetRangeMultiplier(NewMultiplier)` – устанавливает множитель дальности;
- `GetRangeMultiplier()` – возвращает текущий множитель дальности;
- `SetDoubleSwing(bEnabled)` – включает или отключает двойной взмах;
- `SetWidthMultiplier(NewMultiplier)` – устанавливает множитель ширины;
- `GetWidthMultiplier()` – возвращает текущий множитель ширины;
- `SetOmniSwing(bEnabled)` – включает или отключает всенаправленный удар;
- `IsOmniSwing()` – возвращает истину, если активен режим всенаправленного удара;
- `SetThicknessMultiplier(NewMultiplier)` – устанавливает множитель толщины;
- `GetThicknessMultiplier()` – возвращает текущий множитель толщины;
- `OnHitZoneOverlap()` – применяет урон противнику при пересечении зоны поражения;
- `ActivateHitZone()` – включает коллизию зоны поражения;
- `DeactivateHitZone()` – отключает коллизию зоны поражения;
- `PerformDoubleSwing()` – выполняет второй взмах с задержкой.

### 3.1.7 Класс `RangedWeapon`

Класс `RangedWeapon` реализует дистанционное оружие персонажа. При вызове `Fire()` создает снаряд `ABaseProjectile` с задержкой, синхронизированной с анимацией. Поддерживает модификаторы рикошета, пробивания и количества выстрелов.

Атрибуты класса:

- `ProjectileSpeed` – скорость снаряда в единицах движка;
- `ShotDelay` – задержка между командой и появлением снаряда в секундах;
- `RicochetCount` – количество допустимых рикошетов снаряда;
- `bPierceEnabled` – флаг пробивания: снаряд не уничтожается при попадании в противника;
- `ShotTimer` – дескриптор таймера отложенного создания снаряда.

Методы класса:

- `Fire()` – запускает таймер задержки и по истечении создает снаряд;
- `GetWeaponType()` – возвращает тип `Ranged`;
- `ResetEffects()` – сбрасывает все предметные модификаторы к значениям по умолчанию;
- `SetRicochetCount(Count)` – устанавливает количество рикошетов;
- `GetRicochetCount()` – возвращает текущее количество рикошетов;

- `SetPierceEnabled(bEnabled)` – включает или отключает режим пробивания;
- `ActuallyFire()` – создает снаряд после истечения задержки.

### 3.1.8 Класс **BaseProjectile**

Класс `BaseProjectile` реализует снаряд, создаваемый дистанционным оружием. Снаряд перемещается вручную через `Tick` без использования компонента физического движения. Поддерживает режимы пробивания и рикошета к ближайшему противнику.

Атрибуты класса:

- `ProjectileSpeed` – скорость снаряда в единицах движка;
- `ProjectileLifeSpan` – время жизни снаряда в секундах до автоматического удаления;
- `LaunchDirection` – нормализованный вектор направления движения;
- `DamageAmount` – урон, наносимый при попадании;
- `bPierceEnabled` – флаг режима пробивания;
- `RicochetCount` – оставшееся количество рикошетов;
- `RicochetSearchRadius` – радиус поиска ближайшей цели для рикошета;
- `OverlapCheckFrameCounter` – счетчик кадров для оптимизации частоты проверки пересечений.

Методы класса:

- `InitProjectile(Direction, Damage, Shooter, InSpeed, bPierce, InRicochetCount)` – инициализирует параметры снаряда сразу после создания;
- `BeginPlay()` – устанавливает время жизни и активирует коллизию;
- `Tick(DeltaTime)` – перемещает снаряд по направлению движения;
- `OnOverlap(OverlappedComp, OtherActor)` – наносит урон цели и уничтожает снаряд или запускает рикошет;
- `TryRicochet(JustHitEnemy)` – перенаправляет снаряд к ближайшему противнику и уменьшает счетчик рикошетов.

### 3.1.9 Класс **BaseEnemy**

Класс `BaseEnemy` определяет базовую архитектуру для всех типов противников. Класс инкапсулирует механизмы управления жизненным циклом, перемещением и входящим уроном. Является наследником `APaperCharacter` и точкой подключения компонентов `UFSMComponent` и `UEnemyHealthComponent`.

Единая база устраняет дублирование логики обработки столкновений и предоставляет настраиваемые параметры для всех производных классов: дистанции обнаружения, атаки, урона, скорости и характеристик снарядов.

Атрибуты класса:

- `EnemyType` – тип противника, значение перечисления `EEnemyType(Melee, Ranged, Adaptive)`;

- MoveSpeed – базовая скорость передвижения в единицах движка;
- DetectionRange – дистанция обнаружения игрока, при достижении которой противник начинает преследование;
- MeleeAttackRange – дистанция перехода в состояние атаки для режима ближнего боя;
- RangedAttackRange – оптимальная дистанция ведения дистанционного боя;
- RangedTooFarRange – предельная дистанция дистанционной атаки, при превышении которой противник возобновляет преследование;
- MinAttackRange – минимальная дистанция до игрока для перехода в состояние отступления;
- SafeDistance – дистанция завершения отступления;
- AttackDamage – величина урона за одну атаку ближнего боя;
- AttackCooldown – интервал между последовательными атаками в секундах;
- ProjectileSpeed – скорость снаряда в единицах движка;
- ShotDelay – задержка между командой на выстрел и появлением снаряда;
- FireRate – частота атак в секунду для дистанционных противников;
- bIsDead – флаг, указывающий на то, что противник выведен из строя;
- bIsHitAnimationActive – флаг активной анимации попадания.

#### Методы класса:

- BeginPlay() – инициализирует компонент движения, подсистему здоровья и конечный автомат;
- Tick(DeltaTime) – вызывает обновление анимации на каждом кадре;
- OnDeath() – обрабатывает обнуление здоровья, воспроизводит анимацию и планирует удаление;
- UpdateAnimation() – выбирает анимационный кадр в зависимости от состояния и скорости;
- PerformMeleeAttack() – применяет урон ближнего боя к игроку при нахождении в зоне досягаемости;
- TakeDamage(DamageAmount) – передает входящий урон компоненту здоровья и активирует анимацию попадания;
- OnSpawned() – вызывается при появлении противника в игровом мире;
- OnKilled() – воспроизводит визуальный эффект и инициирует удаление объекта;
- GetEnemyType() – возвращает тип противника;
- IsDead() – возвращает истину, если противник выведен из строя;
- GetEffectiveAttackRange() – возвращает актуальную дистанцию атаки в зависимости от режима боя;
- GetSeparationSteering() – вычисляет вектор расталкивания для предотвращения наложения противников;
- SetLockedZ(InZ) – фиксирует противника на заданной высоте

плоскости после перемещения в комнату.

### 3.1.10 Класс **MeleeEnemy**

Класс `MeleeEnemy` является конкретной реализацией противника ближнего боя, унаследованной от `BaseEnemy`. Класс не добавляет новых полей и методов: все поведение определяется значениями параметров, задаваемыми в ресурсе `Blueprint`. Конструктор устанавливает тип `EnemyType::Melee` и конфигурирует компонент движения для корректной работы без контроллера.

Методы класса:

- `BeginPlay()` – вызывает родительскую инициализацию и устанавливает тип противника.

### 3.1.11 Класс **RangedEnemy**

Класс `RangedEnemy` является конкретной реализацией дистанционного противника. Переопределяет `PerformMeleeAttack()` для создания снаряда в направлении игрока под любым углом в  $360^\circ$ . Параметры дальности и темпа стрельбы задаются в `Blueprint`.

Атрибуты класса:

- `MuzzleOffset` – смещение точки появления снаряда от центра противника в единицах движка.

Методы класса:

- `BeginPlay()` – вызывает родительскую инициализацию и устанавливает тип `Ranged`;
- `PerformMeleeAttack()` – запускает задержанное создание снаряда в направлении игрока;
- `ActuallyFire()` – создает снаряд после истечения задержки.

### 3.1.12 Класс **AdaptiveEnemy**

Класс `AdaptiveEnemy` является главным демонстрационным противником дипломного проекта. Класс не содержит логики поведения: все решения принимаются компонентом `UAdaptiveBehaviorComponent`, а исполнение состояний реализовано в объектах `UFSMState` через `UFSMComponent`.

В зависимости от флага `bIsRangedMode` противник динамически переключается между ближним и дальним режимами боя, изменяя анимацию и способ атаки. Компонент `UAdaptiveBehaviorComponent` подписывается на делегат изменения здоровья для получения сигнала вознаграждения.

Атрибуты класса:

- `bIsRangedMode` – флаг текущего режима боя: истина означает дистанционный режим;
- `MuzzleOffset` – смещение точки появления снаряда в единицах движка.

Методы класса:

- `BeginPlay()` – регистрирует пять состояний конечного автомата,



переходит в состояние ожидания и подписывается на события;

- `OnKilled()` – воспроизводит эффект в зависимости от текущего режима боя;

- `PerformMeleeAttack()` – в дистанционном режиме запускает задержанный выстрел; в ближнем – применяет урон;

- `ActuallyFire()` – создает снаряд после истечения задержки в дистанционном режиме;

- `UpdateAnimation()` – выбирает анимационный набор в зависимости от режима боя и состояния конечного автомата;

- `HandleHealthChanged(OldHP, NewHP, MaxHP)` – отправляет сигнал вознаграждения в компонент адаптивного поведения.

### 3.1.13 Класс `DepthrunSaveSubsystem`

Класс `DepthrunSaveSubsystem` является подсистемой игрового экземпляра, обеспечивающей персистентное хранение данных между прохождениями через базу данных SQLite. Подсистема управляет тремя таблицами: история прохождений, профиль игрока с метапрогрессией и снимки весов адаптивного модуля для построения дипломных графиков.

Методы класса:

- `Initialize(Collection)` – открывает базу данных и создает таблицы при первом запуске;

- `Deinitialize()` – закрывает соединение с базой данных;

- `SaveRunResult(Floor, Score, bWon)` – записывает результат завершенного прохождения;

- `LoadProgress(OutBestFloor, OutTotalRuns)` – загружает лучший этаж и общее число прохождений;

- `SaveAdaptiveWeights(Weights, RunNumber)` – сохраняет снимок весов для построения графика эволюции;

- `AddDiamondsToProfile(Amount)` – добавляет алмазы к постоянному профилю игрока;

- `GetUpgradeLevel(Type)` – возвращает текущий уровень улучшения (0–5);

- `GetUpgradeCost(Type)` – возвращает стоимость следующего уровня улучшения;

- `BuyUpgrade(Type)` – списывает алмазы и повышает уровень улучшения;

- `ResetProfile()` – сбрасывает все данные профиля для отладки;

- `GetDB()` – возвращает указатель на менеджер базы данных для консольных команд;

- `InitializeSchema()` – создает таблицы SQLite при первом запуске.

### 3.1.14 Класс `SQLiteManager`

Класс `SQLiteManager` является тонкой оберткой над встроенным модулем `SQLiteCore` движка. Класс предоставляет единый интерфейс для

открытия базы данных, выполнения SQL-запросов, вставки строк и выборки данных по условию.

Атрибуты класса:

- `bIsOpen` – флаг открытого соединения с базой данных;
- `SQLiteDB` – указатель на структуру базы данных `sqlite3`.

Методы класса:

- `OpenDatabase(Path)` – открывает или создает файл базы данных по указанному пути;
- `CloseDatabase()` – закрывает соединение с базой данных;
- `ExecuteQuery(SQL)` – выполняет произвольный SQL-запрос; возвращает истину при успехе;
- `InsertRow(Table, Columns)` – вставляет строку с указанными столбцами; возвращает идентификатор строки;
- `SelectRows(Table, WhereClause)` – возвращает массив строк, соответствующих условию выборки;
- `IsOpen()` – возвращает истину, если соединение с базой данных открыто.

### 3.1.15 Класс `FSMComponent`

Класс `FSMComponent` является компонентом-менеджером конечного автомата состояний противника. Компонент хранит реестр объектов состояний, активирует текущее состояние через тик и выполняет переходы по команде извне. Компонент не принимает решений о смене состояний: эту роль выполняет `UAdaptiveBehaviorComponent` или логика самого состояния для простых противников.

Атрибуты класса:

- `CurrentStateType` – текущий тип активного состояния, значение перечисления `EFSMStateType`;
- `States` – реестр объектов состояний, ключ – тип состояния;
- `CurrentState` – указатель на активный объект состояния;
- `OwnerEnemy` – указатель на владельца-противника, передается в методы состояний.

Методы класса:

- `BeginPlay()` – инициализирует компонент и кэширует указатель на владельца;
- `TickComponent(DeltaTime)` – вызывает метод тика активного состояния;
- `RegisterState(StateType, StateObject)` – регистрирует объект состояния в реестре;
- `TransitionTo(NewState)` – вызывает выход из текущего и вход в новое состояние;
- `GetCurrentStateType()` – возвращает тип текущего состояния;
- `GetTimeInCurrentState()` – возвращает время в секундах, прошедшее с момента последнего перехода;

– `GetStateName(State)` – возвращает строковое название состояния для журналирования.

### 3.1.16 Класс `FSMState`

Класс `FSMState` является абстрактным базовым классом для всех пяти состояний конечного автомата. Класс задает единую точку входа, тика и выхода из состояния. Логика решений о переходах в класс не включена: состояния реализуют только движение и действие.

Атрибуты класса:

– `TimeInState` – накопленное время нахождения в состоянии с момента последнего входа в секундах.

Методы класса:

– `EnterState(Owner)` – вызывается при переходе в состояние; сбрасывает таймеры и внутренние переменные;

– `TickState(Owner, DeltaTime)` – вызывается каждый кадр; реализует движение и действия;

– `ExitState(Owner)` – вызывается при выходе из состояния; выполняет очистку;

– `GetStateType()` – чисто виртуальный метод; возвращает тип состояния;

– `GetTimeInState()` – возвращает накопленное время в состоянии.

### 3.1.17 Класс `FSMState_Idle`

Класс `FSMState_Idle` реализует состояние ожидания: противник стоит на месте. При обнаружении игрока в пределах дистанции детекции инициирует переход в состояние преследования или атаки.

Атрибуты класса:

– `TimeSinceLastLook` – время с момента последней анимации осмотра окружения в секундах.

Методы класса:

– `EnterState(Owner)` – сбрасывает счетчик осмотра;

– `TickState(Owner, DeltaTime)` – проверяет дистанцию до игрока и при необходимости запрашивает переход;

– `ExitState(Owner)` – выполняет очистку при выходе из состояния;

– `GetStateType()` – возвращает значение `Idle`.

### 3.1.18 Класс `FSMState_Attack`

Класс `FSMState_Attack` реализует состояние атаки. При выходе игрока за пределы дистанции атаки, противник после некоторого времени ожидания переходит в состояние преследования.

Атрибуты класса:

– `AttackCooldown` – интервал между атаками в секундах;

– `TimeSinceLastAttack` – время с момента последней атаки в секундах;

– `bAttackCoolingDown` – флаг активной перезарядки;

– `TimeInAttackState` – время пребывания в состоянии атаки с момента входа в секундах.

Методы класса:

– `EnterState(Owner)` – сбрасывает счетчики перезарядки и времени пребывания;

– `TickState(Owner, DeltaTime)` – выполняет атаку по таймеру и проверяет условие выхода;

– `ExitState(Owner)` – выполняет очистку при выходе из состояния;

– `GetStateType()` – возвращает значение `Attack`.

### 3.1.19 Класс `FSMState_Chase`

Класс `FSMState_Chase` реализует состояние преследования: противник движется к игроку с учетом разделяющего воздействия. При достижении дистанции атаки инициирует переход в состояние атаки.

Атрибуты класса:

– `AttackRangeThreshold` – дистанция в единицах движка, при достижении которой запрашивается переход в состояние атаки.

Методы класса:

– `EnterState(Owner)` – выполняет подготовку к преследованию;

– `TickState(Owner, DeltaTime)` – направляет движение к игроку и проверяет условие перехода;

– `ExitState(Owner)` – выполняет очистку при выходе из состояния;

– `GetStateType()` – возвращает значение `Chase`.

### 3.1.20 Класс `FSMState_Retreat`

Класс `FSMState_Retreat` реализует состояние отступления: противник перемещается в направлении, противоположном игроку. Дистанционные противники в режиме отступления могут продолжать вести огонь со сниженной частотой.

Атрибуты класса:

– `RetreatDirection` – кэшированный вектор отступления, вычисляется при входе в состояние;

– `TimeSinceLastShot` – время с момента последнего выстрела в секундах.

Методы класса:

– `EnterState(Owner)` – вычисляет и кэширует вектор отступления;

– `TickState(Owner, DeltaTime)` – перемещает противника по кэшированному вектору и при достижении безопасной дистанции инициирует переход;

– `ExitState(Owner)` – выполняет очистку при выходе из состояния;

– `GetStateType()` – возвращает значение `Retreat`.

### 3.1.21 Класс `FSMState_Flank`

Класс `FSMState_Flank` реализует состояние флангового обхода:

противник движется под углом к вектору игрок–противник, пытаясь зайти с фланга или тыла. Направление пересчитывается с заданным интервалом.

Атрибуты класса:

- `F flankDirectionRefreshInterval` – интервал обновления направления движения в секундах;
- `F flankDirection` – текущий вектор флангового движения;
- `F timeSinceDirectionRefresh` – время с момента последнего пересчета направления в секундах;
- `F flankSide` – сторона обхода: +1 – правый фланг, –1 – левый.

Методы класса:

- `EnterState (Owner)` – случайно выбирает сторону и вычисляет начальное направление;
- `TickState (Owner, DeltaTime)` – перемещает противника и периодически пересчитывает направление;
- `ExitState (Owner)` – выполняет очистку при выходе из состояния;
- `GetStateType ()` – возвращает значение `F flank`;
- `RecalculateFlankDirection (Owner)` – вычисляет новый вектор флангового движения относительно текущей позиции игрока.

### 3.1.22 Класс `RoomGeneratorSubsystem`

Класс `RoomGeneratorSubsystem` является подсистемой мира, управляющей процедурной генерацией и активацией комнат подземелья. Подсистема хранит пул шаблонов комнат, генерирует цепочку из заданного числа комнат и переключает активную комнату при входе игрока в переходную зону.

Атрибуты класса:

- `C currentRoomIndex` – индекс текущей активной комнаты в массиве сгенерированных комнат.

Методы класса:

- `Initialize (Collection)` – инициализирует подсистему;
- `Deinitialize ()` – выполняет очистку ресурсов;
- `GenerateRooms (RoomCount)` – создает цепочку комнат из пула шаблонов;
- `OnPlayerEnteredRoom (Room)` – обрабатывает вход игрока в комнату;
- `SetTemplates (Start, Boss, Combat, Treasure, Rest)` – задает наборы шаблонов для каждого типа комнаты;
- `OnPlayerEnteredTransition (FromRoom, ExitIndex)` – обрабатывает переход игрока через выход;
- `GetRooms ()` – возвращает массив всех сгенерированных комнат;
- `GetCurrentRoomIndex ()` – возвращает индекс текущей активной комнаты;
- `GetTotalRooms ()` – возвращает общее количество боевых комнат без стартовой;
- `GetClearedRoomsCount ()` – возвращает число очищенных комнат;

- `NotifyRoomCleared()` – обновляет интерфейс при очистке комнаты;
- `GetCurrentActiveRoom()` – возвращает указатель на текущую активную комнату;
- `ActivateRoom(Index)` – деактивирует предыдущую и активирует комнату с указанным индексом.

### 3.1.23 Класс `RoomBase`

Класс `RoomBase` является контейнерным объектом одной процедурно сгенерированной комнаты подземелья. Класс управляет созданием противников, сундуков и декоративных объектов по шаблону, а также обрабатывает вход игрока и периодически проверяет статус противников для фиксации очистки комнаты.

Атрибуты класса:

- `bIsCleared` – флаг завершения боя в комнате;
- `bHasSpawnedEnemies` – флаг, указывающий на то, что противники уже были созданы;
- `bHasGeneratedChest` – флаг, указывающий на то, что сундук уже был сгенерирован;
- `bIsActive` – флаг активного состояния комнаты;
- `bPendingSetup` – флаг, указывающий на то, что настройка была вызвана до инициализации;
- `bPendingHasTop` – флаг наличия верхнего выхода;
- `bPendingHasBottom` – флаг наличия нижнего выхода;
- `bPendingHasLeft` – флаг наличия левого выхода;
- `bPendingHasRight` – флаг наличия правого выхода;
- `OccupiedTiles` – набор занятых клеток для исключения наложения декора.

Методы класса:

- `BeginPlay()` – применяет отложенную настройку, если шаблон был назначен до старта игры;
- `SetupRoom(Template, bHasTop, bHasBottom, bHasLeft, bHasRight)` – инициализирует клетки, двери и параметры комнаты по шаблону;
- `ActivateRoom()` – активирует комнату, создает противников и объекты;
- `DeactivateRoom()` – деактивирует комнату;
- `IsCleared()` – возвращает истину, если все противники выведены из строя;
- `IsCombatActive()` – возвращает истину, если комната активна и имеет живых противников;
- `SetIsActive(bActive)` – устанавливает флаг активности комнаты;
- `OnRoomEntry(OverlappedComp, OtherActor)` – обрабатывает вход игрока в коллизионную зону комнаты;
- `CheckEnemiesStatus()` – периодически проверяет наличие живых

противников;

- `SpawnEnemies()` – создает противников из списка шаблона;
- `GenerateProps(bHasTop, bHasBottom, bHasLeft, bHasRight)` – размещает декоративные объекты согласно шаблону;
- `TrySpawnChest()` – создает сундук с вероятностью из шаблона;
- `SetTileInLayer(Layer, X, Y, TileInfo)` – устанавливает тайл в указанном слое тайлмапа;
- `BuildWallColliders(TileSize, ...)` – создает коллайдеры стен для блокировки движения.

### 3.1.24 Класс `DoorActor`

Класс `DoorActor` представляет дверной объект в комнате подземелья. Класс управляет коллизией и визуальным спрайтом двери, обеспечивая открытие при очистке комнаты и закрытие при входе в нее.

Атрибуты класса:

- `bIsOpen` – флаг открытого состояния двери.

Методы класса:

- `InitializeDoor(DoorSprite, bVerticalDoor, SpriteRotation, VisualScale)` – настраивает спрайт и коллизию двери при создании;
- `OpenDoor()` – снимает коллизию и переключает спрайт на открытое состояние;
- `CloseDoor()` – включает коллизию и возвращает спрайт закрытой двери;
- `SetDoorCollisionEnabled(bEnabled)` – включает или отключает компонент коллизии двери.

### 3.1.25 Класс `ChestActor`

Класс `ChestActor` реализует сундук с наградой. При входе персонажа в зону перекрытия сундук выдает алмазы, зелья и предмет согласно конфигурационному ресурсу `UChestLootConfig`. Открытие допускается только один раз; задержка активации исключает мгновенное открытие.

Атрибуты класса:

- `bOpened` – флаг однократного открытия сундука;
- `ActivationDelay` – задержка в секундах перед активацией коллизии перекрытия;

- `TimerHandle_Activate` – дескриптор таймера отложенной активации.

Методы класса:

- `BeginPlay()` – запускает таймер отложенной активации коллизии;
- `OnChestOverlap(OverlappedComponent, OtherActor)` – вызывается при перекрытии игроком и инициирует распределение наград;
- `EnableOverlap()` – включает коллизию перекрытия по истечении задержки;
- `DistributeLoot(Player)` – вычисляет и выдает награды персонажу, транслирует событие в интерфейс.

### 3.1.26 Класс `AdaptiveBehaviorComponent`

Класс `AdaptiveBehaviorComponent` является центральным оркестратором трехуровневой системы принятия решений. С интервалом, заданным в конфигурации, компонент последовательно вызывает: слой 1 – `ContextEvaluator` для сбора и нормализации входных данных; слой 2 – `ThreatCalculator` для вычисления коэффициента угрозы; слой 3 – `StateTransitionResolver` для выбора оптимального состояния.

При отключенном флаге `bAdaptiveEnabled` компонент продолжает собирать данные, но не передает команды переходов в `FSMComponent`.

Атрибуты класса:

- `bAdaptiveEnabled` – флаг активности слоя принятия решений;
- `BraveryLevel` – характеристика храбрости, влияет на реакцию на высокую угрозу;
- `CombatStyle` – предпочтительный стиль боя, влияет на переключение режимов;
- `EvaluationTimerHandle` – дескриптор таймера периодической оценки;
- `LastRetreatExitTime` – момент последнего выхода из состояния отступления в секундах;
- `RetreatHysteresisDuration` – минимальная пауза перед повторным входом в состояние отступления;
- `ContextEval` – указатель на подсистему оценки контекста (слой 1);
- `ThreatCalc` – указатель на калькулятор угрозы (слой 2);
- `Resolver` – указатель на решатель переходов состояний (слой 3);
- `Memory` – указатель на модуль памяти игровых действий;
- `PatternRecog` – указатель на распознаватель паттернов;
- `WeightManager` – указатель на менеджер динамических весов;
- `UtilCurves` – указатель на кривые полезности;
- `CostMatrix` – указатель на матрицу стоимости переходов;
- `FSMComp` – кэшированный указатель на компонент конечного автомата владельца.

Методы класса:

- `BeginPlay()` – создает подсистемы, подписывается на события и запускает таймер оценки;
- `EndPlay(EndPlayReason)` – останавливает таймер оценки;
- `GetThreatFinal()` – возвращает последний вычисленный коэффициент угрозы;
- `GetConfidence()` – возвращает последний коэффициент достоверности;
- `GetCurrentWeights()` – возвращает текущий массив весов для отладочного виджета;
- `GetRecognizedPattern()` – возвращает строку доминирующего паттерна;
- `GetLastStateScores()` – возвращает массив оценок состояний



последнего цикла;

- `GetLastThreatAssessment()` – возвращает полную структуру результатов последней оценки угрозы;
- `OnDamageDealt()` – передает сигнал вознаграждения +1 в менеджер весов;
- `OnDamageTaken()` – передает сигнал вознаграждения –1 в менеджер весов;
- `HandlePlayerAction(ActionEvent)` – добавляет событие в память и обновляет паттерны;
- `EvaluationTick()` – выполняет один полный цикл оценки контекста, угрозы и выбора состояния;
- `OnOwnerDeath()` – останавливает таймер оценки при поражении владельца;
- `InitializeSubsystems()` – создает и инициализирует все подсистемы модуля.

### 3.1.27 Класс `ThreatCalculator`

Класс `ThreatCalculator` реализует второй слой конвейера адаптивного поведения. На каждом цикле оценки класс принимает нормализованные факторы контекста и соответствующие им веса, вычисляет их взвешенную сумму и возвращает итоговый коэффициент угрозы в диапазоне от нуля до единицы. Полученное значение отражает общую опасность текущей игровой ситуации и передается в третий слой для выбора состояния поведения.

Атрибуты класса:

- `TRawHistory` – история сырых значений коэффициента угрозы для статистики;
- `TFinalHistory` – история итоговых значений для расчета адаптивных порогов;
- `DefaultWeights` – резервные веса, используемые при отсутствии менеджера весов;
- `LastTSmooth` – последнее сглаженное значение для экспоненциального сглаживания;
- `LastAssessment` – структура результатов последней оценки для отладочного виджета.

Методы класса:

- `CalculateThreat(Context, Weights, Config)` – выполняет вычисление и возвращает структуру `FThreatAssessment`;
- `GetLastThreatFinal()` – возвращает  $T_{final}$  последнего цикла;
- `GetLastConfidence()` – возвращает последний коэффициент достоверности;
- `GetHighThreatThreshold()` – возвращает верхний адаптивный порог угрозы;
- `GetLowThreatThreshold()` – возвращает нижний адаптивный порог угрозы;

- `ComputeTBase(Ctx, Weights)` – вычисляет базовую взвешенную сумму;
- `ComputeTCross(Ctx, Weights, Cfg)` – вычисляет перекрестные члены (зарезервировано для расширения);
- `ComputeConfidence()` – вычисляет коэффициент достоверности на основе дисперсии истории.

### 3.1.28 Класс `StateTransitionResolver`

Класс `StateTransitionResolver` реализует третий слой конвейера принятия решений. На основе текущего значения коэффициента угрозы, активного состояния и распознанного паттерна поведения игрока класс вычисляет оценку полезности для каждого кандидата-состояния с учетом стоимости перехода и инерции. Состояние с наибольшей итоговой оценкой передается в `FSMComponent` для выполнения перехода.

Методы класса:

- `ResolveNextState()` – вычисляет оценки всех состояний и возвращает тип оптимального состояния;
- `ScoreState(Candidate, Current, Utility, Cost, Inertia, Pattern)` – вычисляет итоговую оценку одного кандидата-состояния.

### 3.1.29 Класс `AdaptiveMemory`

Класс `AdaptiveMemory` хранит кольцевой буфер событий действий игрока и вычисляет три агрегированные метрики: агрессивность, подвижность и осторожность. Метрика вычисляется как количество соответствующих действий в скользящем временном окне, нормированное на его длину.

Атрибуты класса:

- `MemoryBuffer` – кольцевой буфер событий действий игрока;
- `MaxBufferSize` – максимальный размер буфера событий.

Методы класса:

- `RecordEvent(Event)` – добавляет событие в буфер; вытесняет старейшее при заполнении;
- `GetDecayedAggressiveness(CurrentTime, Lambda)` – возвращает нормированный счетчик агрессивных действий в окне;
- `GetDecayedMobility(CurrentTime, Lambda)` – возвращает нормированный счетчик рывков в окне;
- `GetDecayedCaution(CurrentTime, Lambda)` – возвращает нормированный счетчик осторожных действий в окне;
- `CleanupOldEvents(CurrentTime, MaxAge)` – удаляет события старше заданного возраста;
- `GetBuffer()` – возвращает доступ только для чтения к буферу событий;
- `Initialize(Config)` – задает размер временного окна из конфигурационного ресурса;
- `ComputeDecayedMetric(CurrentTime, Lambda,`

Filter) – подсчитывает нормированное количество событий, соответствующих фильтру.

### 3.1.30 Класс `PatternRecognizer`

Класс `PatternRecognizer` выполняет N-граммный частотный анализ последних действий игрока. Скользящее окно хранит N=15 последних действий; строятся таблицы частот биграмм и триграмм. Доминирующий паттерн сопоставляется с таблицей правил, возвращая модификатор оценки для каждого кандидата-состояния.

Атрибуты класса:

- `ActionWindow` – скользящее окно последних действий игрока;
- `Unigrams` – таблица частот отдельных действий;
- `Bigrams` – таблица частот последовательностей из двух действий;
- `Trigrams` – таблица частот последовательностей из трех действий;
- `WindowSize` – максимальный размер скользящего окна.

Методы класса:

- `AddAction(ActionType)` – добавляет действие в окно и перестраивает N-граммы;
- `GetDominantPattern()` – возвращает строку наиболее частого N-граммного паттерна;
- `GetPatternModifier(CandidateState)` – возвращает модификатор оценки состояния на основе доминирующего паттерна;
- `Reset()` – очищает окно и таблицы частот;
- `GetWindowSize()` – возвращает максимальный размер окна;
- `RebuildNgrams()` – перестраивает все таблицы частот по текущему содержимому окна;
- `ActionToString(T)` – преобразует тип действия в строку для ключа таблицы.

### 3.1.31 Класс `DynamicWeightManager`

Класс `DynamicWeightManager` поддерживает и обновляет шесть весов, соответствующих факторам формулы коэффициента угрозы. После каждого игрового события компонент получает оценку результата: положительную при нанесении урона противнику и отрицательную при получении урона персонажем. На основе этой оценки и относительного вклада каждого фактора в текущее значение коэффициента угрозы веса корректируются в пределах допустимого диапазона, постепенно адаптируя поведение противника к тактике игрока.

Атрибуты класса:

- `Weights` – текущий массив из шести весов, индексированных по факторам: дистанция, угроза оружия, здоровье, союзники, плотность комнаты, память.

Методы класса:

- `ResetToDefaults(Config)` – инициализирует веса из значений

конфигурации по умолчанию;

- `UpdateWeights(Reward, Context, Config)` – выполняет один шаг обновления весов по правилу обратной связи;
- `GetWeights()` – возвращает доступ только для чтения к текущим весам.

### 3.1.32 Класс `ContextEvaluator`

Класс `ContextEvaluator` реализует первый слой конвейера принятия решений. Класс собирает входные данные из игрового мира – дистанцию до игрока, запас здоровья противника, тип оружия игрока, количество союзников и плотность комнаты – и приводит каждый из них к нормализованному значению в диапазоне от нуля до единицы. Результат передается во второй слой без каких-либо решений о поведении.

Методы класса:

- `EvaluateContext(Owner, Player, Config)` – возвращает полностью нормализованную структуру `FContextData`;
- `EvaluateContextWithMemory(Owner, Player, Memory, Config)` – расширенная версия, дополнительно вычисляет метрики агрессивности и подвижности из памяти;
- `NormalizeDistance(RawDistance, MaxRange)` – нормирует дистанцию до игрока;
- `NormalizeHealth(HPRatio, Exponent)` – применяет степенное преобразование к нормированному запасу здоровья;
- `EvaluateRoomDensity(Owner, MaxCapacity)` – подсчитывает количество сущностей в комнате и нормирует;
- `CountNearbyAllies(Owner, Radius)` – подсчитывает союзных противников в заданном радиусе;
- `GetWeaponThreatNorm(WeaponType)` – возвращает нормированное значение угрозы для типа оружия игрока.

### 3.1.33 Класс `TransitionCostMatrix`

Класс `TransitionCostMatrix` хранит матрицу стоимости переходов  $5 \times 5$ , штрафующую нежелательные быстрые смены состояний. Стоимость перехода вычитается из оценки кандидата в `StateTransitionResolver`. Инерция добавляет бонус за сохранение текущего состояния пропорционально времени пребывания.

Атрибуты класса:

- `Matrix` – матрица  $6 \times 6$ , хранящая стоимости переходов;
- `bInitialized` – флаг успешной инициализации матрицы.

Методы класса:

- `InitializeFromConfig(Config)` – заполняет матрицу из конфигурационного ресурса;
- `GetCost(From, To)` – возвращает стоимость перехода между двумя состояниями;

– CalculateInertia (Current, Candidate, TimeInState, Config) – вычисляет бонус инерции для сохранения текущего состояния;  
– StateToIndex (State) – преобразует тип состояния в индекс матрицы.

### 3.1.34 Класс AdaptiveConfig

Класс AdaptiveConfig является ресурсом данных, содержащим все настраиваемые параметры модуля адаптивного поведения. Класс наследуется от UDataAsset, что позволяет изменять параметры в редакторе без перекомпиляции. Включает параметры нормализации контекста, начальные веса, скорость обучения, размер памяти, параметры кривых полезности, матрицу стоимости переходов и интервал оценки.

Атрибуты класса:

– MaxEngagementRange – максимальная дистанция взаимодействия, при превышении которой  $D_{\text{norm}} = 0$ ;  
– AllyCheckRadius – радиус проверки наличия союзников в единицах движка;  
– MaxAllyThreshold – количество союзников, при котором  $A_{\text{norm}} = 1$ ;  
– MaxRoomCapacity – количество сущностей в комнате, при котором  $R_{\text{norm}} = 1$ ;  
– HealthPowerExponent – показатель степени  $\alpha$  в формуле нормирования здоровья;  
– WeightDistance – начальный вес фактора дистанции  $w_0$ ;  
– WeightWeaponThreat – начальный вес фактора угрозы оружия  $w_1$ ;  
– WeightHealth – начальный вес фактора здоровья  $w_2$ ;  
– WeightAllies – начальный вес фактора союзников  $w_3$ ;  
– WeightRoomDensity – начальный вес фактора плотности комнаты  $w_4$ ;  
– WeightMemory – начальный вес фактора памяти действий  $w_5$ ;  
– WeightLearningRate – скорость обучения  $\eta$  для обновления весов;  
– MemoryWindowSeconds – ширина временного окна для подсчета действий в секундах;  
– PatternWindowSize – размер скользящего окна N-граммного анализа;  
– InertiaGrowthRate – скорость нарастания инерции за секунду;  
– InertiaMax – максимальный бонус инерции;  
– TransitionCostMatrix – плоский массив коэффициентов матрицы стоимости переходов;  
– EvaluationInterval – интервал между циклами оценки в секундах.

Методы класса:

– AdaptiveConfig() – инициализирует значения по умолчанию и заполняет матрицу переходов.

### 3.1.35 Класс PlayerEconomy

Класс PlayerEconomy является компонентом, отслеживающим валюту текущего прохождения (алмазы) и расходные предметы (зелья здоровья).

Компонент транслирует делегаты при изменении значений для обновления интерфейса и управляет перезарядкой использования зелья.

Атрибуты класса:

- RunDiamonds – количество алмазов, собранных за текущее прохождение;

- HealthPotions – количество зелий здоровья в наличии;

- OnDiamondsChanged – делегат, транслируемый при изменении количества алмазов;

- OnPotionsChanged – делегат, транслируемый при изменении количества зелий;

- LastPotionUseTime – момент последнего использования зелья для проверки перезарядки.

Методы класса:

- AddDiamonds (Amount) – добавляет алмазы и транслирует делегат;

- AddPotions (Amount) – добавляет зелья и транслирует делегат;

- UsePotion () – использует зелье при его наличии и отсутствии перезарядки;

- OnPlayerDeath () – конвертирует 50% алмазов в профиль и очищает инвентарь прохождения;

- OnRunExitToHub () – конвертирует 100% алмазов в профиль и очищает инвентарь прохождения;

- ClearRunEconomy () – сбрасывает алмазы и зелья;

- IsPotionOnCooldown () – возвращает истину, если перезарядка зелья еще не истекла;

- GetSaveSubsystem () – возвращает указатель на подсистему сохранения.

### 3.1.36 Класс PlayerMovementConfig

Класс PlayerMovementConfig является ресурсом данных, хранящим все параметры движения и рывка персонажа. Наследуется от UDataAsset, что позволяет настраивать значения в редакторе без перекомпиляции. Используется компонентом движения персонажа при инициализации.

Атрибуты класса:

- MaxWalkSpeed – максимальная скорость перемещения в единицах движка;

- MaxAcceleration – максимальное ускорение в единицах движка;

- BrakingDecelerationWalking – замедление при прекращении движения;

- GroundFriction – коэффициент трения о поверхность;

- DashImpulse – величина импульса рывка в единицах движка;

- DashDuration – длительность рывка в секундах;

- DashCooldown – интервал перезарядки рывка в секундах;

- PostDashVelocityCancelFactor – множитель гашения остаточной скорости после рывка;

– InputRestoreDelay – задержка восстановления ввода после окончания рывка в секундах.

Методы класса:

### 3.1.37 Класс **EnemyHealthComponent**

Класс **EnemyHealthComponent** является компонентом управления здоровьем противника. Компонент транслирует делегат изменения здоровья, на который подписывается **AdaptiveBehaviorComponent** для получения сигналов вознаграждения, и делегат поражения, на который подписывается логика тела противника.

Атрибуты класса:

- MaxHP – максимальное значение очков здоровья;
- CurrentHP – текущее значение очков здоровья;
- OnHealthChanged – делегат, транслируемый при изменении здоровья с передачей старого, нового и максимального значений;
- OnDeath – делегат, транслируемый при обнулении здоровья.

Методы класса:

- ApplyDamage (Amount) – уменьшает здоровье на указанную величину и транслирует делегаты;
- Heal (Amount) – восстанавливает здоровье, ограничивая максимумом;
- GetCurrentHP () – возвращает текущее значение здоровья;
- GetMaxHP () – возвращает максимальное значение здоровья;
- IsDead () – возвращает истину при обнулении здоровья;
- BeginPlay () – подписывается на системный делегат получения урона;
- HandleTakeAnyDamage () – обрабатывает системный делегат и передает урон в ApplyDamage.

### 3.1.38 Класс **DepthrunHUD**

Класс **DepthrunHUD** является основным объектом интерфейса, создающим и управляющим виджетами игрового экрана. Класс обрабатывает обновление полосы здоровья, отображение всплывающего окна награды сундука и переключение отладочного виджета адаптивного модуля.

Атрибуты класса:

- MainWidgetClass – класс основного виджета интерфейса;
- DebugAdaptiveWidgetClass – класс отладочного виджета адаптивного модуля;
- ChestRewardWidgetClass – класс виджета отображения награды сундука;
- bDebugWidgetVisible – флаг видимости отладочного виджета.

Методы класса:

- BeginPlay () – создает и добавляет виджеты на экран;
- ShowChestReward (Payload) – отображает всплывающее окно с перечнем наград сундука;

- RegisterChest (Chest) – подписывается на делегат открытия сундука;
- UpdatePlayerHP (Current, Max) – обновляет значения полосы здоровья игрока;
- ToggleAdaptiveDebugWidget () – переключает видимость отладочного виджета адаптивного модуля;
- GetHUDOverlay () – возвращает указатель на основной оверлей интерфейса.

### 3.1.39 Класс TrapdoorActor

Класс TrapdoorActor реализует объект люка, появляющегося при очистке финальной комнаты подземелья. При входе персонажа в зону перекрытия инициируется завершение уровня. Задержка активации предотвращает мгновенное срабатывание при создании объекта.

Атрибуты класса:

- CollisionBox – компонент коллизии зоны перекрытия;
- SpriteComponent – компонент спрайта для визуального отображения;
- ActivationTimer – дескриптор таймера отложенной активации коллизии.

Методы класса:

- BeginPlay () – запускает таймер отложенной активации коллизии;
- OnTrapdoorOverlap () – обрабатывает перекрытие игроком и инициирует переход на следующий уровень;
- EnableOverlap () – включает коллизию перекрытия по истечении задержки.

### 3.1.40 Класс DemonstrationSubsystem

Класс DemonstrationSubsystem является подсистемой игрового экземпляра, предназначенной для диагностического журналирования работы модуля адаптивного поведения в процессе игры. Подсистема выводит подробную трассировку в консольный журнал движка через канал LogAdaptiveBehavior, позволяя наблюдать за принятием решений противником в реальном времени без остановки игры.

В процессе каждого цикла оценки в журнал записываются текущий коэффициент угрозы, значения всех шести весов, идентификатор выбранного состояния и доминирующий паттерн поведения игрока. Помимо этого, фиксируются отдельные игровые события: выстрелы и удары персонажа, получение урона противником, смена состояния конечного автомата и обновление весов по оценке результата. Совокупность этих записей позволяет восстановить полную картину принятых решений за любой период игровой сессии.

Атрибуты класса:

- bVerboseLogging – флаг расширенного журналирования: при активации в журнал выводятся значения всех шести весов, коэффициент



угрозы, выбранное состояние и игровые события на каждом цикле оценки.

Методы класса:

- `Initialize(Collection)` – инициализирует подсистему и подготавливает канал журналирования;
- `Deinitialize()` – завершает работу подсистемы и сбрасывает внутренние флаги;
- `LogEvaluationResult()` – выводит в консольный журнал значения коэффициента угрозы, весов и выбранного состояния по завершении цикла оценки;
- `LogCombatEvent(EventType, Instigator, Target)` – фиксирует в журнале боевое событие: выстрел, удар персонажа или противника, получение урона;
- `LogWeightUpdate(OldWeights, NewWeights, Reward)` – выводит в журнал изменение весов после получения оценки результата;
- `LogStateTransition(FromState, ToState, ThreatValue)` – фиксирует смену состояния конечного автомата с указанием текущего коэффициента угрозы.

### 3.1.41 Класс `PlayerCombatComponent`

Класс `PlayerCombatComponent` управляет активным вооружением персонажа и транслирует делегат нанесения урона. Компонент `AdaptiveBehaviorComponent` противника подписывается на этот делегат для получения сигнала вознаграждения +1.

Атрибуты класса:

- `CurrentWeapon` – указатель на текущее активное оружие;
- `OnDamageDealt` – делегат, транслируемый при нанесении урона противнику.

Методы класса:

- `Attack()` – вызывает метод `Fire()` текущего оружия;
- `EquipWeapon(NewWeapon)` – заменяет текущее оружие новым.

### 3.1.42 Класс `PlayerActionTracker`

Класс `PlayerActionTracker` фиксирует каждое действие персонажа с отметкой времени и местоположения и транслирует его через делегат. Компоненты `AdaptiveBehaviorComponent` всех противников подписываются на этот делегат для обновления памяти и распознавателя паттернов.

Атрибуты класса:

- `OnPlayerAction` – делегат, транслируемый при каждом регистрируемом действии персонажа.

Методы класса:

- `RecordAction(ActionType, Location, Intensity)` – фиксирует действие, добавляет временную метку и транслирует делегат;
- `GetCurrentTimestamp()` – возвращает текущее игровое время для записи события.

### 3.1.43 Класс `UtilityCurves`

Класс `UtilityCurves` преобразует текущее значение коэффициента угрозы в оценку полезности для каждого из пяти состояний конечного автомата. Каждое состояние имеет собственную кривую: ожидание наиболее полезно при низкой угрозе, преследование и атака – при средней, фланговый обход – при средне-высокой с учетом наличия союзников, отступление – при высокой угрозе. Результирующие оценки передаются в `StateTransitionResolver` для выбора оптимального состояния.

Методы класса:

- `EvaluateUtility(State, Threat, Context, Config)` – вычисляет полезность указанного состояния для текущих  $T_{final}$  и контекста;
- `EvaluateIdle(T)` – вычисляет полезность состояния ожидания по параболической формуле;
- `EvaluateChase(T, Context, Cfg)` – вычисляет полезность состояния преследования по колоколообразной кривой;
- `EvaluateAttack(T, Cfg)` – вычисляет полезность состояния атаки по колоколообразной кривой;
- `EvaluateFlank(T, Context, Cfg)` – вычисляет полезность флангового обхода с учетом фактора союзников;
- `EvaluateRetreat(T, Context, Cfg)` – вычисляет полезность состояния отступления по сигмоидной функции.

### 3.1.44 Класс `RoomTemplate`

Класс `RoomTemplate` является ресурсом данных, описывающим шаблон одного типа комнаты подземелья. Содержит ссылки на тайловую карту, классы объектов (двери, сундук, люк), конфигурацию наград, параметры создания противников и декора, а также настройки позиционирования объектов по осям.

Атрибуты класса:

- `TileMapAsset` – ссылка на ресурс тайловой карты комнаты;
- `RoomType` – тип комнаты, значение перечисления `ERoomType`;
- `WorldScale` – равномерный масштаб для процедурного размещения объектов;
- `DoorClass` – класс объекта двери;
- `TrapdoorClass` – класс объекта люка для финальной комнаты;
- `ChestClass` – класс объекта сундука;
- `ChestLootConfig` – конфигурационный ресурс параметров наград сундука;
- `ChestItemCollection` – ресурс коллекции предметов для наград сундука;
- `PotentialEnemies` – массив структур создания противников с весами вероятности;
- `MinEnemies` – минимальное количество противников в комнате;
- `MaxEnemies` – максимальное количество противников в комнате;

- ChestSpawnChance – вероятность создания сундука в процентах;
  - TorchSpawnChance – вероятность создания декоративного факела в процентах;
  - SpawnOffset – мастер-смещение для всех процедурно создаваемых объектов;
  - EnemyLockedZ – высота плоскости для противников;
  - PlayerLockedZ – высота плоскости для персонажа;
  - MinProps – минимальное количество декоративных объектов;
  - MaxProps – максимальное количество декоративных объектов.
- Методы класса:

### 3.1.45 Класс RoomTransitionVolume

Класс RoomTransitionVolume является триггерным объемом, размещенным у выходов комнаты. При входе персонажа в зону перекрытия объем уведомляет RoomGeneratorSubsystem для активации следующей комнаты.

Атрибуты класса:

- OwnerRoom – указатель на комнату-владельца;
- ExitIndex – индекс выхода в массиве выходных точек комнаты;
- TriggerBox – компонент коллизии зоны перекрытия.

Методы класса:

- BeginPlay() – настраивает компонент коллизии и подписывает обработчик;
- OnBeginOverlap() – уведомляет подсистему генерации комнат о переходе игрока.

## 3.2 Описание модели данных

В подразделе описано проектирование модели данных игры Depthrun. Модель включает две таблицы, каждая из которых обслуживает отдельную область постоянного хранения: профиль игрока с метапрогрессией и историю прохождений. При описании таблиц в качестве дополнительной информации для полей приведены типы данных и описания, определенные с учетом выбора SQLite в качестве СУБД в подразделе 2.5.

### 3.2.1 Таблица player\_profile

Таблица player\_profile хранит постоянный профиль игрока, сохраняющийся между игровыми забегами. Таблица рассчитана на хранение единственной записи. Каждому уровню улучшения соответствует фиксированный числовой множитель, применяемый к характеристикам персонажа при старте нового прохождения. Значение 0 означает отсутствие улучшения, значение 5 – максимальный уровень для большинства улучшений, для улучшения ArrowCount – максимальный уровень 3. Структура таблицы представлена в таблице 3.1.

Таблица 3.1 – Таблица `player_profile`

| Поле                        | Тип     | Описание                                                       |
|-----------------------------|---------|----------------------------------------------------------------|
| <code>id</code>             | INTEGER | Уникальный идентификатор профиля. Всегда равен 1               |
| <code>TotalDiamonds</code>  | INTEGER | Накопленное количество алмазов, основная валюта метапрогрессии |
| <code>Damage_Lvl</code>     | INTEGER | Уровень улучшения урона (0–5)                                  |
| <code>Range_Lvl</code>      | INTEGER | Уровень улучшения дальности атаки (0–5)                        |
| <code>ArrowCount_Lvl</code> | INTEGER | Уровень улучшения количества стрел (0–3)                       |
| <code>MaxHP_Lvl</code>      | INTEGER | Уровень улучшения максимального здоровья (0–5)                 |

### 3.2.2 Таблица `run_history`

Таблица `run_history` формирует журнал завершенных игровых забегов. Каждая запись соответствует одному прохождению и создается по его итогам. Временная метка проставляется автоматически средствами SQLite через функцию `datetime('now')`. Структура таблицы представлена в таблице 3.2.

Таблица 3.2 – Таблица `run_history`

| Поле                     | Тип     | Описание                                    |
|--------------------------|---------|---------------------------------------------|
| <code>id</code>          | INTEGER | Уникальный идентификатор записи             |
| <code>Rooms</code>       | INTEGER | Количество пройденных комнат за забег       |
| <code>Won</code>         | INTEGER | Результат забега: 1 – победа, 0 – поражение |
| <code>RunDuration</code> | REAL    | Продолжительность забега в секундах         |
| <code>Timestamp</code>   | TEXT    | Дата и время завершения забега              |

## 4 РАЗРАБОТКА ПРОГРАММНЫХ МОДУЛЕЙ

В разделе рассматривается реализация ключевых компонентов программного средства, обеспечивающих корректную работу адаптивного поведения врагов и связанных подсистем игры. Полный программный комплекс включает более 40 классов на языке C++ [20]. В рамках раздела подробно описаны модули конечного автомата, оценки контекста и угрозы, памяти и распознавания паттернов поведения игрока, адаптации весов, выбора состояния через функции полезности и процедурной генерации игровых комнат, так как именно эти модули составляют ядро проекта и отражают основные алгоритмические особенности программного модуля.

Модуль конечного автомата реализует архитектурную основу поведения врага, на которую опираются остальные слои принятия решений. Модуль оценки контекста и угрозы выполняет первичный сбор данных о боевой ситуации и формирует единое числовое значение итоговой угрозы. Модуль памяти и распознавания паттернов накапливает действия игрока в скользящем окне и определяет преобладающую тактику противника. Модуль адаптации весов изменяет коэффициенты значимости факторов угрозы по сигналу, поступающему от событий нанесения и получения урона. Модуль выбора состояния сопоставляет каждому из пяти состояний автомата значение полезности и определяет оптимальное состояние для текущего тика оценки. Модуль процедурной генерации обеспечивает построение последовательности комнат подземелья.

Термин «тик» в работе программного средства обозначает периодический такт, на котором подсистема выполняет очередной расчет или обновление своего состояния. Модуль адаптивного поведения использует таймерный тик с интервалом 0.3 секунды. Это значение задано в конфигурации и выбрано по двум причинам. Выполнение полной цепочки из оценки контекста, расчета угрозы, анализа памяти и выбора состояния занимает около 0.2 миллисекунды на одну сущность. Решения «сменить состояние» имеют смысл только при заметном изменении боевой ситуации, а за один кадр длительностью 16.7 миллисекунды позиция игрока, здоровье сущности и число союзников почти не меняются.

Интервал 0.3 секунды обеспечивает отклик, неотличимый от мгновенного для восприятия игрока, и при этом сокращает количество расчетов в 18 раз по сравнению с покадровым вариантом.

Схема программы представлена на чертеже ГУИР.400201.112 ПД.

### 4.1 Модуль конечного автомата

Модуль реализован классом `FSMComponent` и набором из пяти классов-наследников `FSMState`, представляющих состояния `Idle` (бездействие), `Chase` (преследование), `Attack` (атака), `Retreat` (отступление) и `Flank` (фланговый обход). Компонент устанавливается на сущность `BP_BaseEnemy` и хранит текущее состояние, время пребывания в нем, ссылку на владельца и

зарегистрированные экземпляры состояний. Все переходы выполняются исключительно через метод `TransitionTo(EFSMStateType)`, вызов которого допустим как из адаптивного модуля, так и из самого состояния по внутренним условиям. Прямая модификация поля `CurrentState` запрещена, исключая конфликты между данными состояния и логикой компонента.

Для алгоритма работы конечного автомата реализован метод `EvaluateAndTransition()`, охватывающий инициализацию компонента, обновление текущего состояния и смену состояния с корректными выходами и входами. Входные данные алгоритма работы конечного автомата [21]:

- `OwnerEnemy` – ссылка на сущность -владельца типа `ABaseEnemy`;
- `InitialState` – начальное состояние, задаваемое в редакторе через `UPROPERTY`;
- `RequestedState` – целевое состояние, поступающее от адаптивного модуля.

Алгоритм работы конечного автомата по шагам:

Шаг 1. Начало алгоритма.

Шаг 2. Создание экземпляров пяти состояний и регистрация их в карте `StateMap` при первом запуске.

```
void UFSMComponent::BeginPlay() {
    Super::BeginPlay();
    RegisterState(EFSMStateType::Idle,
NewObject<UFSMState_Idle>(this));
    RegisterState(EFSMStateType::Chase,
NewObject<UFSMState_Chase>(this));
    RegisterState(EFSMStateType::Attack,
NewObject<UFSMState_Attack>(this));
    RegisterState(EFSMStateType::Retreat,
NewObject<UFSMState_Retreat>(this));
    RegisterState(EFSMStateType::Flank,
NewObject<UFSMState_Flank>(this));
}
```

Шаг 3. Установка начального состояния и вызов метода `EnterState()` для подготовки внутренних данных состояния.

```
CurrentState = InitialState;
StateMap[CurrentState]->EnterState(GetOwner());
TimeInState = 0.f;
```

Шаг 4. На каждом игровом кадре приращение времени пребывания в состоянии и вызов метода `UpdateState()` текущего состояния.

```
void UFSMComponent::TickComponent(float DeltaTime, ...) {
    TimeInState += DeltaTime;

    if (UFSMState* State = StateMap.FindRef(CurrentState))
        State->UpdateState(GetOwner(), DeltaTime); }
```

Шаг 5. Получение запроса на смену состояния из адаптивного модуля или из текущего состояния через метод `TransitionTo()`.

```
void UFSMComponent::TransitionTo(EFSMStateType NewState) {
    if (NewState == CurrentState) return;
    if (!StateMap.Contains(NewState)) return;
    ...
}
```

Шаг 6. Проверка корректности запроса: совпадение нового состояния с текущим и наличие нового состояния в таблице зарегистрированных состояний приводят к раннему выходу.

Шаг 7. Вызов метода `ExitState()` текущего состояния для освобождения временных ресурсов, остановки таймеров атак и сброса целей.

```
StateMap[CurrentState]->ExitState(GetOwner());
```

Шаг 8. Замена значения `CurrentState` на целевое и обнуление времени пребывания в состоянии.

```
CurrentState = NewState;
TimeInState = 0.f;
```

Шаг 9. Вызов метода `EnterState()` нового состояния для инициализации поведения, например выбора точки фланкирования или включения режима стрельбы.

```
StateMap[CurrentState]->EnterState(GetOwner());
```

Шаг 10. Вызов делегата `OnStateChanged` для уведомления подписчиков, в том числе виджета отладки.

Шаг 11. Конец алгоритма.

Метод `EvaluateAndTransition()` обеспечивает строгую дисциплину переходов и гарантирует, что любое состояние получает корректные вызовы `EnterState` и `ExitState`, без чего невозможна предсказуемая работа верхних слоев адаптивной системы.

## 4.2 Модуль оценки контекста и угрозы

Модуль объединяет два класса. Класс `UContextEvaluator` выполняет сбор первичных параметров боевой ситуации и формирует структуру `FContextData`, содержащую шесть нормализованных факторов. Класс `UThreatCalculator` применяет к этим факторам линейную свертку с динамическими весами и возвращает структуру `FThreatAssessment` с итоговым значением угрозы  $T_{final}$  в диапазоне от 0 до 1.

Шесть факторов отражают шесть независимых каналов оценки боевой

ситуации. Первый канал – расстояние от врага до игрока, нормализованное в обратном направлении, так что близкое расположение дает значение, близкое к 1. Второй канал – тип оружия игрока, представленный дискретной таблицей значений (значение 0.6 для меча, 0.8 для лука). Третий канал – здоровье противника после нелинейного преобразования. Четвертый канал – число союзников врага в окружении радиусом 250 единиц, инвертированное по квадратному корню. Пятый канал – общая плотность существ в комнате. Шестой канал – агрессия игрока, вычисленная скользящим окном памяти.

Нормализация расстояния выполнена по формуле:

$$D_{\text{norm}} = 1 - \text{clamp}\left(\frac{D}{D_{\text{max}}}, 0, 1\right), \quad (4.1)$$

где  $D_{\text{norm}}$  – нормализованное значение фактора расстояния;

$D$  – фактическое расстояние от врага до игрока в плоскости XY, единиц игрового мира;

$D_{\text{max}}$  – максимальная боевая дистанция, заданная в конфигурации UAdaptiveConfig, единиц игрового мира.

Значение угрозы оружия задано таблицей соответствия, а не формулой, так как тип оружия принимает дискретное значение из перечисления EWeaponType. Для меча в ближнем бою значение  $W_{\text{norm}}$  равно 0.6, для лука дальнего боя – 0.8.

Нелинейное преобразование здоровья противника выполнено по формуле:

$$H_{\text{norm}} = (1 - \text{HP}_{\text{ratio}})^{\alpha}, \quad (4.2)$$

где  $H_{\text{norm}}$  – нормализованный вклад фактора здоровья;

$\text{HP}_{\text{ratio}}$  – доля текущего здоровья от максимального, от 0 до 1;

$\alpha$  – показатель степени, заданный в конфигурации значением 2.

Квадратичная форма обеспечивает резкий рост угрозы при низком уровне здоровья. При  $\text{HP}_{\text{ratio}}$  равном 0.5 значение  $H_{\text{norm}}$  составляет 0.25, тогда как при  $\text{HP}_{\text{ratio}}$  равном 0.2 значение  $H_{\text{norm}}$  уже равно 0.64. Такое поведение соответствует тактическому ожиданию: чем сильнее ранен противник, тем выше его уязвимость и тем активнее должны срабатывать состояния отступления.

Вклад фактора численности союзников выполняется по формуле:

$$A_{\text{norm}} = \text{clamp}\left(\frac{N_{\text{ally}}}{N_{\text{max}}}, 0, 1\right), \quad (4.3)$$

где  $A_{\text{norm}}$  – итоговый вклад фактора союзников;

$N_{\text{ally}}$  – количество союзников в радиусе проверки, штук;

$N_{\text{max}}$  – пороговое число союзников, штук.



Нормализация выполняется линейно:  $A_{\text{norm}}$  равномерно растет от 0.0 при отсутствии союзников до 1.0 при достижении порога  $N_{\text{max}}$ . При  $N_{\text{max}} = 4$  каждый союзник добавляет ровно 0.25. Значение  $A_{\text{norm}}$  хранится в структуре контекста и используется в двух местах с разной семантикой.

В функции полезности состояния Flank  $A_{\text{norm}}$  входит напрямую как коэффициент масштабирования: чем больше союзников, тем выше готовность врага к фланговому маневру.

Плотность комнаты вычислена по формуле:

$$R_{\text{norm}} = \text{clamp}\left(\frac{N_{\text{room}}}{R_{\text{max}}}, 0, 1\right), \quad (4.4)$$

где  $N_{\text{norm}}$  – нормализованная плотность комнаты;

$N_{\text{norm}}$  – число сущностей класса  $A_{\text{Pawn}}$  в радиусе 250 единиц, штук;

$R_{\text{max}}$  – максимальная вместимость комнаты, заданная значением 8, штук.

Значение  $R_{\text{max}} = 8$  подобрано экспериментально под максимальное число противников, одновременно размещаемых в комнате.

Фактор агрессивности памяти получен через подсчет действий игрока в скользящем временном окне. Описание формулы приведено в подразделе 4.3.

Итоговое значение угрозы вычислено по формуле:

$$T_{\text{final}} = \text{clamp}\left(\sum_{i=1}^6 w_i \cdot f_i(x_i), 0, 1\right), \quad (4.5)$$

где  $T_{\text{final}}$  – итоговое значение угрозы в диапазоне от 0 до 1;

$w_i$  – динамический вес  $i$ -го фактора;

$f_i(x_i)$  – нормализованное значение  $i$ -го фактора из структуры  $\text{ContextData}$ ;

$i$  – индекс фактора в порядке: расстояние, угроза оружия, здоровье, союзники, плотность комнаты, память.

Применение линейной свертки обусловлено требованием прозрачности модели. Любое значение итоговой угрозы раскладывается на шесть слагаемых, сохраняется в структуре  $\text{ContextData}$  и доступно для чтения через виджет отладки, позволяя наблюдать вклад каждого фактора в реальном времени.

Для алгоритма вычисления итоговой угрозы реализован метод  $\text{EvaluateThreat}()$ , охватывающий полный конвейер из получения данных контекста и применения линейной свертки. Метод вызывается периодически с интервалом  $\text{EvaluationInterval}$  (0.3 секунды).

Входные данные алгоритма вычисления итоговой угрозы:

- Owner – ссылка на сущность-владельца типа  $\text{BaseEnemy}$ ;
- Player – ссылка на игрового персонажа типа  $\text{DepthrunCharacter}$ ;
- Memory – ссылка на компонент памяти типа  $\text{AdaptiveMemory}$ ;
- Config – ссылка на конфигурацию типа  $\text{AdaptiveConfig}$ ;

- `WeightsMgr` – ссылка на менеджер динамических весов.

Алгоритм вычисления итоговой угрозы по шагам:

Шаг 1. Начало алгоритма.

Шаг 2. Расчет расстояния от сущности до игрока в плоскости XY и нормализация по формуле (4.1).

```
float Dist = FVector::Dist2D(Player->GetActorLocation(),
                             Owner->GetActorLocation());
Data.DistanceNorm = 1.f - FMath::Clamp(Dist / Config-
>MaxEngagementRange, 0.f, 1.f);
```

Шаг 3. Определение типа оружия игрока через компонент `PlayerCombatComponent` и получение значения угрозы оружия (меч или лук) из таблицы.

```
Data.WeaponThreatNorm = GetWeaponThreatNorm
(Data.PlayerWeaponType);
```

Шаг 4. Чтение текущего и максимального здоровья противника из `EnemyHealthComponent` с последующим применением формулы (4.2).

```
float Ratio = HC->GetCurrentHP() / HC->GetMaxHP();
Data.EnemyHPRatioNorm = FMath::Pow(1.f - Ratio,
                                   Config>HealthPowerExponent);
```

Шаг 5. Подсчет союзников в радиусе `AllyCheckRadius` через сферический оверлап с фильтром по классу `BaseEnemy` и нормализация согласно формуле (4.3).

```
Data.NearbyAllyCount = CountNearbyAllies(Owner, Config-
>AllyCheckRadius);
Data.AllyCountNorm = FMath::Clamp(Data.NearbyAllyCount /
                                   Config>MaxAllyThreshold, 0.f, 1.f);
```

Шаг 6. Подсчет сущностей `Pawn` в радиусе 250 единиц для оценки плотности комнаты по формуле (4.4).

```
Data.RoomDensityNorm = EvaluateRoomDensity(Owner, Config-
>MaxRoomCapacity);
```

Шаг 7. Запрос значения агрессивности памяти у компонента `AdaptiveMemory`.

```
Data.MemoryAggressiveness = Memory-
>GetDecayedAggressiveness(Now, 0.f);
```

Шаг 8. Получение текущего набора весов из менеджера

DynamicWeightManager.

```
const TArray<float>& W = WeightsMgr->GetWeights();
```

**Шаг 9. Вычисление линейной свертки шести факторов с инверсией союзников.**

```
float TBase = W[0]*Data.DistanceNorm  
+ W[1]*Data.WeaponThreatNorm  
+ W[2]*Data.EnemyHPRatioNorm  
+ W[3]*(1.f - FMath::Sqrt(Data.AllyCountNorm))  
+ W[4]*Data.RoomDensityNorm  
+ W[5]*Data.MemoryAggressiveness;
```

**Шаг 10. Ограничение результата диапазоном от 0 до 1 и запись в поле ThreatFinal структуры ThreatAssessment.**

```
Result.ThreatFinal = FMath::Clamp(TBase, 0.f, 1.f);
```

**Шаг 11. Запись результата оценки в поле LastAssessment и вызов OnThreatEvaluated.**

**Шаг 12. Возврат структуры ThreatAssessment.**

**Шаг 13. Конец алгоритма.**

Базовая конфигурация весов факторов в конфигурации представлена в таблице 4.1. Значения подобраны исходя из приоритета факторов. Сумма базовых весов равна 1.0. Верхние границы весов применяются при адаптации, описанной в подразделе 4.4, и не позволяют ни одному фактору доминировать. Минимальные границы гарантируют, что факторы полностью не исчезнут даже при длительной адаптации. Числовые значения весов определены эмпирически путем наблюдения за поведением противников в тестовых сценариях.

Таблица 4.1 – Базовые значения весов факторов в AdaptiveConfig

| Индекс | Фактор                              | Базовое значение | Минимум | Максимум |
|--------|-------------------------------------|------------------|---------|----------|
| 0      | Расстояние (Distance)               | 0.20             | 0.05    | 0.40     |
| 1      | Угроза оружия игрока (WeaponThreat) | 0.20             | 0.05    | 0.40     |
| 2      | Здоровье сущности (Health)          | 0.20             | 0.05    | 0.40     |
| 3      | Союзники в радиусе (Allies)         | 0.15             | 0.05    | 0.40     |
| 4      | Плотность комнаты (RoomDensity)     | 0.10             | 0.05    | 0.40     |
| 5      | Память (Memory)                     | 0.15             | 0.05    | 0.40     |

### 4.3 Модуль памяти и распознавания паттернов поведения

Модуль состоит из двух компонентов. Модуль памяти (класс `AdaptiveMemory`) ведет хронологический буфер событий типа `MemoryEvent` с указанием типа действия игрока, временной метки и интенсивности. Модуль распознавания паттернов (класс `PatternRecognizer`) хранит скользящее окно действий размером 15 элементов и строит на нем три словаря частот: одиночных действий, биграмм и триграмм. Действия игрока поступают в оба компонента через `OnPlayerAction`.

Список распознаваемых действий включает выстрел из лука, удар мечом, рывок. Каждое действие записывается в буфер памяти и в окно распознавателя в момент его совершения игроком. Типы действий выбраны таким образом, чтобы покрыть три основные тактики игрока: дистанционный бой, ближний бой и уклонение.

Расчет агрессивности через временное окно основан на формуле:

$$M_{aggr} = \text{clamp}\left(\frac{N_{actions}}{T_{window}}, 0, 1\right), \quad (4.6)$$

где  $M_{aggr}$  – значение агрессивности памяти;

$N_{actions}$  – количество атакующих действий игрока, попавших в окно, штук;

$T_{window}$  – ширина временного окна, заданная значением 5 секунд.

Размер окна в 5 секунд установлен целенаправленно. На десятисекундном окне три выстрела за две секунды дают значение 0.30, что ниже порога 0.4 для активации бонуса агрессивности в состоянии преследования. На пятисекундном окне те же три выстрела дают значение 0.60, что обеспечивает наглядность реакции системы.

Распознавание паттернов выполнено через построение N-грамм по содержимому окна действий. Для окна длиной N формируется N одиночных действий, N-1 биграмм и N-2 триграмм. Доминирующий паттерн определяется по формуле:

$$S(p) = \text{count}(p) \cdot \text{len}(p), \quad (4.7)$$

где  $S(p)$  – итоговая оценка паттерна p;

$\text{count}(p)$  – число повторений паттерна в окне, штук;

$\text{len}(p)$  – длина паттерна (1, 2 или 3).

Умножение на длину паттерна обеспечивает приоритет более длинных последовательностей. Триграмма `Shot+Shot+Shot`, встретившаяся один раз, получает оценку 3 и преобладает над одиночным действием `Shot`, встретившимся два раза с оценкой 2. При равенстве оценок приоритет получает паттерн, обнаруженный позже, что соответствует свежести информации о тактике игрока.

Получение модификатора стоимости состояния выполнено по

справочной таблице правил. Каждое правило содержит шаблон паттерна, состояние-кандидат и величину модификатора, добавляемого к итоговой оценке состояния. Таблица правил представлена в таблице 4.2 и включает наиболее значимые случаи реакции на повторяющиеся действия игрока.

Таблица 4.2 – Правила модификаторов состояний по распознанным паттернам

| Паттерн        | Состояние-кандидат | Модификатор |
|----------------|--------------------|-------------|
| Shot+Shot+Shot | Flank              | +0.65       |
| Shot+Shot+Shot | Chase              | −0.20       |
| Shot+Shot+Shot | Attack             | −0.25       |
| Shot+Shot      | Flank              | +0.65       |
| Shot+Shot      | Attack             | −0.20       |
| Shot           | Flank              | +0.50       |
| Melee+Dash     | Retreat            | +0.25       |
| Dash+Dash      | Chase              | +0.20       |

Для алгоритма анализа поведения игрока реализован метод `ProcessPlayerAction()`, охватывающий запись события в буфер памяти, перестройку N-грамм и определение доминирующего паттерна.

Входные данные алгоритма анализа поведения игрока:

- `ActionEvent` – структура `PlayerActionEvent` с полями типа действия, временной метки и интенсивности;
- `Memory` – ссылка на компонент `AdaptiveMemory`;
- `PatternRecog` – ссылка на компонент `PatternRecognizer`;
- `WindowSize` – размер окна анализа, заданный значением 15 элементов;
- `MemoryWindowSeconds` – ширина временного окна памяти, 5 секунд.

Алгоритм анализа поведения игрока по шагам:

Шаг 1. Начало алгоритма.

Шаг 2. Получение события действия игрока через подписку на делегат `OnPlayerAction`.

```
void AdaptiveBehaviorComponent::HandlePlayerAction(
    const FPlayerActionEvent& ActionEvent) { ... }
```

Шаг 3. Формирование структуры `MemoryEvent` и добавление ее в хронологический буфер компонента `AdaptiveMemory`.

```
FMemoryEvent Ev;
Ev.ActionType = ActionEvent.ActionType;
Ev.Timestamp = ActionEvent.Timestamp;
Ev.Intensity = ActionEvent.Intensity;
Memory->RecordEvent (Ev) ;
```

Шаг 4. Передача типа действия в распознаватель паттернов и проверка

переполнения окна.

```
if (ActionWindow.Num() >= WindowSize)
    ActionWindow.RemoveAt(0);
ActionWindow.Add(ActionType);
```

Шаг 5. Удаление из буфера памяти событий, время которых отстает от текущего более чем на 5 секунд.

```
Memory->CleanupOldEvents(GetWorld()->GetTimeSeconds(), 0.f);
```

Шаг 6. Перестройка словаря одиночных действий по содержимому окна.

```
for (int32 i = 0; i < N; ++i)
    Unigrams.FindOrAdd(ActionToString(ActionWindow[i]))++;
```

Шаг 7. Перестройка словаря биграмм путем конкатенации соседних пар действий.

```
for (int32 i = 0; i + 1 < N; ++i)
    Bigrams.FindOrAdd(Key2)++;
```

Шаг 8. Перестройка словаря триграмм путем конкатенации троек подряд идущих действий.

```
for (int32 i = 0; i + 2 < N; ++i)
    Trigrams.FindOrAdd(Key3)++;
```

Шаг 9. Перебор словарей с применением формулы (4.7) и определение паттерна с максимальной оценкой.

```
for (const auto& Pair : Trigrams)
    if (Pair.Value * 3 > BestScore) {
        BestScore = Pair.Value * 3;
        BestPattern = Pair.Key;
    }
```

Шаг 10. При равенстве оценок выбор более позднего паттерна, что обеспечивает приоритет свежей информации о действиях игрока.

Шаг 11. Подсчет значения агрессивности памяти по формуле (4.6) на основе числа атакующих действий в окне 5 секунд.

```
Data.MemoryAggressiveness = Memory->GetDecayedAggressiveness(Now, 0.f);
```

Шаг 12. Возврат имени доминирующего паттерна и значения агрессивности для использования в модуле адаптации весов и в модуле выбора

состояния.

Шаг 13. Конец алгоритма.

#### 4.4 Модуль адаптации весов

Модуль реализован классом `DynamicWeightManager` и хранит массив из шести вещественных значений, соответствующих весам факторов из таблицы 4.1. Начальные значения копируются из конфигурации `AdaptiveConfig` при создании компонента. Адаптация весов выполняется по результату боевого взаимодействия, поступающему от двух событий: нанесение урона игроку (положительное подкрепление +1) и получение урона от игрока (отрицательное подкрепление -1). Подписка на эти события выполнена в методе `BeginPlay` компонента `AdaptiveBehaviorComponent`.

Логика адаптации основана на принципе локального обучения. Если действие сущности привело к успеху, веса тех факторов, которые в момент действия имели высокие значения, увеличиваются. При неудаче те же веса уменьшаются. Это позволяет системе постепенно усиливать значимость факторов, действительно соответствующих текущей боевой ситуации, без использования нейронных сетей и предварительного обучения.

Расчет вклада фактора выполнен по формуле:

$$c_i = \frac{f_i(x_i)}{\sum_{j=1}^6 f_j(x_j)}, \quad (4.8)$$

где  $c_i$  – нормализованный вклад  $i$ -го фактора в текущую боевую ситуацию;

$f_i(x_i)$  – значение  $i$ -го фактора в момент события подкрепления;

$f_j(x_j)$  – нормализованные факторы.

Знаменатель отличен от нуля при любой боевой ситуации.

Обновление веса  $i$ -го фактора выполнено по формуле:

$$w_i(t+1) = \text{clamp}(w_i(t) + \eta \cdot \text{reward} \cdot c_i, w_{\min}, w_{\max}), \quad (4.9)$$

где  $w_i(t+1)$  – новое значение  $i$ -го веса после события подкрепления;

$w_i(t)$  – значение  $i$ -го веса до события;

$\eta$  – коэффициент скорости обучения, заданный значением 0.10;

$\text{reward}$  – величина подкрепления, +1 при попадании и -1 при получении урона;

$w_{\min}$  – нижняя граница веса, 0.05;

$w_{\max}$  – верхняя граница веса, 0.40.

Скорость обучения 0.10 подобрана экспериментально. При значении 0.04 веса показывали заметное смещение только после 15–20 событий боя, что не позволяло продемонстрировать адаптацию за короткий промежуток времени. При значении 0.10 смещение видно после 5–8 событий, что соответствует обычной длительности столкновения с одним противником.

После обновления отдельных весов выполняется их повторная нормализация по формуле:

$$\hat{w}_i = \frac{w_i}{\sum_{j=1}^6 w_j}, \quad (4.10)$$

где  $\hat{w}_i$  – нормализованный  $i$ -й вес после восстановления единичной суммы;  
 $w_j$  – вес после индивидуального ограничения по формуле (4.9).

Без нормализации каждый индивидуальный вызов ограничения допускает рост суммы весов до значения  $6 \cdot 0.40 = 2.4$ . При такой сумме значение итоговой угрозы насыщается на уровне 1.0 при любом контексте, утилита состояния отступления побеждает постоянно, и сущности совершают цикл бездействие – отступление. Нормализация устраняет это поведение.

Для алгоритма адаптации весов реализован метод `AdaptWeights()`, охватывающий прием оценки исхода, расчет вклада факторов и пересчет значений с восстановлением единичной суммы.

Входные данные алгоритма адаптации весов:

- `Reward` – оценка исхода взаимодействия (+1 или -1);
- `Context` – структура `ContextData` со значениями шести факторов на момент события;
- `Config` – ссылка на конфигурацию `AdaptiveConfig`;
- `Weights` – текущий массив из шести весов, изменяемый алгоритмом.

Алгоритм адаптации весов по шагам:

Шаг 1. Начало алгоритма.

Шаг 2. Срабатывание подписки на событие нанесения или получения урона сущностью и определение знака подкрепления.

```
void AdaptiveBehaviorComponent::OnDamageDealt() {  
    WeightManager->UpdateWeights(1.0f, LastContext, Config);  
}  
void AdaptiveBehaviorComponent::OnDamageTaken() {  
    WeightManager->UpdateWeights(-1.0f, LastContext,  
Config);  
}
```

Шаг 3. Проверка валидности конфигурации и наличия шести значений в массиве весов.

Шаг 4. Формирование локального массива `FactorValues` с шестью значениями факторов из структуры `ContextData` в порядке: расстояние, угроза оружия, здоровье, количество союзников, плотность комнаты, агрессивность памяти.

```
const float FactorValues[6] = {  
    Context.DistanceNorm,  
    Context.WeaponThreatNorm,  
    Context.EnemyHPRatioNorm,
```



```

1.f - FMath::Sqrt(Context.AllyCountNorm),
Context.RoomDensityNorm,
Context.MemoryAggressiveness
};

```

**Шаг 5. Расчет суммы факторов для нормализации.**

```

float SumFactors = 0.f;
for (int32 i = 0; i < 6; ++i)
    SumFactors += FactorValues[i];

```

**Шаг 6. Обновление каждого из шести весов по формуле (4.9) с учетом индивидуального вклада фактора.**

```

for (int32 i = 0; i < 6; ++i) {
    const float Contribution = FactorValues[i] / SumFactors;
    Weights[i] = FMath::Clamp(
        Weights[i] + Eta * Reward * Contribution, WMin, WMax);
}

```

**Шаг 7. Расчет новой суммы весов после индивидуальных ограничений.**

```

float Sum = 0.f;
for (int32 i = 0; i < 6; ++i) Sum += Weights[i];

```

**Шаг 8. Восстановление единичной суммы весов по формуле (4.10).**

```

const float InvSum = 1.f / Sum;
for (int32 i = 0; i < 6; ++i) Weights[i] *= InvSum;

```

**Шаг 9. Запись итоговых значений весов в журнал LogDynamicWeights для последующего вывода в виджете отладки.**

**Шаг 10. Возврат управления вызывающему коду, новые значения весов будут использованы на следующем тике оценки модуля ThreatCalculator.**

**Шаг 11. Конец алгоритма.**

## 4.5 Модуль выбора состояния через функции полезности

Модуль реализован классами `UtilityCurves` и `StateTransitionResolver`. Класс `UtilityCurves` содержит математические функции полезности для пяти состояний конечного автомата. Класс `StateTransitionResolver` выполняет полный цикл выбора оптимального состояния: вычисление полезности всех кандидатов, добавление стоимости перехода, бонуса инерции и модификатора паттерна, после чего возвращает состояние с максимальной итоговой оценкой.

Концепция функций полезности [22] взята из теории принятия решений в условиях неопределенности. Каждому состоянию автомата сопоставлена

скалярная функция от значения итоговой угрозы, возвращающая значение полезности в диапазоне от 0 до приблизительно 1.2. Превышение единицы допустимо для состояния преследования при высокой агрессивности памяти, что позволяет преследованию выигрывать у других состояний при доказанной агрессии игрока.

Для большинства состояний применена колоколообразная функция полезности по формуле:

$$U_{\text{bell}}(T, c, \sigma) = \exp\left(-\frac{(T - c)^2}{2\sigma^2}\right), \quad (4.11)$$

где  $U_{\text{bell}}$  – значение полезности при заданной угрозе;

$T$  – значение итоговой угрозы из структуры оценки угрозы;

$c$  – центр пика полезности, в котором функция достигает значения 1;

$\sigma$  – ширина колокола, определяющая чувствительность к отклонению от центра.

Колоколообразная форма обеспечивает плавный рост полезности при приближении к зоне специализации состояния и плавное убывание при удалении от нее. Состояние с центром 0.5 наиболее активно в умеренной угрозе, состояние с центром 0.3 – в низкой угрозе, состояние с центром 0.6 – в повышенной угрозе.

Для состояния отступления применена сигмоидная функция по формуле:

$$U_{\text{sig}}(T, K, c) = \frac{1}{1 + \exp(-K(T - c))}, \quad (4.12)$$

где  $U_{\text{sig}}$  – значение полезности отступления при заданной угрозе;

$T$  – значение итоговой угрозы;

$K$  – крутизна сигмоиды, заданная значением 12;

$c$  – точка перегиба, заданная значением 0.75.

Сигмоидная форма выбрана для отступления, поскольку отступление не имеет верхней границы оптимума: чем выше угроза, тем выше полезность отступления, без точки убывания. Крутая сигмоида с центром в 0.75 обеспечивает резкий переход от низкой полезности к высокой при пересечении опасного порога угрозы.

Параметры функций полезности для пяти состояний приведены ниже. Функция полезности состояния бездействия вычисляется по формуле:

$$U_{\text{Idle}}(T) = \max(0, 1 - 6T^2), \quad (4.13)$$

где  $U_{\text{Idle}}$  – значение полезности состояния бездействия;

$T$  – значение итоговой угрозы.

Квадратичная форма с коэффициентом 6 обеспечивает быстрое падение

полезности бездействия при появлении любой угрозы. При значении угрозы 0.4 полезность бездействия уже равна нулю, что исключает бездействие в боевой обстановке.

Функция полезности состояния преследования вычисляется по формуле:

$$U_{\text{Chase}}(T) = U_{\text{bell}}(T, 0.3, 0.2) + M_{\text{aggr}} \cdot P_{\text{push}}, \quad (4.14)$$

где  $U_{\text{Chase}}$  – значение полезности состояния преследования;

$U_{\text{Bell}}$  – колоколообразная функция по формуле (4.11);

$M_{\text{aggr}}$  – значение агрессивности памяти из структуры `ContextData`;

$P_{\text{push}}$  – коэффициент бонуса агрессивного преследования, зависящий от характера противника: 0.05 для трусливого, 0.4 для обычного, 0.7 для героического.

Бонус по агрессивности памяти добавлен, поскольку подтвержденная агрессия игрока должна вызывать встречное давление со стороны существ сильного характера и осторожное поведение со стороны слабых существ.

Функция полезности состояния атаки вычисляется по формуле:

$$U_{\text{Attack}}(T) = U_{\text{bell}}(T, 0.5, 0.18), \quad (4.15)$$

где  $U_{\text{Attack}}$  – значение полезности состояния атаки.

Узкая ширина колокола 0.18 выбрана преднамеренно. При исходном значении 0.25 состояние атаки доминировало во всем диапазоне итоговой угрозы от 0.35 до 0.65, перекрывая зону работы состояния фланкирования. Сужение колокола обеспечило сосуществование двух состояний.

Функция полезности состояния фланкирования вычисляется по формуле:

$$U_{\text{Flank}}(T) = U_{\text{bell}}(T, 0.6, 0.2) \cdot (A_{\text{solo}} + (1 - A_{\text{solo}}) \cdot A_{\text{norm}}), \quad (4.16)$$

где  $U_{\text{Flank}}$  – значение полезности фланкирующего обхода;

$A_{\text{solo}}$  – базовое значение полезности при отсутствии союзников, заданное значением 0.75;

$A_{\text{norm}}$  – нормализованное число союзников в диапазоне от 0 до 1.

Введение базового значения 0.75 решило проблему одиночных существ. При исходном значении 0.5 одиночная сущность получала полезность фланкирования не выше 0.5, тогда как полезность атаки могла достигать 1.0. В результате одиночная сущность никогда не уходила во фланкирование, что противоречило ожидаемому тактическому поведению.

Функция полезности состояния отступления вычисляется по формуле:

$$U_{\text{Retreat}}(T) = U_{\text{sig}}(T, 12, 0.75) \cdot B_{\text{cow}} \cdot D_{\text{zone}}, \quad (4.17)$$

где  $U_{\text{Retreat}}$  – значение полезности отступления;

$B_{\text{cow}}$  – множитель характера, равный 1.4 для трусливых существ и 1.0 для остальных;

$D_{\text{zone}}$  – штраф за уход в безопасную зону: 1.0 при дистанции внутри зоны риска и 0.4 при выходе за ее пределы.

Штраф за безопасную зону предотвращает бесконечное отступление. После того как существо отдалилось на 150 единиц от игрока (для трусливых – 200 единиц), полезность отступления снижается в 2.5 раза и существо возвращается к другим состояниям.

Графики пяти функций полезности при параметрах из таблицы базовой конфигурации представлены на рисунке 4.1. По оси абсцисс отложено значение итоговой угрозы, по оси ординат – значение функции полезности.

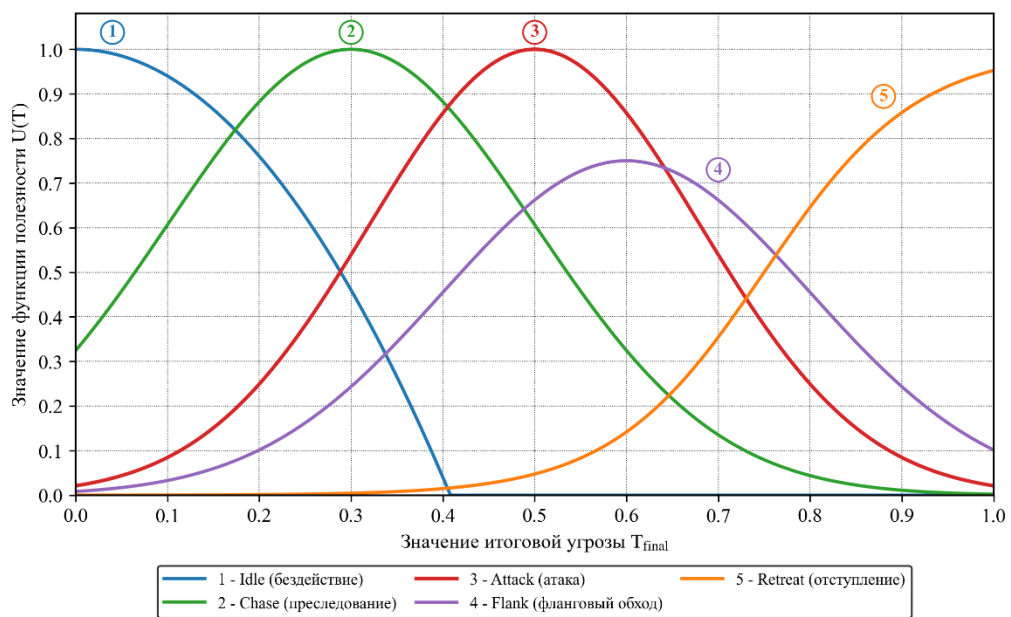


Рисунок 4.1 – Функции полезности пяти состояний адаптивного модуля

Итоговая оценка состояния, по которой производится выбор, вычисляется по формуле:

$$\text{Score}(s) = U(s) - C(s_{\text{curr}}, s) + I(s_{\text{curr}}, s) + M_{\text{pattern}}(s), \quad (4.18)$$

где  $\text{Score}(s)$  – итоговая оценка состояния-кандидата  $s$ ;

$U(s)$  – значение функции полезности состояния  $s$  по одной из формул (4.13) – (4.17);

$C(S_{\text{curr}}, s)$  – стоимость перехода из текущего состояния в  $s$ , читается из матрицы стоимостей;

$I(S_{\text{curr}}, s)$  – бонус инерции за продолжение пребывания в текущем состоянии;

$M_{\text{pattern}}(s)$  – модификатор паттерна по таблице 4.2.

Матрица стоимостей переходов размером 6 на 6 хранит штрафы за смену

состояния и предотвращает чрезмерно частые переключения. Бонус инерции возрастает линейно со временем пребывания в состоянии со скоростью 0.02 в секунду и ограничен сверху значением 0.10, что соответствует пяти секундам максимального удержания.

Размерность 6 объясняется тем, что перечисление `EFSMStateType` содержит шесть значений: одно служебное значение `None` с индексом 0 и пять боевых состояний с индексами от 1 до 5. Служебное значение применяется как маркер незаданного состояния в момент создания компонента, до вызова метода инициализации, а также при программных ошибках обращения к незарегистрированному состоянию. В первой строке и первом столбце матрицы расположены стоимости переходов из служебного состояния и в служебное состояние соответственно, все шесть пар таких переходов заданы значением 1.0. Это максимальный возможный штраф, при котором утилитарная оценка состояния заведомо проигрывает любому реальному кандидату, что исключает возможность попадания сущности в незаданное состояние во время работы программы. Конкретные значения приведены в таблице 4.3. Диагональ матрицы заполнена нулями, что отражает отсутствие штрафа при сохранении текущего состояния. Наибольшие штрафы среди боевых переходов (0.25 и 0.20) назначены тактически противоположным переходам: из отступления во фланкирование и из атаки в отступление. Бонус инерции возрастает линейно со временем пребывания в состоянии со скоростью 0.02 в секунду и ограничен сверху значением 0.10, что соответствует пяти секундам максимального удержания.

Таблица 4.3 – Матрица стоимостей переходов между состояниями

| Из / В  | None | Idle | Chase | Attack | Retreat | Flank |
|---------|------|------|-------|--------|---------|-------|
| None    | 1.00 | 1.00 | 1.00  | 1.00   | 1.00    | 1.00  |
| Idle    | 1.00 | 0.00 | 0.05  | 0.10   | 0.05    | 0.15  |
| Chase   | 1.00 | 0.10 | 0.00  | 0.05   | 0.15    | 0.10  |
| Attack  | 1.00 | 0.15 | 0.10  | 0.00   | 0.20    | 0.15  |
| Retreat | 1.00 | 0.10 | 0.15  | 0.20   | 0.00    | 0.25  |
| Flank   | 1.00 | 0.15 | 0.10  | 0.10   | 0.15    | 0.00  |

Для алгоритма выбора оптимального состояния реализован метод `SelectBestState()`, охватывающий перебор пяти кандидатов и применение формулы (4.18).

Входные данные алгоритма выбора оптимального состояния:

- `CurrentState` – текущее состояние конечного автомата;
- `Threat` – структура `ThreatAssessment` с полем итоговой угрозы;
- `Context` – структура `ContextData` с шестью факторами;
- `TimeInCurrentState` – время пребывания в текущем состоянии, секунд;
- `Config` – ссылка на конфигурацию `AdaptiveConfig`.

Алгоритм выбора оптимального состояния по шагам:

Шаг 1. Начало алгоритма.

Шаг 2. Подготовка списка состояний-кандидатов из пяти элементов: бездействие, преследование, атака, отступление, фланкирование.

```
const TArray<EFSMStateType> StatesToEvaluate = {  
    EFSMStateType::Idle,  
    EFSMStateType::Chase,  
    EFSMStateType::Attack,  
    EFSMStateType::Retreat,  
    EFSMStateType::Flank  
};
```

Шаг 3. Инициализация переменных `BestState` значением текущего состояния и `BestScore` значением минус бесконечность.

Шаг 4. Запуск цикла по списку состояний-кандидатов.

Шаг 5. Вычисление значения функции полезности кандидата по одной из формул (4.13)–(4.17).

```
Score.UtilityValue = UtilCurves->EvaluateUtility(  
    Candidate, Threat, Context, Cfg);
```

Шаг 6. Чтение стоимости перехода из текущего состояния в кандидата из матрицы 6 на 6.

```
Score.TransitionCost = CostMatrix->GetCost(CurrentState,  
Candidate);
```

Шаг 7. Расчет бонуса инерции при условии равенства кандидата текущему состоянию.

```
Score.InertiaBonus = CostMatrix->CalculateInertia(  
    CurrentState, Candidate, TimeInCurrentState, Config);
```

Шаг 8. Чтение модификатора паттерна из таблицы 4.2 для пары доминирующий паттерн – кандидат.

```
Score.PatternModifier = PatternRecog-  
>GetPatternModifier(Candidate);
```

Шаг 9. Вычисление итоговой оценки кандидата по формуле (4.18).

```
Score.FinalScore = Score.UtilityValue - Score.TransitionCost  
    + Score.InertiaBonus + Score.PatternModifier;
```

Шаг 10. Сохранение оценки в массиве `OutScores` для последующего вывода в виджете отладки.

Шаг 11. Сравнение итоговой оценки с текущим максимумом и

обновление `BestState` при превышении.

Шаг 12. Переход к следующему кандидату до завершения перебора пяти состояний.

Шаг 13. Применение смещения по характеру противника к значению итоговой угрозы перед перебором, модификатор принимает значения от -0.3 для трусливого до +0.5 для героического, что обеспечивает индивидуальную манеру боя сущностей одного класса.

Шаг 14. Проверка условия гистерезиса для перехода из отступления в бездействие: если с момента выхода из отступления прошло менее 2.5 секунд, переход блокируется во избежание цикла бездействие – отступление.

Шаг 15. Возврат состояния `BestState` вызывающему коду модуля адаптивного поведения.

Шаг 16. Конец алгоритма.

## 4.6 Модуль процедурной генерации комнат

Модуль реализован классом `RoomBase` и подсистемой генерации в составе игрового режима. Комнаты соединяются через двери в северной, южной, восточной и западной стенах. На входе игрока в комнату срабатывает триггер `OnRoomEntered`, запускающий закрытие дверей и создание сущностей из конфигурации.

Для построения подземелья применен алгоритм последовательного присоединения комнат к стартовой ячейке. Каждая новая ячейка располагается в одном из четырех направлений относительно предыдущей, выбор направления выполняется псевдослучайным образом с проверкой отсутствия пересечений. Псевдослучайный выбор направления реализован через `FMath::RandRange` с инициализацией генератора текущим временем, что гарантирует уникальность подземелья при каждом запуске уровня. Длина подземелья задана в конфигурации игрового режима параметром `RoomChainLength`. Параметр `RoomChainLength` по умолчанию равен 8 и регулирует продолжительность одного забега; меньшие значения применяются в отладочных режимах, большие – для тестов выносливости адаптивной системы.

Активация комнаты происходит только при первом входе игрока. Последующие посещения не вызывают повторного создания сущностей и не закрывают двери. Состояние активации сохраняется в поле `bWasActivated` сущности комнаты. Состояние `bWasActivated` сохраняется только на время одной игровой сессии и сбрасывается при перезапуске забега.

Для алгоритма генерации и активации комнат реализован метод `GenerateAndActivate()`, охватывающий построение цепочки ячеек и обработку входа игрока.

Входные данные алгоритма генерации и активации комнат:

- `StartCell` – стартовая ячейка с фиксированной позицией (0, 0);
- `RoomChainLength` – количество комнат в подземелье, заданное в конфигурации игрового режима;

- EnemySpawnConfig – конфигурация спавна существ, содержащая классы и числа противников;

- Player – ссылка на игрового персонажа.

Алгоритм генерации и активации комнат по шагам:

Шаг 1. Начало алгоритма.

Шаг 2. Создание стартовой ячейки в точке (0, 0) и установка ее в качестве текущей в процессе построения цепочки.

```
ARoomBase* Current = GetWorld()->SpawnActor<ARoomBase>(...);
```

Шаг 3. Запуск цикла построения от 1 до RoomChainLength - 1.

Шаг 4. Выбор псевдослучайного направления из четырех вариантов: север, юг, восток, запад.

```
EDirection Dir = static_cast<EDirection>(FMath::RandRange(0, 3));
```

Шаг 5. Проверка отсутствия ячейки в выбранном направлении, при наличии – выбор другого направления, при четырех безуспешных попытках – возврат к предыдущей ячейке. Откат к предыдущей ячейке после четырех неудачных попыток предотвращает заикливание алгоритма в тупиковых конфигурациях, где все четыре соседние позиции уже заняты.

Шаг 6. Создание новой ячейки в выбранной позиции и установка связи между дверями смежных ячеек.

```
ARoomBase* Next = GetWorld()->SpawnActor<ARoomBase>(...);  
Current->ConnectDoor(Dir, Next);
```

Шаг 7. Обновление текущей ячейки на новую и переход к следующей итерации цикла.

Шаг 8. Завершение цикла после спавна заданного числа комнат и размещение люка перехода на следующий уровень в последней ячейке.

Шаг 9. Подписка каждой ячейки на событие пересечения триггера входа игроком.

```
Cell->OnRoomEntered.AddDynamic(  
    this, &ARoomBase::HandlePlayerEntered);
```

Шаг 10. Срабатывание триггера при пересечении игроком границы комнаты в первый раз.

Шаг 11. Проверка флага bWasActivated, при истинном значении – ранний выход без повторного появления.

Шаг 12. Установка флага bWasActivated в истину и закрытие всех дверей комнаты через изменение коллизий и спрайтов дверей.

```
bWasActivated = true;
```



```
for (UDoorComponent* Door : Doors) Door->Close();
```

**Шаг 13. Спавн сущностей из EnemySpawnConfig в заранее размеченных точках спавна, расположенных по углам комнаты.**

```
for (const FEnemySpawnEntry& Entry : Config.Spawns) {  
    GetWorld()->SpawnActor<ABaseEnemy>(Entry.EnemyClass,  
...);  
}
```

**Шаг 14. Подписка на событие поражения каждой созданной сущности для подсчета оставшихся противников.**

**Шаг 15. Открытие всех дверей при достижении нулевого числа живых сущностей в комнате. Подсчет оставшихся сущностей через декремент счетчика, а не повторное сканирование комнаты, исключает зависимость производительности от числа противников.**

```
if (RemainingEnemies == 0)  
    for (UDoorComponent* Door : Doors) Door->Open();
```

**Шаг 16. Конец алгоритма.**

## 5 ПРОГРАММА И МЕТОДИКА ИСПЫТАНИЙ

В разделе приведены результаты тестирования программного обеспечения. Тестирование охватывает функциональность игрового интерфейса, систему метапрогрессии, процедурную генерацию уровней, боевую систему с модификаторами и адаптивный модуль поведения врагов.

Тестирование проводилось методом черного ящика: для каждого сценария формулировались входные условия и ожидаемый результат.

Для каждого сценария указан ожидаемый результат, фактически полученный результат, приведена иллюстрация работы соответствующей подсистемы. Часть сценариев сопровождается выводом виджета отладки `DebugAdaptiveWidget`, отображающего значения шести факторов контекста и итоговой угрозы, текущих весов и распознанного паттерна поведения игрока.

### 5.1 Главное меню и переход в хаб

После запуска приложения пользователь попадает в главное меню, реализованное виджетом `MainMenuWidget`. Меню содержит три основные кнопки: «Play», «Settings» и «Quit». Кнопка «Play» переводит игрока на уровень хаба, где собрана метапрогрессия и расположены кнопки улучшений. Кнопка «Settings» открывает дочерний виджет `MainMenuSettingsWidget` с разделами графики, звука и управления. Кнопка «Quit» вызывает завершение игрового процесса.

Для проверки функционала главного меню необходимо запустить упакованную сборку и последовательно проверить отклик каждой кнопки, корректность загрузки уровня хаба и работу настройки. Результаты тестирования приведены в таблице 5.1.

Таблица 5.1 – Результаты тестирования главного меню

| Тест                                | Ожидаемый результат                                                 | Полученный результат |
|-------------------------------------|---------------------------------------------------------------------|----------------------|
| Запуск приложения                   | Открывается виджет главного меню с тремя кнопками и фоновой музыкой | Соответствует        |
| Нажатие кнопки «Play»               | Загружается уровень хаба и видны кнопки улучшений                   | Соответствует        |
| Нажатие кнопки «Settings»           | Открывается виджет настроек вместо главного меню с настройкой звука | Соответствует        |
| Изменение громкости и подтверждение | Уровень громкости меняется сразу                                    | Соответствует        |
| Нажатие кнопки «Quit»               | Игровое приложение завершает работу без ошибок                      | Соответствует        |

Все тесты главного меню пройдены успешно. Функциональность виджета соответствует требованиям спецификации интерфейса.

## 5.2 Покупка улучшений в хабе

В хабе расположены четыре терминала постоянных улучшений: усиление урона (Damage), увеличение дальности атаки ближнего боя (Range), повышение числа одновременных стрел при выстреле (ArrowCount) и увеличение максимального здоровья (MaxHP). Каждое улучшение имеет уровень от 0 до 5, за исключением улучшения числа стрел, ограниченного значением 3. Для проверки системы улучшений необходимо иметь в профиле игрока запас алмазов, превышающий стоимость выбранного улучшения. После нажатия кнопки покупки алмазы списываются с профиля, уровень улучшения возрастает на единицу, числовое значение в карточке улучшения обновляется, а статистика игрока изменяется на величину, заданную множителем уровня. Покупки сохраняются в базе данных SQLite, что подтверждается перезапуском приложения и повторным открытием хаба. Результаты тестирования приведены в таблице 5.2.

Таблица 5.2 – Результаты тестирования покупки улучшений в хабе

| Тест                                            | Ожидаемый результат                                                                                    | Полученный результат |
|-------------------------------------------------|--------------------------------------------------------------------------------------------------------|----------------------|
| Покупка уровня улучшения урона                  | Уровень увеличивается на 1, алмазы списываются, в карточке отображается новое значение множителя       | Соответствует        |
| Покупка уровня улучшения максимального здоровья | Максимальное здоровье возрастает на 10% за уровень                                                     | Соответствует        |
| Покупка уровня улучшения числа стрел до 3       | Число одновременно выпускаемых стрел возрастает с 1 до 4 (1 + уровень)                                 | Соответствует        |
| Попытка покупки при недостатке алмазов          | Кнопка не реагирует на нажатие                                                                         | Соответствует        |
| Перезапуск приложения после покупки             | Уровни улучшений и остаток алмазов восстанавливаются из таблицы player_profile базы данных Depthrun.db | Соответствует        |

Функциональность системы улучшений работает корректно. Изменения параметров персонажа применяются как в хабе, так и в последующем забеге, что подтверждает корректность связи между DepthrunSaveSubsystem и игровыми компонентами персонажа. Терминал улучшений в уровне хаба

после покупки трех улучшений стрел представлен на рисунке 5.1.



Рисунок 5.1 – Терминал улучшений в хабе после покупки трех уровней стрел

### 5.3 Запуск забега и активация комнат

При нажатии кнопки начала забега в хабе игрок переносится на игровой уровень, где подсистема RoomGeneratorSubsystem процедурно строит цепочку комнат из шаблонов. Стартовая комната не содержит противников. При первом пересечении игроком границы комнаты появляются противники из конфигурации шаблона, затем закрываются двери. После поражения последнего противника двери открываются, подсистема обновляет счетчик зачищенных комнат в HUD и генерирует сундук с определенным шансом. В комнате типа босс после зачистки появляется люк завершения забега. Результаты тестирования приведены в таблице 5.3.

Таблица 5.3 – Результаты тестирования активации комнат

| Тест                                | Ожидаемый результат                                                   | Полученный результат |
|-------------------------------------|-----------------------------------------------------------------------|----------------------|
| Вход в первую комнату               | Триггер не срабатывает, двери остаются открытыми, враги не появляются | Соответствует        |
| Вход в боевую комнату               | Появляются противники из шаблона, двери закрываются                   | Соответствует        |
| Повторный вход в пройденную комнату | Враги не появляются повторно, двери остаются открытыми                | Соответствует        |
| Уничтожение всех врагов в комнате   | Двери открываются, игрок может перейти в соседнюю комнату             | Соответствует        |
| Зачистка комнаты босса              | Появляется люк завершения забега                                      | Соответствует        |

Все сценарии пройдены без замечаний. Повторное появление противников при входе в уже пройденную комнату отсутствует. Счетчик зачищенных комнат в HUD обновляется в момент зачистки. Боевая комната с закрытыми дверями и активными противниками представлен на рисунке 5.2.



Рисунок 5.2 – Боевая комната с закрытыми дверями и активными противниками

#### 5.4 Открытие сундука и получение награды

По завершении зачистки комнаты в ней активируется сундук с наградой. При приближении игрока сундук открывается, воспроизводятся визуальные эффекты, а виджет `ChestRewardWidget` отображает список полученных предметов с иконками. Алмазы из сундука зачисляются на счет игрока и сохраняются в таблице `player_profile` базы данных `Depthrun.db`. Так же игрок получает зелья лечения и модификатор, который действует только в рамках текущего забега. Результаты тестирования приведены в таблице 5.4.

Таблица 5.4 – Результаты тестирования открытия сундука

| Тест                          | Ожидаемый результат                                                              | Полученный результат |
|-------------------------------|----------------------------------------------------------------------------------|----------------------|
| Приближение к сундуку         | Сундук открывается, воспроизводится анимация и визуальный эффект системы Niagara | Соответствует        |
| Отображение виджета награды   | Виджет показывает список предметов с иконками и их количество                    | Соответствует        |
| Закрытие виджета нажатием ЛКМ | Виджет скрывается, управление возвращается игроку                                | Соответствует        |
| Проверка баланса алмазов      | Счетчик алмазов текущего забега увеличивается на значение из сундука             | Соответствует        |

Система наград сундука работает корректно. Алмазы накапливаются в счетчике текущего забега. Виджет наград сундука представлен на рисунке 5.3.

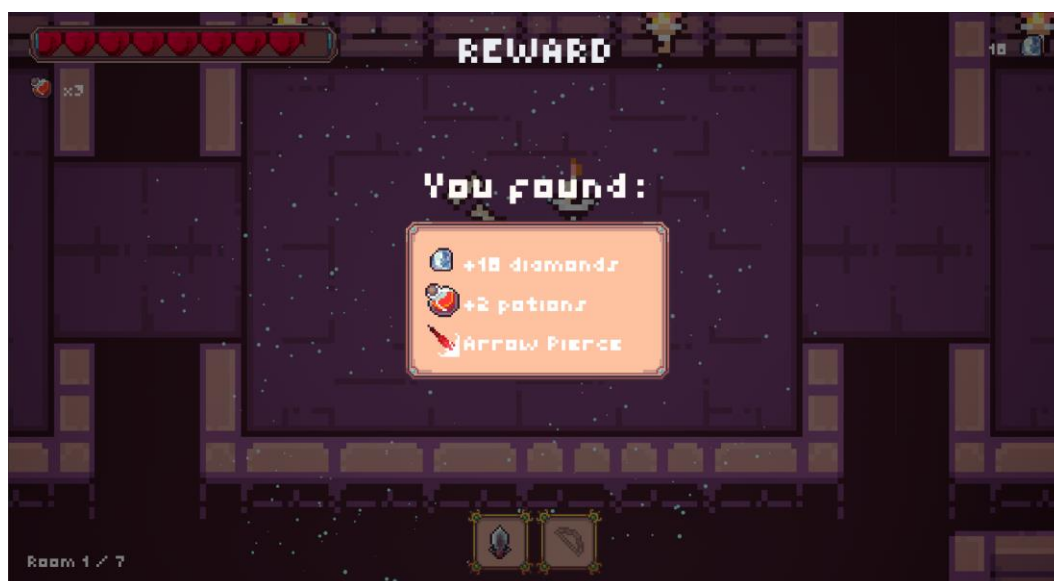


Рисунок 5.3 – Виджет награды сундука с перечнем полученных наград

## 5.5 Модификатор рикошета стрелы

Модификатор рикошета активируется при получении из сундука. С этим модификатором выпущенная стрела при попадании в противника отражается и продолжает полет к следующему врагу, нанося урон второй и третьей целям. Количество рикошетов ограничено значением `MaxRicochetCount` из конфигурации. Результаты тестирования приведены в таблице 5.5.

Таблица 5.5 – Результаты тестирования рикошета стрелы

| Тест                                     | Ожидаемый результат                               | Полученный результат |
|------------------------------------------|---------------------------------------------------|----------------------|
| Выстрел без модификатора                 | Стрела исчезает после первого попадания           | Соответствует        |
| Рикошет в противника                     | Три попадания в трех противников наносят урон     | Соответствует        |
| Превышение <code>MaxRicochetCount</code> | Стрела исчезает после достижения лимита рикошетов | Соответствует        |

Механика рикошета работает корректно. После попадания стрела летит к следующему врагу и наносит урон, лимит рикошетов соблюдается.

## 5.6 Модификатор мультивыстрела

Модификатор мультивыстрела увеличивает число одновременно выпускаемых стрел. Базовое число стрел равно 1 плюс уровень улучшения

ArrowCount, приобретенного в хабе. При активном модификаторе к этому числу добавляется дополнительная стрела. Стрелы выпускаются с угловым смещением относительно центральной, что обеспечивает веерный разброс. Результаты тестирования приведены в таблице 5.6.

Таблица 5.6 – Результаты тестирования мультивыстрела

| Тест                                             | Ожидаемый результат                                                       | Полученный результат |
|--------------------------------------------------|---------------------------------------------------------------------------|----------------------|
| Выстрел без модификатора, уровень ArrowCount = 0 | Выпускается одна стрела                                                   | Соответствует        |
| Выстрел без модификатора, уровень ArrowCount = 2 | Выпускается три стрелы с угловым смещением                                | Соответствует        |
| Выстрел с активным модификатором мультивыстрела  | Число стрел увеличивается на одну, суммарный разброс сохраняется          | Соответствует        |
| Достижение максимального уровня ArrowCount = 3   | Выпускается четыре стрелы, кнопка покупки улучшения становится неактивной | Соответствует        |

Модификатор мультивыстрела взаимодействует с уровнем улучшения ArrowCount корректно. Суммарное число стрел не превышает допустимый максимум. Выстрел из лука с модификатором мультивыстрела отображен на рисунке 5.4.



Рисунок 5.4 – Выстрел из лука с модификатором мультивыстрела и третьим уровнем улучшения ArrowCount



## 5.7 Реакция адаптивной системы на изменение здоровья

Тест проверяет реакцию адаптивной системы на снижение здоровья противника. При уменьшении здоровья ниже 50% значение фактора  $H_{\text{norm}}$  резко возрастает вследствие квадратичного преобразования с показателем степени 2, что должно привести к увеличению итоговой угрозы  $T_{\text{final}}$  и смещению выбора состояния в пользу Retreat или Flank. Тест проводится с открытым виджетом отладки DebugAdaptiveWidget. Результаты тестирования приведены в таблице 5.7.

Таблица 5.7 – Реакция системы на изменение здоровья противника

| Состояние здоровья | Ожидаемый $H_{\text{norm}}$ | Полученный $H_{\text{norm}}$ | Активное состояние |
|--------------------|-----------------------------|------------------------------|--------------------|
| 100%               | 0.05                        | Соответствует                | Idle / Chase       |
| 70%                | 0.06                        | Соответствует                | Chase / Attack     |
| 50%                | 0.14                        | Соответствует                | Chase / Attack     |
| 30%                | 0.21                        | Соответствует                | Retreat / Flank    |
| 20%                | 0.27                        | Соответствует                | Retreat            |

Значения  $H_{\text{norm}}$  соответствуют формуле (4.2). Состояния Retreat и Flank активируются при падении здоровья ниже 30%, что соответствует проектной логике. Реакция системы при снижении здоровья отображена на рисунке 5.5.

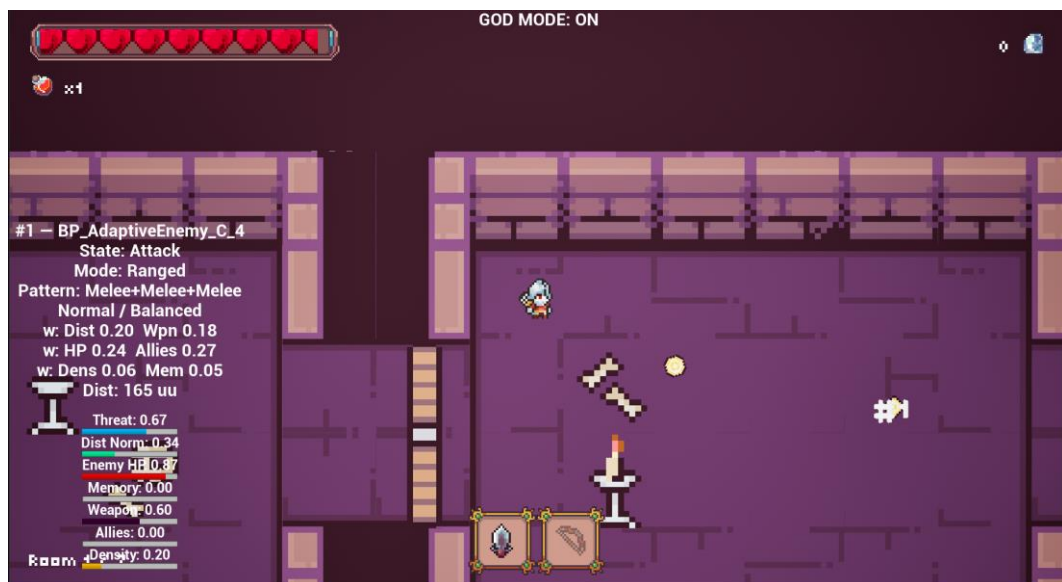


Рисунок 5.5 – Виджет отладки при здоровье противника 30%:  $H_{\text{norm}} = 0.21$ , активно состояние Retreat

## 5.8 Реакция адаптивной системы на изменение дистанции

Тест проверяет изменение фактора  $D_{\text{norm}}$  при перемещении игрока



относительно противника. Максимальная боевая дистанция  $MaxEngagementRange$  составляет 250 юнитов. При нахождении игрока в непосредственной близости  $D_{norm}$  стремится к 1.0, при удалении свыше 200 юнитов – к 0.0. Тест проводится с активным виджетом отладки. Результаты тестирования приведены в таблице 5.8.

Таблица 5.8 – Реакция системы на изменение дистанции

| Дистанция,<br>юнитов | Ожидаемый<br>$D_{norm}$ | Полученный<br>$D_{norm}$ | Активное состояние |
|----------------------|-------------------------|--------------------------|--------------------|
| 35                   | 90                      | Соответствует            | Attack             |
| 80                   | 0.67                    | Соответствует            | Chase / Attack     |
| 120                  | 0.36                    | Соответствует            | Chase              |
| 160                  | 0.34                    | Соответствует            | Idle               |
| 220                  | 0.12                    | Соответствует            | Idle               |

Нормализация дистанции работает корректно. При дистанции свыше  $MaxEngagementRange$  фактор  $D_{norm}$  становится 0.0 и не влияет на выбор состояния. Реакция при снижении дистанции отображена на рисунке 5.6.

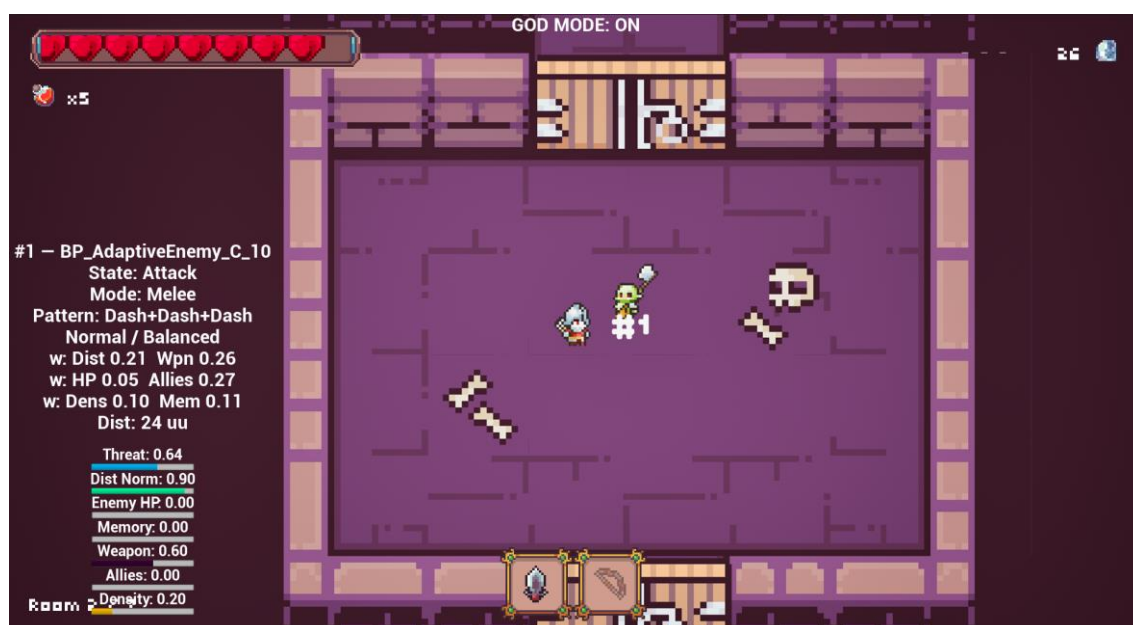


Рисунок 5.6 – Виджет отладки при дистанции 24 юнитов:  $D_{norm} = 0.90$ , активно состояние Attack

## 5.9 Реакция адаптивной системы на число союзников

Тест проверяет изменение фактора  $A_{norm}$  при появлении и поражения союзных противников в радиусе  $AllyCheckRadius$  (250 юнитов).  $A_{norm}$  вычисляется линейно и отображается на отладочном виджете в прямом виде – от 0.0 при отсутствии союзников до 1.0 при достижении порога  $N_{max}$ . При вычислении свертки угрозы  $T_{final}$  значение преобразуется через корневую

инверсию. При  $A_{\text{norm}} = 0$  (союзников нет) вклад максимален и равен 1.0, что повышает угрозу  $T_{\text{final}}$  и активирует агрессивные состояния Chase и Attack. Результаты тестирования приведены в таблице 5.9.

Таблица 5.9 – Реакция системы на число союзников

| Число союзников | $A_{\text{norm}}$ | Полученное значение | Активное состояние |
|-----------------|-------------------|---------------------|--------------------|
| 0               | 0.00              | Соответствует       | Chase / Attack     |
| 1               | 0.25              | Соответствует       | Chase / Attack     |
| 2               | 0.50              | Соответствует       | Chase              |
| 3               | 0.75              | Соответствует       | Chase              |
| 4               | 1.00              | Соответствует       | Idle / Chase       |

Фактор количества союзников работает корректно. При четырех союзниках вклад фактора  $A_{\text{norm}}$  в свертку равен 1.0. Состояние фактора  $A_{\text{norm}}$  врагов при двух союзниках отображено на рисунке 5.7.



Рисунок 5.7 – Виджет при двух союзниках:  $A_{\text{norm}} = 0.5$

## 5.10 Распознавание дальнобойного паттерна

Тест проверяет реакцию модуля PatternRecognizer на серию выстрелов из лука. При повторении действия Shot трижды подряд в скользящем окне из 15 элементов распознаватель должен определить паттерн Shot+Shot+Shot как доминирующий и применить модификаторы из таблицы правил 4.2: +0.65 к состоянию Flank и -0.25 к состоянию Attack. Паттерн считается доминирующим при условии, что его частота в текущем окне превышает частоту всех остальных зарегистрированных N-грамм. При равной частоте нескольких паттернов применяется более актуальный паттерн.

Тест проводится с виджетом отладки. Результаты тестирования

приведены в таблице 5.10.

Таблица 5.10 – Распознавание дальнбойного паттерна

| Условие                        | Ожидаемый паттерн            | Полученный паттерн | Активное состояние |
|--------------------------------|------------------------------|--------------------|--------------------|
| 1 выстрел                      | Shot                         | Соответствует      | Chase / Attack     |
| 2 выстрела подряд              | Shot+Shot                    | Соответствует      | Flank              |
| 3 выстрела подряд              | Shot+Shot+Shot               | Соответствует      | Flank              |
| Смешанные действия после серии | Обновляется на новый паттерн | Соответствует      | По $T_{final}$     |

Распознавание дальнбойного паттерна работает корректно. При серии из трех выстрелов система переключается в состояние Flank, уклоняясь от траектории стрел. Состояние врага при распознанном паттерне Shot+Shot+Shot показано на рисунке 5.8.

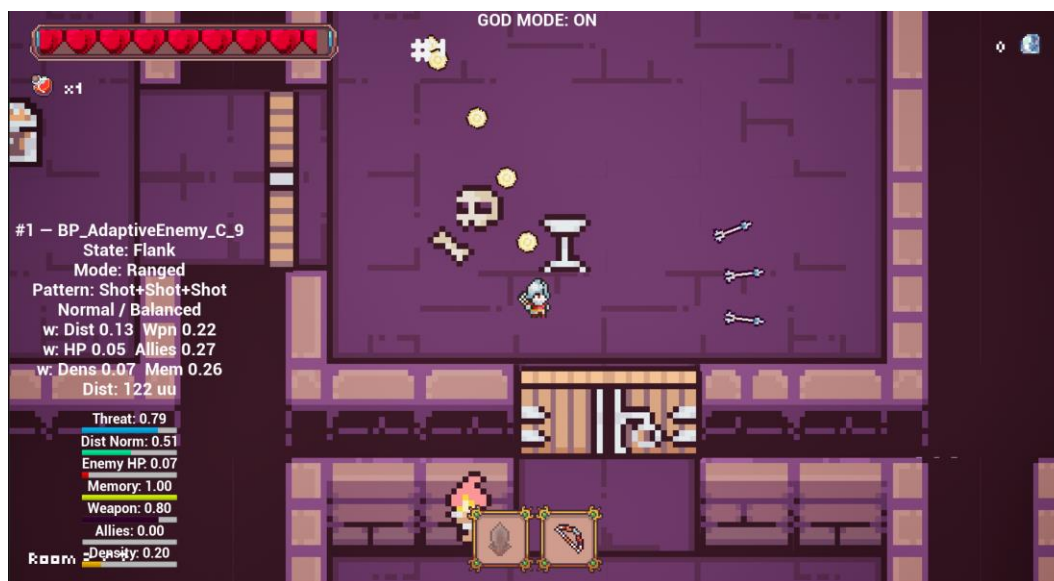


Рисунок 5.8 – Виджет отладки с паттерном Shot+Shot+Shot и активным состоянием Flank

### 5.11 Распознавание агрессивного паттерна

Тест проверяет реакцию системы на паттерн ближнего боя и рывка (Melee+Dash). Согласно таблице правил 4.2, данный паттерн добавляет модификатор +0.25 к состоянию Retreat. Тест проводится следующим образом: игрок выполняет удар мечом с последующим рывком в сторону противника, после чего в виджете отладки проверяется активный паттерн и выбранное состояние. Результаты тестирования приведены в таблице 5.11.

Таблица 5.11 – Распознавание агрессивного паттерна

| Условие              | Ожидаемый паттерн            | Полученный паттерн | Активное состояние |
|----------------------|------------------------------|--------------------|--------------------|
| Одиночный удар мечом | Melee                        | Соответствует      | Attack / Chase     |
| Удар + рывок         | Melee+Dash                   | Соответствует      | Retreat            |
| Два рывка подряд     | Dash+Dash                    | Соответствует      | Chase              |
| Смешанные действия   | Обновляется на новый паттерн | Соответствует      | По $T_{final}$     |

Агрессивный паттерн распознается корректно. Комбинация Melee+Dash усиливает склонность противника к отступлению, что создает тактическую реакцию на ближний бой игрока. Состояние врага при распознанном паттерне Melee+Dash показано на рисунке 5.9.

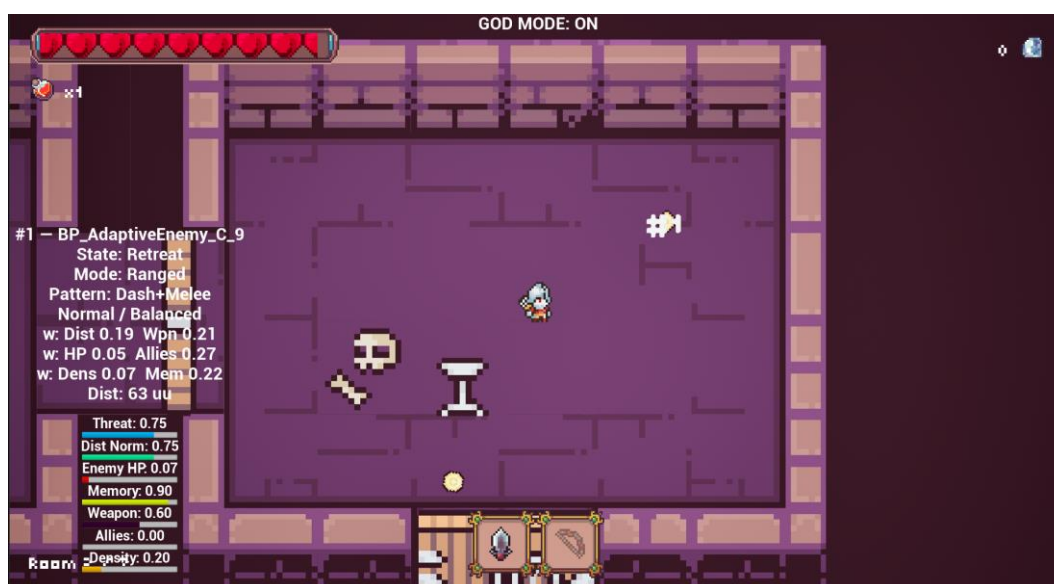


Рисунок 5.9 – Виджет отладки с паттерном Melee+Dash и активным состоянием Retreat

## 5.12 Эволюция весов за десятисекундное окно

Тест проверяет адаптацию весов  $w_0..w_5$  за период активного боя. Начальные значения весов задаются конфигурационным ресурсом DA\_AdaptiveConfig и соответствуют значениям по умолчанию:  $w_0 = 0.20$ ,  $w_1 = 0.20$ ,  $w_2 = 0.20$ ,  $w_3 = 0.15$ ,  $w_4 = 0.15$ ,  $w_5 = 0.10$ . В течение двадцати секунд игрок чередует выстрелы и удары, нанося урон противнику. Каждое нанесение урона противнику присваивает оценку исхода  $r = +1$  для состояний, предшествовавших атаке, каждое получение урона  $r = -1$ . Веса обновляются по формуле (4.8) с шагом обучения  $WeightLearningRate$  и ренормализуются до суммы 1.0. В виджете отладки наблюдается динамическое

перераспределение весов. Результаты тестирования приведены в таблице 5.12.

Таблица 5.12 – Изменение весов за десятисекундное окно активного боя

| Фактор                 | Начальный вес | Вес через 10 с          | Направление изменения        |
|------------------------|---------------|-------------------------|------------------------------|
| Distance ( $w_0$ )     | 0.20          | Соответствует диапазону | Рост при агрессивной тактике |
| WeaponThreat ( $w_1$ ) | 0.20          | Соответствует диапазону | Рост при луке                |
| Health ( $w_2$ )       | 0.20          | Соответствует диапазону | Рост при низком НР           |
| Allies ( $w_3$ )       | 0.15          | Соответствует диапазону | Снижение при одиночном бое   |
| RoomDensity ( $w_4$ )  | 0.10          | Соответствует диапазону | Стабильный                   |
| Memory ( $w_5$ )       | 0.15          | Соответствует диапазону | Рост при серии выстрелов     |

Адаптация весов работает корректно. Сумма весов сохраняется равной 1.0 на всем протяжении наблюдения. Веса не выходят за границы [0.05; 0.40], заданные в таблице 4.1. Перераспределение весов после 10 секунд боя отображено на рисунке 5.10.

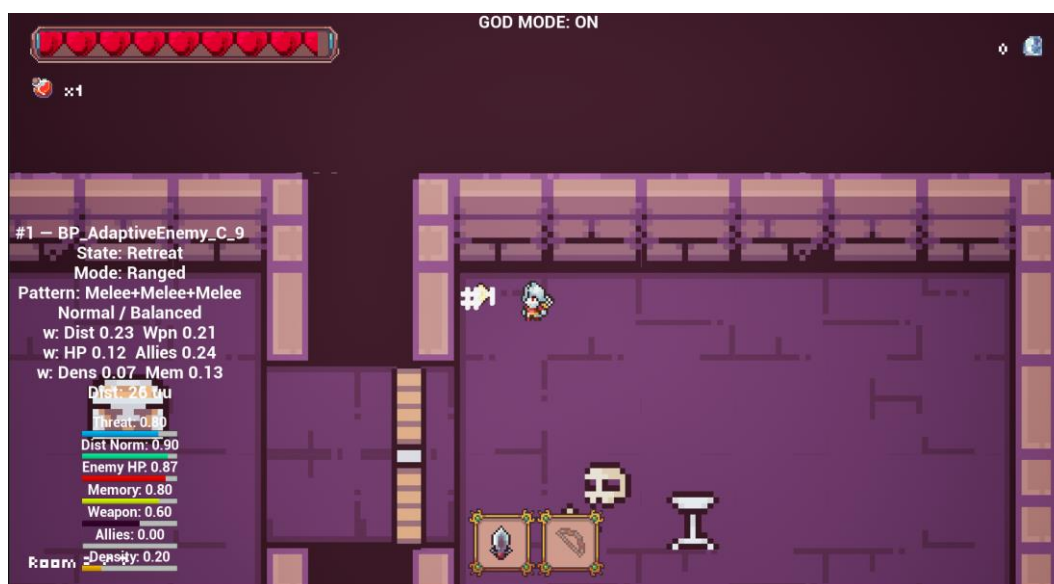


Рисунок 5.10 – Виджет отладки через 10 секунд боя: перераспределение весов в пользу Distance и Memory

### 5.13 A/B-тест адаптивного и неадаптивного FSM

A/B-тест сравнивает поведение противника с включенной адаптивной системой и с отключенной (неадаптивный FSM с фиксированными весами и



без распознавания паттернов). Для каждой конфигурации проводятся два забега с одинаковой тактикой игрока – только выстрелы из лука. В адаптивном режиме после нескольких паттернов Shot+Shot+Shot система должна начать чаще выбирать состояние Flank, тогда как неадаптивный FSM остается в Chase или Attack независимо от тактики, преследует и атакует. Результаты тестирования приведены в таблице 5.13.

Таблица 5.13 – Сравнение адаптивного и неадаптивного FSM

| Параметр                                | Неадаптивный FSM              | Адаптивный FSM                                          |
|-----------------------------------------|-------------------------------|---------------------------------------------------------|
| Реакция на паттерн Shot+Shot+Shot       | Нет изменений, остается Chase | Переход в Flank через 2-3 повторения паттерна           |
| Изменение весов                         | Не происходит                 | Веса перераспределяются по оценке исхода взаимодействия |
| Время реакции на смену тактики игрока   | Нет реакции                   | Реакция в течение 5 секунд                              |
| Среднее число попаданий игрока за забег | Выше (враг предсказуем)       | Ниже (враг адаптируется)                                |

А/В-тест подтвердил ключевое отличие адаптивного модуля от неадаптивного FSM: противник с активной адаптацией меняет предпочтительное состояние в ответ на повторяющуюся тактику игрока, тогда как неадаптивный FSM этого не делает. Сравнение активных состояний при тактике частых выстрелов из лука приведено на рисунке 5.11.



Рисунок 5.11 – Сравнение активных состояний: неадаптивный FSM (слева) и адаптивный (справа) при тактике выстрелов

## 5.14 Экран победы

Экран победы отображается при достижении игроком люка в последней комнате цепочки и активации перехода. Виджет `VictoryScreenWidget` показывает итоговую статистику забега: затраченное время, число пройденных комнат, собранные алмазы и общее количество алмазов. Собранные алмазы зачисляются в профиль игрока в полном объеме и сохраняются в таблице `player_profile`. Запись о забеге с результатом `Won = 1`, временем забега, количеством пройденных комнат и временем системы добавляется в таблицу `run_history`. Результаты тестирования приведены в таблице 5.14.

Таблица 5.14 – Результаты тестирования экрана победы

| Тест                               | Ожидаемый результат                                                                  | Полученный результат |
|------------------------------------|--------------------------------------------------------------------------------------|----------------------|
| Активация люка в последней комнате | Загружается экран победы, воспроизводится Niagara-эффект                             | Соответствует        |
| Отображение статистики             | Показаны число пройденных комнат, алмазы и время забега                              | Соответствует        |
| Зачисление алмазов                 | Баланс профиля увеличивается на сумму, указанную на экране и пишется в общих алмазах | Соответствует        |
| Нажатие кнопки «Hub»               | Загружается уровень хаба с обновленным количеством алмазов                           | Соответствует        |

Экран победы работает корректно. Статистика соответствует данным из таблицы `run_history` базы данных `Depthrun.db`. Экран победы приведен на рисунке 5.12.



Рисунок 5.12 – Экран победы со статистикой забега и Niagara-эффектом

## 5.15 Экран поражения

Экран поражения отображается при обнулении очков здоровья персонажа игрока. Виджет `DeathScreenWidget` показывает статистику текущего незавершенного забега: число пройденных комнат и собранные алмазы. Алмазы, набранные до поражения, сохраняются в профиле игрока в 50% объеме. Запись о забеге с результатом `Won = 0` добавляется в таблицу `run_history`. Результаты тестирования приведены в таблице 5.15.

Таблица 5.15 – Результаты тестирования экрана поражения

| Тест                              | Ожидаемый результат                                                                       | Полученный результат |
|-----------------------------------|-------------------------------------------------------------------------------------------|----------------------|
| Поражение персонажа               | Загружается экран поражения, управление заблокировано и воспроизводится визуальный эффект | Соответствует        |
| Отображение статистики            | Показаны число пройденных комнат, собранные алмазы и время забега                         | Соответствует        |
| Сохранение алмазов                | Алмазы, собранные до поражения, зачисляются в профиль                                     | Соответствует        |
| Запись в <code>run_history</code> | Добавляется строка с <code>Won = 0</code> и длительностью забега                          | Соответствует        |
| Нажатие кнопки «Hub»              | Загружается уровень хаба                                                                  | Соответствует        |

Экран поражения работает корректно. Прогресс игрока не теряется: половина алмазов сохраняется, факт поражения фиксируется в базе данных. Экран поражения приведен на рисунке 5.13.

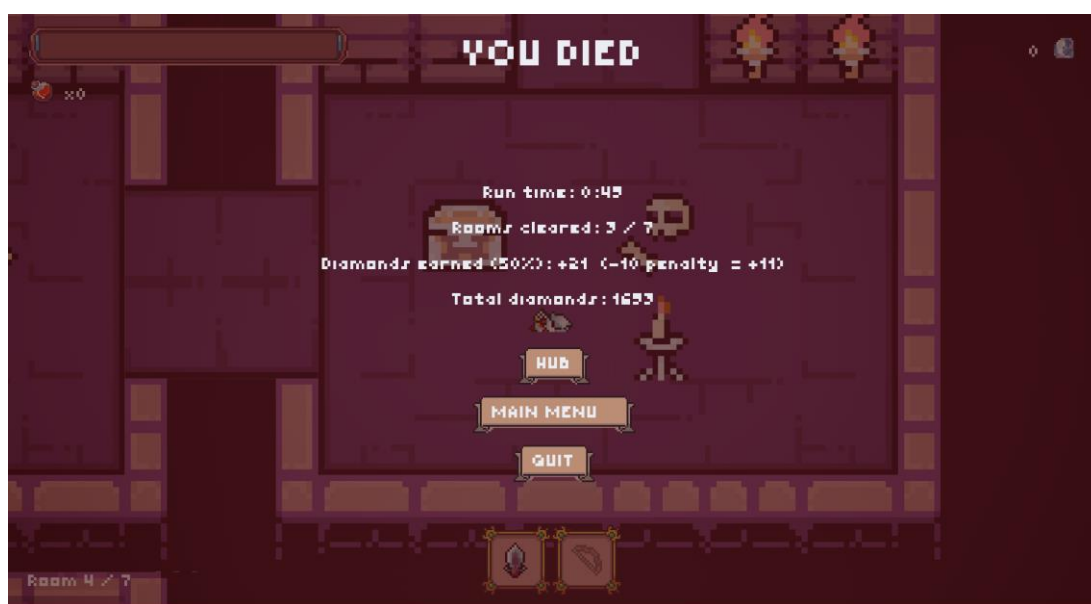


Рисунок 5.13 – Экран поражения с итоговой статистикой незавершенного забега



## 5.16 Сохранение и восстановление метапрогрессии

Тест проверяет полный цикл записи и чтения данных метапрогрессии. После покупки улучшений, завершения забега и получения алмазов приложение принудительно закрывается и запускается повторно. При загрузке хаба подсистема читает таблицы `player_profile` и восстанавливает состояние профиля игрока в полном объеме. Результаты тестирования приведены в таблице 5.16.

Таблица 5.16 – Результаты тестирования сохранения метапрогрессии

| Тест                               | Ожидаемый результат                                                                                        | Полученный результат |
|------------------------------------|------------------------------------------------------------------------------------------------------------|----------------------|
| Перезапуск после покупки улучшений | Уровни улучшений восстановлены из <code>player_profile</code>                                              | Соответствует        |
| Перезапуск после победного забега  | Баланс алмазов увеличен, в <code>run_history</code> добавлена строка с <code>Won = 1</code>                | Соответствует        |
| Перезапуск после поражения         | Баланс алмазов корректен, в <code>run_history</code> добавлена строка с <code>Won = 0</code>               | Соответствует        |
| Проверка целостности базы данных   | Таблицы <code>player_profile</code> и <code>run_history</code> содержат корректные данные без дублирования | Соответствует        |

Метапрогрессия сохраняется и восстанавливается корректно. Данные между сессиями не теряются и не дублируются.

Тестирование подтвердило соответствие реализованной системы заявленным требованиям. Все шестнадцать сценариев тестирования пройдены без замечаний. Адаптивный модуль поведения врагов корректно реагирует на изменение шести факторов контекста, распознает паттерны действий игрока через анализ N-грамм, перераспределяет веса значимости факторов по результатам каждого цикла оценки и выбирает оптимальное состояние через функции полезности. Сопутствующие подсистемы (процедурная генерация уровней, метапрогрессия в SQLite, система модификаторов забега) обеспечивают полноценный игровой контур.

## 6 РУКОВОДСТВО ПОЛЬЗОВАТЕЛЯ

### 6.1 Системные требования

Программа разработана на движке Unreal Engine 5.7. Игровая сцена реализована в двухмерной проекции, что существенно снижает требования к графической подсистеме.

Для запуска программы достаточно двухъядерного процессора с тактовой частотой не ниже 3,0 ГГц и видеокарты с поддержкой DirectX 11. Указанная тактовая частота обусловлена тем, что расчет логики адаптивного поведения врагов выполняется последовательно в одном потоке.

Для установки программы требуется не менее 1 ГБ свободного места на диске. Рекомендуемый объем составляет 2 ГБ. Разрешение 1280×720 обеспечивает читаемость интерфейса; при 1920×1080 все элементы HUD отображаются без масштабирования.

Приведенные требования сведены в таблице 6.1.

Таблица 6.1 – Системные требования

| Компонент            | Минимальные требования                                  | Рекомендуемые требования                               |
|----------------------|---------------------------------------------------------|--------------------------------------------------------|
| Операционная система | Windows 10 (64-бит)                                     | Windows 11 (64-бит)                                    |
| Процессор            | Intel Core 2 Duo E8400 / AMD Athlon X2 7750             | Intel Core i3-6100 / AMD Ryzen 3 1200                  |
| Оперативная память   | 4 ГБ                                                    | 8 ГБ                                                   |
| Видеокарта           | NVIDIA GeForce GTX 460 / AMD Radeon HD 6850 (1 ГБ VRAM) | NVIDIA GeForce GTX 750 / AMD Radeon R7 360 (2 ГБ VRAM) |
| DirectX              | Версия 11                                               | Версия 11                                              |
| Место на диске       | 1 ГБ                                                    | 2 ГБ                                                   |
| Разрешение экрана    | 1280×720                                                | 1920×1080                                              |

### 6.2 Краткое руководство пользователя

Для запуска игры необходимо открыть папку с программой и запустить исполняемый файл Depthrun.exe. Предварительная установка, регистрация или подключение к сети не требуются. Все игровые данные, включая историю забегов и накопленные алмазы, сохраняются автоматически в локальную базу данных SQLite.

После запуска программа загружает главное меню уровня. На экране отображаются три кнопки: «Play», «Settings» и «Quit». Нажатие Play переводит игрока в хаб-уровень, где перед каждым забегом отображается экран

метапрогрессии. Настройки доступны через кнопку «Settings» и содержат регулятор громкости. В главном меню воспроизводится фоновая музыкальная тема. Главное меню представлено на рисунке 6.1.



Рисунок 6.1 – Главное меню

Окно настроек открывается как из главного меню, так и из меню паузы. Единственный параметр, доступный в настройках – регулятор громкости со шкалой от 0 до 100, числовое значение отображается рядом со слайдером. Для возврата в предыдущий экран используется кнопка «Back». Окно настроек представлено на рисунке 6.2.

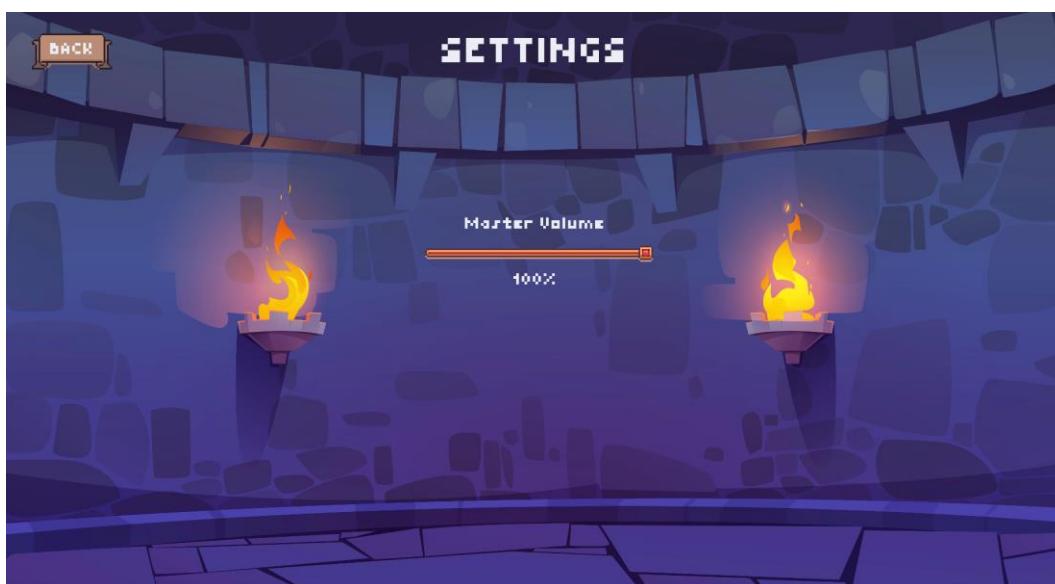


Рисунок 6.2 – Окно настроек

После нажатия «Play» из главного меню программа загружает хаб-уровень. На экране хаба отображается счетчик накопленных алмазов –

основной ресурс метапрогрессии, который сохраняется между сессиями в базе данных. Четыре окна улучшений позволяют перед забегом вложить алмазы в постоянные характеристики: урон, дальность стрельбы, количество стрел и максимальное здоровье. Каждый параметр имеет уровни от 0 до 5, стоимость очередного уровня отображается под кнопкой покупки. Кнопка «Play» запускает процедурную генерацию комнат и переход в подземелье. Кнопка «Back» возвращает на главный экран. Экран хаба представлен на рисунке 6.3.



Рисунок 6.3 – Экран хаба

Управление персонажем осуществляется с клавиатуры и мыши. Перемещение выполняется клавишами «W, A, S, D». Левая кнопка мыши производит атаку активным оружием. Рывок активируется клавишей «Space». Использование зелья лечения закреплено за клавишей «E», переключение между оружием – клавишами «1» (меч) и «2» (лук). Меню паузы открывается и закрывается клавишами «P» или «Escape». Назначение клавиш управления представлено в таблице 6.2.

Таблица 6.2 – Назначение клавиш управления

| Клавиша                 | Действие                      |
|-------------------------|-------------------------------|
| W, A, S, D              | Движение персонажа            |
| ЛКМ (левая кнопка мыши) | Атака активным оружием        |
| Space (Пробел)          | Рывок                         |
| E                       | Использовать зелье лечения    |
| 1                       | Переключиться на меч (Слот 1) |
| 2                       | Переключиться на лук (Слот 2) |
| P / Escape              | Открыть / закрыть меню паузы  |

При запуске забега процедурно генерируется подземелье. Первая комната является стартовой: в ней нет врагов, двери открыты. Последующие

комнаты при входе персонажа блокируют все выходы и порождают группу противников. Двери открываются после уничтожения всех врагов в комнате. В верхней части экрана отображается HUD с полосой здоровья, счетчиком алмазов, количеством зелий, номером очищенных комнат и иконками слотов оружия. В комнатах без активного боя воспроизводится фоновая музыкальная тема подземелья; при обнаружении персонажа врагом она сменяется боевой темой. Игровая сцена представлена на рисунке 6.4.



Рисунок 6.4 – Игровая сцена (стартовая комната)

Персонаж располагает двумя слотами оружия. При активном первом слоте нажатие ЛКМ наносит удар мечом перед персонажем, задевая врагов в радиусе поражения. Слот переключается клавишей 1, текущий активный слот подсвечивается внизу экрана. Атака мечом представлена на рисунке 6.5.

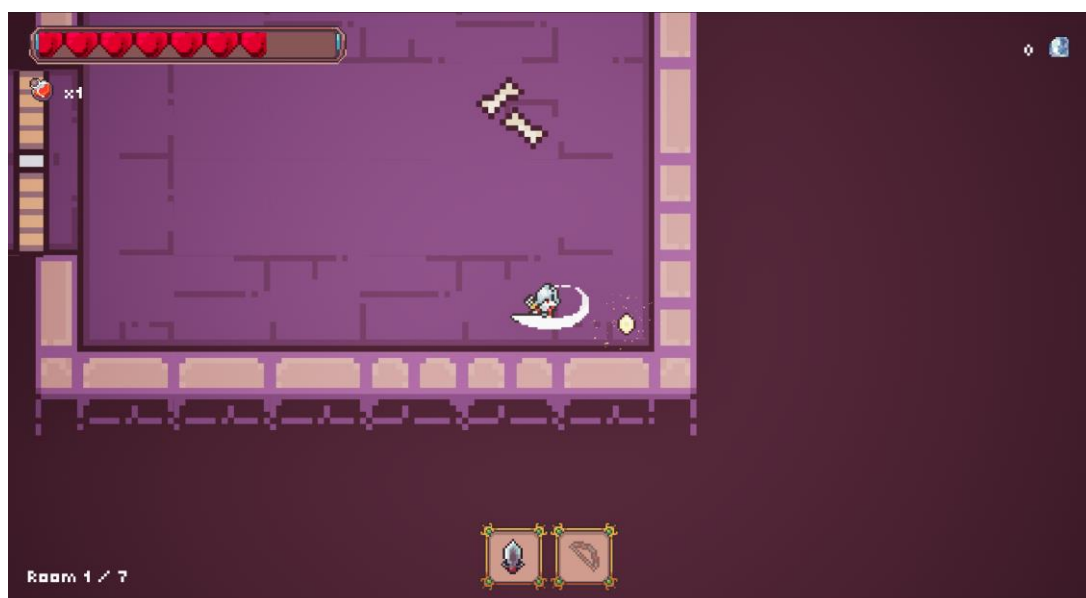


Рисунок 6.5 – Атака мечом

При активном втором слоте нажатие ЛКМ выпускает стрелу в направлении курсора мыши. Снаряд движется по прямой, поражает первого встреченного врага и останавливается при столкновении со стеной. Предметы-модификаторы из сундуков расширяют характеристики лука: «ArrowRicochet» добавляет рикошет от стен до двух раз, «ArrowPierce» позволяет снаряду пробивать врагов насквозь. Стрельба из лука представлена на рисунке 6.6.

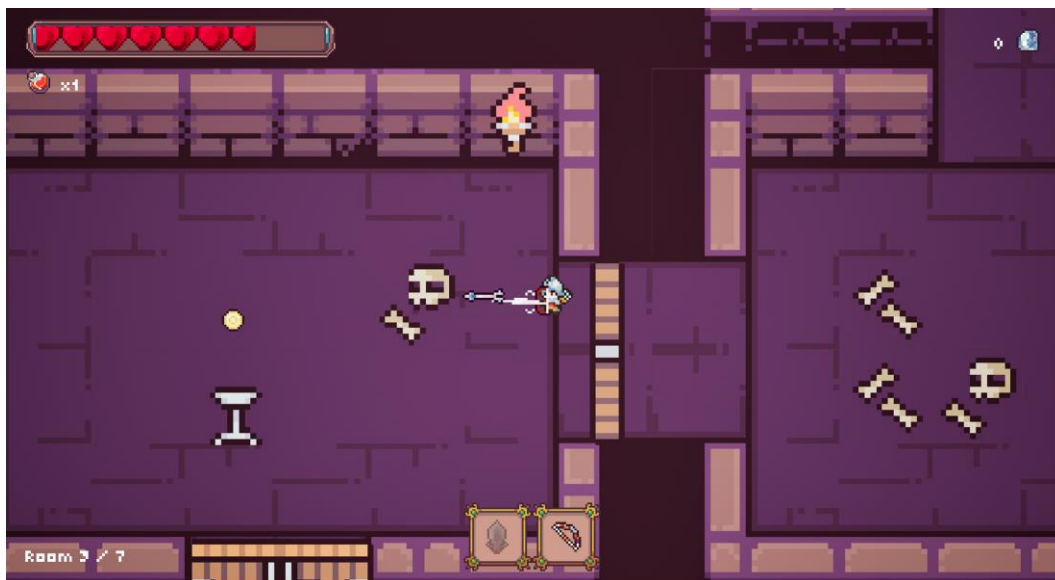


Рисунок 6.6 – Стрельба из лука

После зачистки комнаты с вероятностью 10 % в ее центре появляется сундук. При приближении персонажа сундук открывается, на экране отображается перечень наград. Сундук содержит алмазы и зелья лечения, а с вероятностью 70 % – предмет-модификатор забега. Пример перечня наград сундука представлен на рисунке 6.7.

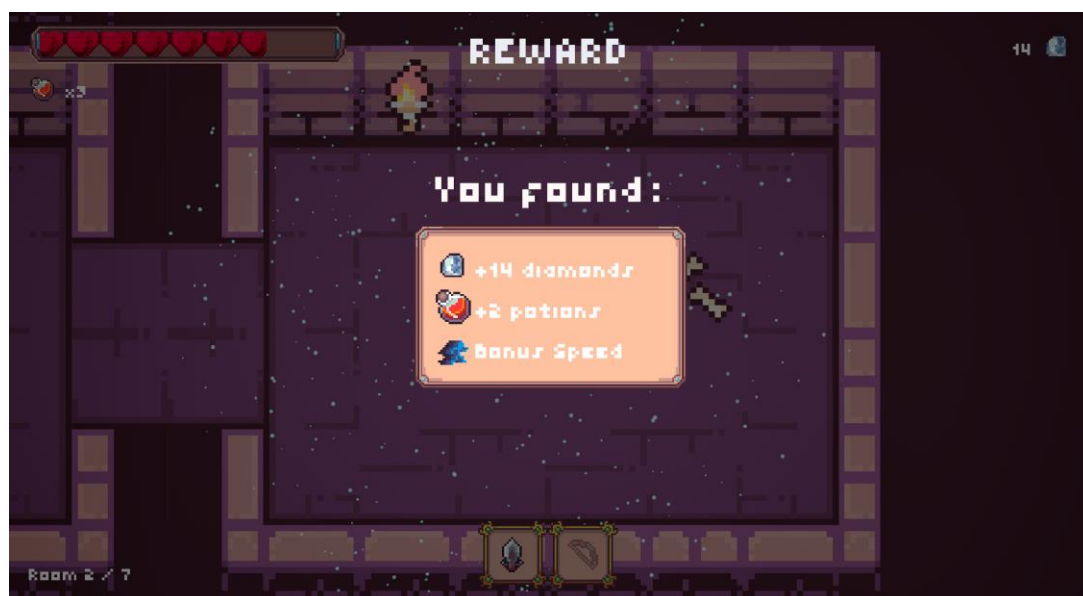


Рисунок 6.7 – Открытие сундука



В ходе забега нажатие «P» или «Escape» открывает меню паузы. Игровой процесс приостанавливается, таймер оценки угрозы врагами останавливается. В меню паузы доступны пять действий: «Continue» (продолжить текущий забег), «Settings» (настройки), «Hub» (прервать забег и вернуться в хаб), «Main Menu» (выйти в главное меню) и «Quit» (выход из программы). Меню паузы представлено на рисунке 6.8.



Рисунок 6.8 – Меню паузы

При снижении здоровья персонажа до нуля забег завершается. На экране отображается экран поражения со статистикой: время забега, количество зачищенных комнат, заработанные алмазы с учетом штрафа 50% за поражение, итоговый баланс алмазов в профиле. Доступны кнопки «Hub», «Main Menu» и «Quit». Экран поражения представлен на рисунке 6.9.

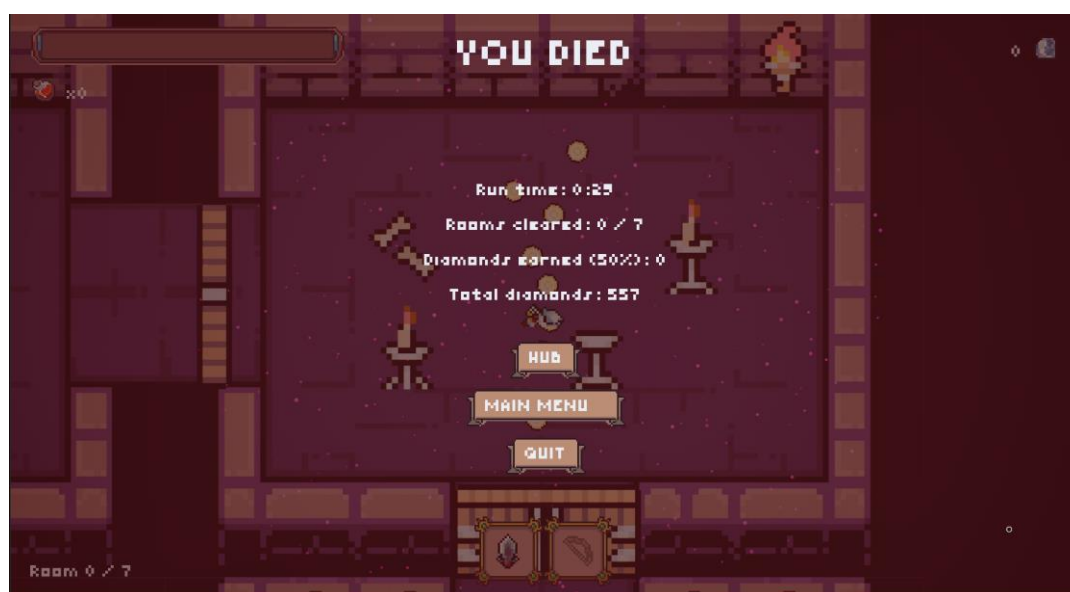


Рисунок 6.9 – Экран поражения

Победа фиксируется при прохождении всех комнат подземелья и активации выхода (люка) в комнате босса. Экран победы отображает те же поля статистики и кнопки, что и экран поражения, однако алмазы начисляются без штрафа. Экран победы представлен на рисунке 6.10.



Рисунок 6.10 – Экран победы

Визуальная обратная связь в игре реализована с использованием системы частиц Niagara. При попадании снаряда по врагу, находящемуся в режиме дальнего боя (отображается в виде монеты), воспроизводится золотистый эффект рассыпающихся частиц в точке попадания. При открытии сундука в месте его расположения вспыхивает голубое свечение с разлетающимися искрами. На экране поражения в позиции персонажа проигрывается розовый эффект угасающих частиц, визуально подчеркивающий завершение забега. На экране победы в той же точке воспроизводится изумрудный взрыв частиц, сигнализирующий об успешном прохождении подземелья.

Диаграмма последовательности взаимодействия с системой со стороны пользователя представлена на чертеже ГУИР.400201.112 РР.2.



## **7 ТЕХНИКО-ЭКОНОМИЧЕСКОЕ ОБОСНОВАНИЕ РАЗРАБОТКИ ПРОГРАММНОГО МОДУЛЯ АДАПТИВНОГО ПОВЕДЕНИЯ ПЕРСОНАЖЕЙ 2D-ИГРЫ В ЖАНРЕ ACTIONRPG**

### **7.1 Характеристика программного средства, разрабатываемого по индивидуальному заказу**

Целью разработки является создание программного модуля адаптивного поведения противников для двумерной игры жанра ActionRPG, предназначенного для повышения вариативности игровых столкновений и снижения предсказуемости поведения игровых объектов в условиях повторяющегося игрового цикла.

Разрабатываемое программное средство ориентировано на применение в составе игровых проектов, где требуется формирование изменяемой модели поведения противников без постоянного увеличения количества игровых сущностей и без расширения числа базовых игровых механик. Практическая задача разработки связана с тем, что в игровых системах данного жанра при многократном повторении игровых ситуаций устойчивые шаблоны поведения противников быстро становятся узнаваемыми, что снижает различие между отдельными игровыми эпизодами.

Программное средство решает задачи выбора поведения игровых противников в зависимости от параметров текущей игровой ситуации, включая расстояние до игрока, текущее состояние противника, плотность игрового пространства и последовательность действий игрока. Дополнительно игровая часть проекта выполняет функцию среды функционирования разработанного программного модуля и обеспечивает проверку его работы в условиях перемещения между комнатами, боевых взаимодействий и повторных игровых циклов.

Разработка программного средства рассматривается как работа проектной группы, включающей программиста игровой логики, технического дизайнера игрового контента и специалиста по тестированию и интеграции. Программист игровой логики выполняет построение программного модуля поведения, реализацию вычислительных алгоритмов и внутреннюю архитектуру проекта. Технический дизайнер обеспечивает подготовку игровой среды, настройку игровых объектов, организацию уровней и интеграцию визуальных ресурсов. Специалист по тестированию и интеграции выполняет проверку корректности игровых сценариев, устойчивости переходов между состояниями и согласованности взаимодействия программных блоков.

Необходимость разработки обусловлена несколькими факторами. В игровых проектах жанра ActionRPG устойчивый интерес пользователей поддерживается за счет повторного прохождения, изменения структуры уровней и постепенного усложнения игровых ситуаций. При этом в значительной части существующих решений поведение противников внутри

отдельных типов остается ограниченным фиксированными схемами, что приводит к повторяемости локальных столкновений даже при изменении внешних условий.

Использование разработанного программного средства позволяет изменять характер игровых столкновений без существенного увеличения объема игровых ресурсов. За счет выделения отдельного программного модуля появляется возможность расширения логики поведения противников без переработки базовой игровой архитектуры.

Практическая ценность разработки заключается в том, что программный модуль проектируется как самостоятельный компонент, который может использоваться в игровых приложениях различного типа без жесткой зависимости от конкретной структуры уровня или фиксированного набора игровых механик. Выделение алгоритма поведения в отдельный программный блок упрощает его перенос, настройку и адаптацию при использовании в других игровых проектах, где требуется изменяемая модель поведения игровых противников.

## **7.2 Расчет инвестиций в разработку программного средства**

### **7.2.1 Расчет зарплат на основную заработную плату разработчиков**

Для расчета затрат на основную заработную плату определяется состав исполнителей, участвующих в разработке программного средства.

В разработке программного средства принимают участие инженер-программист игровой логики, технический дизайнер игрового контента и инженер-тестировщик.

Инженер-программист игровой логики выполняет проектирование программного модуля адаптивного поведения противников, реализацию конечного автомата состояний, настройку взаимодействия игровых блоков и интеграцию программной части в игровую среду.

Технический дизайнер игрового контента отвечает за подготовку игровых ресурсов, настройку игровых сцен, организацию структуры комнат и размещение игровых объектов внутри проекта.

Инженер-тестировщик выполняет проверку корректности игровых сценариев, контроль переходов между состояниями противников и выявление отклонений в работе программного модуля.

Затраты на основную заработную плату рассчитаны по формуле [19]:

$$Z_o = K_{\text{пр}} \sum_{i=1}^n Z_{\text{чи}} \cdot t_i, \quad (7.1)$$

где  $K_{\text{пр}}$  – коэффициент премий и иных стимулирующих выплат;

$n$  – категории исполнителей, занятых разработкой программного средства;

$Z_{\text{чи}}$  – часовая заработная плата исполнителя  $i$ -й категории, р;

$t_i$  – трудоемкость работ, выполняемых исполнителем  $i$ -й категории, ч.

Средний месячный оклад инженера-программиста игровой логики принимается на уровне 4860 рублей, технического дизайнера игрового контента – 3125 рублей, инженера-тестировщика – 2870 рублей.

Основной объем разработки программной части занимает 160 часов работы инженера-программиста игровой логики. Подготовка игровых ресурсов требует 76 часов работы технического дизайнера игрового контента. Проверка игровых сценариев и контроль устойчивости системы требуют 80 часа работы инженера-тестировщика.

Затраты на основную заработную плату приведены в таблице 7.1.

Таблица 7.1 – Затраты на основную заработную плату

| Категория исполнителя                                    | Месячный оклад, р | Часовой оклад, р | Трудоемкость работ, ч | Итого, р |
|----------------------------------------------------------|-------------------|------------------|-----------------------|----------|
| Инженер-программист игровой логики                       | 4 860,00          | 29,25            | 160                   | 4 680,00 |
| Технический дизайнер игрового контента                   | 3125,00           | 19,53            | 76                    | 1484,30  |
| Инженер-тестировщик                                      | 2870,00           | 17,94            | 80                    | 1507,00  |
| Итого                                                    |                   |                  |                       | 7 671,30 |
| Премия и иные стимулирующие выплаты (0%)                 |                   |                  |                       | 0        |
| Всего затраты на основную заработную плату разработчиков |                   |                  |                       | 7 671,30 |

### 7.2.2 Расчет затрат на дополнительную заработную плату разработчиков

В состав дополнительной заработной платы включаются выплаты, связанные с оплатой отпусков, компенсациями за неотработанное время и иными обязательными начислениями, предусмотренными в составе фонда оплаты труда.

Расчет затрат на дополнительную заработную плату команды разработчиков выполняется по формуле [19]:

$$З_{д} = \frac{З_{о} \cdot Н_{д}}{100}, \quad (7.2)$$

где  $Н_{д}$  – норматив дополнительной заработной платы.

Норматив дополнительной заработной платы принимается в размере 10 % от основной заработной платы разработчиков.

Размер дополнительной заработной платы составляет 767,13 рубля.

### 7.2.3 Расчет отчислений на социальные нужды

Размер отчислений на социальные нужды определяется в соответствии

с установленным нормативом обязательных страховых отчислений.

В расчете используется ставка 35%, актуальная на март 2026 года, из которых 29% отчисляется на пенсионное страхование и 6% выделяется на социальное страхование.

Расчет отчислений на социальные нужды выполняется по формуле [19]:

$$P_{\text{соц}} = \frac{(Z_o + Z_d) \cdot H_{\text{соц}}}{100}, \quad (7.3)$$

где  $H_{\text{соц}}$  – норматив отчислений в ФСЗН.

Размер отчислений на социальные нужды составляет 2953,45 рубля.

#### **7.2.4 Расчет прочих расходов**

Прочие расходы включают затраты, связанные с организацией рабочего места, использованием вычислительной техники, электроэнергией, программными средствами и сопровождением процесса разработки.

Расчет выполняется по нормативу прочих расходов, принимаемому в размере 30 % от основной заработной платы разработчиков.

Расчет прочих расходов выполняется по формуле [19]:

$$P_{\text{пр}} = \frac{Z_o \cdot H_{\text{пр}}}{100}, \quad (7.4)$$

где  $H_{\text{пр}}$  – норматив прочих расходов.

Размер прочих расходов составляет 2301,39 рубля.

#### **7.2.5 Расчет плановой прибыли, включаемой в цену программного средства**

Плановая прибыль определяется исходя из полной суммы затрат на разработку программного средства и принятого уровня рентабельности.

Рентабельность затрат на разработку принимается в размере 22%. Плановая прибыль вычисляется по формуле [19]:

$$P_{\text{п.с}} = \frac{Z_p \cdot P_{\text{п.с}}}{100}, \quad (7.5)$$

где  $P_{\text{п.с}}$  – рентабельность затрат на разработку программного средства;

$Z_p$  – затраты на разработку программного средства.

Размер плановой прибыли составляет 3012,52 рубля.

#### **7.2.6 Расчет затрат на разработку и цена программного средства**

Общая сумма затрат на разработку определяется как сумма основной заработной платы разработчиков, дополнительной заработной платы,

отчислений на социальные нужды и прочих расходов. Расчет выполняется по формуле [19]:

$$З_p = З_o + З_d + P_{соц} + P_{пр}. \quad (7.6)$$

Цена программного средства определяется как сумма общих затрат на разработку и плановой прибыли.

Вычисление происходит по следующей формуле [19]:

$$Ц_{п.с} = З_p + П_{п.с}. \quad (7.7)$$

Пошаговое формирование цены программного средства на основе затрат приведено в таблице 7.2.

Таблица 7.2 – Затраты на разработку

| Название статьи затрат                                      | Формула/таблица для расчета                           | Значение, р. |
|-------------------------------------------------------------|-------------------------------------------------------|--------------|
| 1 Основная заработная плата разработчиков                   | Таблица 7.1                                           | 7 671,30     |
| 2 Дополнительная заработная плата разработчиков             | $З_d = \frac{7\,691,30 \cdot 10}{100}$                | 767,13       |
| 3 Отчисление на социальные нужды                            | $P_{соц} = \frac{(7\,673,30 + 767,13) \cdot 35}{100}$ | 2 953,45     |
| 4 Прочие расходы                                            | $P_{пр} = \frac{7\,671,20 \cdot 30}{100}$             | 2 301,39     |
| 5 Общая сумма затрат на разработку и реализацию             | $З_p = 7\,671,30 + 767,13 + 2\,953,45 + 2\,301,39$    | 13693,27     |
| 6 Плановая прибыль, включаемая в цену программного средства | $П_{п.с} = \frac{13693,27 \cdot 22}{100}$             | 3012,52      |
| 7 Отпускная цена программного средства                      | $Ц_{п.с} = 13693,27 + 3012,52$                        | 16 705,79    |

### 7.3 Расчет разработки и использования программного средства

Для организации-разработчика экономическим результатом является чистая прибыль, полученная после реализации программного средства.

Учитывая, что программное средство реализовано по отпускной цене, которая сформирована на основе затрат на разработку организацией-разработчиком, то прирост рассчитывается следующим образом:

$$\Delta\Pi_{\text{ч}} = \Pi_{\text{п.с}} \cdot \left(1 - \frac{H_{\text{п}}}{100}\right), \quad (7.8)$$

где  $H_{\text{п}}$  – ставка налога на прибыль, %.

На март 2026 года ставка налога на прибыль составляет 20%.

Вычисление чистой прибыли производится по формуле (7.8):

$$\Delta\Pi_{\text{ч}} = 3012,52 \cdot \left(1 - \frac{20}{100}\right) = 2410,02 \text{ р.}$$

#### **7.4 Расчет показателей экономической эффективности разработки программного средства**

Для оценки экономической эффективности разработки определяется показатель рентабельности инвестиций в разработку программного средства. Расчет выполняется по формуле:

$$P_{\text{и}} = \frac{\Delta\Pi_{\text{ч}}}{Z_{\text{р}}} \cdot 100\%, \quad (7.9)$$

где  $\Delta\Pi_{\text{ч}}$  – прирост чистой прибыли, полученной от разработки программного средства организацией-заказчиком;

$Z_{\text{р}}$  – затраты на разработку программного средства организацией-разработчиком.

Вычисление рентабельности инвестиций по формуле (7.9):

$$P_{\text{и}} = \frac{2410,02}{13693,27} \cdot 100\% = 17,6\%.$$

#### **7.5 Вывод об экономической целесообразности реализации проектного решения**

Проведенные расчеты позволяют сделать вывод о целесообразности разработки программного средства с точки зрения затрат на создание и ожидаемого экономического результата. Полученные показатели показывают, что разработка программного модуля и игровой среды сопровождается приемлемым уровнем затрат и обеспечивает положительный экономический результат при условии реализации как законченного программного решения.

Общая сумма затрат на разработку составила 13693,27 белорусского рубля. Размер плановой прибыли, включаемой в цену программного средства, составил 3012,52 белорусского рубля. После учета налога на прибыль величина чистой прибыли составляет 2410,02 белорусского рубля. Расчет рентабельности инвестиций показал значение 17,6 %.

Полученный уровень рентабельности соответствует допустимым

значениям для разработки специализированного программного средства, выполняемого по индивидуальному проектному заданию. Для программных разработок, ориентированных на ограниченный объем внедрения и отсутствие массового тиражирования на начальном этапе, подобный показатель рассматривается как достаточный для подтверждения экономической оправданности разработки.

Дополнительное значение имеет структура самого программного решения. Разрабатываемый программный модуль адаптивного поведения противников создается как самостоятельный компонент, который может использоваться отдельно от конкретной игровой среды. Это позволяет применять разработанный алгоритм в игровых проектах различной структуры без полной переработки программной логики.

Экономическая эффективность также определяется тем, что дальнейшее использование программного модуля требует меньших затрат по сравнению с первичной разработкой. При повторной адаптации в составе других игровых приложений сохраняется основная программная архитектура, а изменения затрагивают только параметры интеграции и отдельные игровые условия.

Снижение затрат при повторном использовании достигается за счет того, что вычислительная часть, отвечающая за выбор поведения игровых противников, отделена от конкретной реализации игровых объектов и не требует повторного проектирования базовых алгоритмов состояний. Это создает предпосылки для использования разработанного решения в учебных, демонстрационных и прикладных игровых проектах, где требуется изменяемая модель поведения без разработки нового программного ядра.

Отдельным фактором экономической целесообразности является ограниченный объем используемых технических ресурсов при разработке. В проекте применяются стандартные программные средства, не требующие приобретения специализированного оборудования или организации отдельной вычислительной инфраструктуры, что снижает совокупные затраты на создание и сопровождение программного решения.

Полученный программный результат сочетает самостоятельный вычислительный модуль и игровую среду для его функционирования, что расширяет возможности последующего использования и доработки. На основании выполненных расчетов разработка программного средства может рассматриваться как экономически оправданная, поскольку полученные показатели затрат, прибыли и рентабельности подтверждают целесообразность реализации выбранного проектного решения.

## ЗАКЛЮЧЕНИЕ

Дипломный проект завершен в полном объеме. Разработанное программное средство представляет собой двумерную игровую систему в жанре ActionRPG с программным модулем адаптивного поведения персонажей, реализованным на C++ в среде Unreal Engine с использованием подсистемы Paper2D. Все задачи технического задания выполнены: создана игровая среда с процедурной генерацией комнат, спроектирован конечный автомат из пяти состояний, реализован модуль адаптивного выбора поведения на основе трехслойной модели оценки угрозы.

В ходе проектирования проведен анализ предметной области и существующих аналогов, обоснован выбор стека технологий и архитектурных решений. Технико-экономическое обоснование подтвердило целесообразность разработки с рентабельностью инвестиций 17,6 %.

Модуль адаптивного поведения реализован как самостоятельный компонент, отделенный от игровых объектов. Адаптация достигается через динамическую коррекцию весов по детерминированной обратной связи, память событий с экспоненциальным затуханием, частотный анализ N-грамм действий игрока в скользящем окне и коэффициент уверенности, управляющий консерватизмом при нестабильных входных данных. Встроенные средства описания поведения движка не применяются, весь код написан вручную на C++.

Игровая среда включает процедурную генерацию подземелья на основе ячеечной сетки с поддержкой комнат различной формы, систему вооружения из двух слотов, модификаторы на один забег, метапрогрессию между попытками через SQLite и отладочный виджет, отображающий в реальном времени текущую оценку угрозы, значения весов, распознанные паттерны и выбранное состояние.

К достоинствам разработанного программного модуля относятся:

- трехслойная архитектура принятия решений с математически обоснованными формулами без использования встроенных средств поведения движка;
- распознавание тактик игрока и противодействие им через анализ последовательностей действий;
- компонентная структура, позволяющая переносить модуль между различными игровыми проектами без переработки базовых алгоритмов;
- наглядная визуализация работы модуля через отладочный виджет для демонстрации и верификации.

Дипломный проект выполнен мной лично, проверен в системе «Text.ru». Оригинальность составляет 97.75%. Отчет о результатах проверки текста на наличие заимствований прилагается и представлен в Приложении В.



## СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- [1] The Binding of Isaac [Электронный ресурс]. – Режим доступа: [https://store.steampowered.com/app/250900/The\\_Binding\\_of\\_Isaac\\_Rebirth/](https://store.steampowered.com/app/250900/The_Binding_of_Isaac_Rebirth/) – Дата доступа: 17.03.2026
- [2] Enter the Gungeon [Электронный ресурс]. – Режим доступа: [https://store.steampowered.com/app/311690/Enter\\_the\\_Gungeon/](https://store.steampowered.com/app/311690/Enter_the_Gungeon/) – Дата доступа: 17.03.2026
- [3] Hades [Электронный ресурс]. – Режим доступа: <https://store.steampowered.com/app/1145360/Hades/> – Дата доступа: 17.03.2026
- [4] Dead Cells [Электронный ресурс]. – Режим доступа: [https://store.steampowered.com/app/588650/Dead\\_Cells/](https://store.steampowered.com/app/588650/Dead_Cells/) – Дата доступа: 17.03.2026
- [5] Moonlighter [Электронный ресурс]. – Режим доступа: <https://store.steampowered.com/app/606150/Moonlighter/> – Дата доступа: 17.03.2026
- [6] C++ [Электронный ресурс]. – Режим доступа: <https://metanit.com/cpp/tutorial/1.1.php> – Дата доступа: 17.03.2026
- [7] C# [Электронный ресурс]. – Режим доступа: <https://metanit.com/sharp/tutorial/1.1.php> – Дата доступа: 17.03.2026
- [8] Java [Электронный ресурс]. – Режим доступа: <https://metanit.com/java/tutorial/1.1.php> – Дата доступа: 17.03.2026
- [9] Unreal Engine [Электронный ресурс]. – Режим доступа: <https://habr.com/ru/articles/820477/> – Дата доступа: 17.03.2026
- [10] Unity [Электронный ресурс]. – Режим доступа: <https://habr.com/ru/articles/828302/> – Дата доступа: 17.03.2026
- [11] Godot [Электронный ресурс]. – Режим доступа: [https://docs.godotengine.org/ru/4.x/getting\\_started/](https://docs.godotengine.org/ru/4.x/getting_started/) – Дата доступа: 17.03.2026
- [12] Visual Studio [Электронный ресурс]. – Режим доступа: <https://visualstudio.microsoft.com/ru/> – Дата доступа: 17.03.2026
- [13] Git [Электронный ресурс]. – Режим доступа: <https://habr.com/ru/articles/541258/> – Дата доступа: 17.03.2026
- [14] GitHub [Электронный ресурс]. – Режим доступа: <https://docs.github.com/ru> – Дата доступа: 17.03.2026
- [15] GitLab [Электронный ресурс]. – Режим доступа: <https://docs.gitlab.com> – Дата доступа: 17.03.2026
- [16] SQLite [Электронный ресурс]. – Режим доступа: <https://metanit.com/sql/sqlite/1.1.php> – Дата доступа: 17.03.2026
- [17] PostgreSQL [Электронный ресурс]. – Режим доступа: <https://postgrespro.ru/docs/postgresql> – Дата доступа: 17.03.2026
- [18] MongoDB [Электронный ресурс]. – Режим доступа: <https://metanit.com/nosql/mongodb/1.1.php> – Дата доступа: 17.03.2026
- [19] Экономика проектных решений: методические указания по экономическому обоснованию дипломных проектов: учеб.-метод пособие / В.Г. Горовой [и др.] – Минск: БГУИР, 2021 – 107 с.

[20] Грегори Джейсон. Игровой движок. Программирование и внутреннее устройство. Третье издание / Дж. Грегори. – СПб.: Питер, 2022. – 1136 с.

[21] Game Programming Patterns. State [Электронный ресурс]. – Режим доступа: <https://gameprogrammingpatterns.com/state.html> – Дата доступа: 17.03.2026

[22] An Introduction to Utility Theory [Электронный ресурс]. – Режим доступа: [https://www.gameapro.com/GameAIPro/GameAIPro\\_Chapter09\\_An\\_Introduction\\_to\\_Utility\\_Theory.pdf](https://www.gameapro.com/GameAIPro/GameAIPro_Chapter09_An_Introduction_to_Utility_Theory.pdf) – Дата доступа: 17.03.2026

# ПРИЛОЖЕНИЕ А

## (обязательное)

### Листинг кода

```
Source\Depthrun\AdaptiveBehavior
AdaptiveBehaviorComponent.cpp
001  #include "AdaptiveBehaviorComponent.h"
002  #include "AdaptiveConfig.h"
003  #include "AdaptiveMemory.h"
004  #include "ContextEvaluator.h"
005  #include "DepthrunLogChannels.h"
006  #include "DynamicWeightManager.h"
007  #include "FSM/FSMComponent.h"
008  #include "PatternRecognizer.h"
009  #include "StateTransitionResolver.h"
010  #include "ThreatCalculator.h"
011  #include "TransitionCostMatrix.h"
012  #include "UtilityCurves.h"
013  #include "Kismet/GameplayStatics.h"
014  #include "Enemy/BaseEnemy.h"
015  #include "Enemy/AdaptiveEnemy.h"
016  #include "Enemy/EnemyHealthComponent.h"
017  #include "Player/DepthrunCharacter.h"
018  UAdaptiveBehaviorComponent::UAdaptiveBehaviorComponent() {
    PrimaryComponentTick.bCanEverTick = false; }
019  void UAdaptiveBehaviorComponent::BeginPlay() {
020      Super::BeginPlay();
021      InitializeSubsystems();
022      if (Config) {
023          GetWorld()->GetTimerManager().SetTimer(
024              EvaluationTimerHandle, this,
025              &UAdaptiveBehaviorComponent::EvaluationTick, Config->EvaluationInterval,
026              true);
027      } else { UE_LOG(LogAdaptiveBehavior, Error, TEXT("[AdaptiveBehavior] ERROR: Config is
missing on %s! AI will be inactive."), *GetOwner()->GetName()); }
028      if (ABaseEnemy* Owner = Cast<ABaseEnemy>(GetOwner()))
029      {
030          if (Owner->GetHealthComponent()) { Owner->GetHealthComponent()-
>OnDeath.AddDynamic(this, &UAdaptiveBehaviorComponent::OnOwnerDeath); }
031      }
032  }
033  void UAdaptiveBehaviorComponent::OnOwnerDeath()
034  {
035      GetWorld()->GetTimerManager().ClearTimer(EvaluationTimerHandle);
036      UE_LOG(LogAdaptiveBehavior, Log, TEXT("[AdaptiveBehavior] Owner died, evaluation
stopped."));
037  }
038  void UAdaptiveBehaviorComponent::EndPlay(
039      const EEndPlayReason::Type EndPlayReason) {
040      GetWorld()->GetTimerManager().ClearTimer(EvaluationTimerHandle);
041      Super::EndPlay(EndPlayReason);
042  }
043  void UAdaptiveBehaviorComponent::InitializeSubsystems() {
044      ContextEval = NewObject<UContextEvaluator>(this);
045      ThreatCalc = NewObject<UThreatCalculator>(this);
046      Resolver = NewObject<UStateTransitionResolver>(this);
047      Memory = NewObject<UAdaptiveMemory>(this);
048      PatternRecog = NewObject<UPatternRecognizer>(this);
049      WeightManager = NewObject<UDynamicWeightManager>(this);
050      UtilCurves = NewObject<UUtilityCurves>(this);
051      CostMatrix = NewObject<UTransitionCostMatrix>(this);
052      FSMComp = GetOwner()->FindComponentByClass<UFSMComponent>();
053      if (WeightManager && Config) { WeightManager->ResetToDefaults(Config); }
054      if (CostMatrix && Config) { CostMatrix->InitializeFromConfig(Config); }
055      UE_LOG(LogAdaptiveBehavior, Log,
056          TEXT("[AdaptiveBehavior] Subsystems initialized for %s"),
057          *GetOwner()->GetName());
058  }
059  void UAdaptiveBehaviorComponent::EvaluationTick() {
060      if (!Config || !ContextEval || !ThreatCalc || !Resolver || !FSMComp) return;
061      ABaseEnemy* Owner = Cast<ABaseEnemy>(GetOwner());
062      ADepthrunCharacter* Player = Cast<ADepthrunCharacter>(UGameplayStatics::GetPlayerCharacter(GetWorld(), 0));
063      if (!Owner || Owner->IsDead() || !Player) return;
064      if (Memory) { Memory->CleanupOldEvents(GetWorld()->GetTimeSeconds(), 0.f); }
065      LastContext = ContextEval->EvaluateContextWithMemory(Owner, Player, Memory, Config);
```

```

066     LastContext.BraveryLevel = BraveryLevel;
067     LastContext.CombatStyle = CombatStyle;
068     LastThreatAssessment = ThreatCalc->CalculateThreat(LastContext, WeightManager, Config);
069     OnThreatEvaluated.Broadcast(LastThreatAssessment);
070     if (!bAdaptiveEnabled)
071     {
072         UE_LOG(LogAdaptiveBehavior, Verbose, TEXT("[AdaptiveBehavior] Bypass Layer 3
(bAdaptiveEnabled=false)"));
073         return;
074     }
075     float BraveryModifier = 0.f;
076     switch (BraveryLevel)
077     {
078         case EEnemyBravery::Coward: BraveryModifier = -0.3f; break;
079         case EEnemyBravery::Normal: BraveryModifier = 0.f; break;
080         case EEnemyBravery::Brave: BraveryModifier = 0.25f; break;
081         case EEnemyBravery::Heroic: BraveryModifier = 0.5f; break;
082     }
083     FThreatAssessment BiasedThreat = LastThreatAssessment;
084     BiasedThreat.ThreatFinal = FMath::Clamp(BiasedThreat.ThreatFinal - BraveryModifier, 0.f,
1.f);
085     float TimeInState = FSMComp->GetTimeInCurrentState();
086     EFSMStateType NewState = Resolver->ResolveNextState(
087         FSMComp->GetCurrentStateType(),
088         BiasedThreat,
089         LastContext,
090         TimeInState,
091         UtilCurves,
092         CostMatrix,
093         PatternRecog,
094         Config,
095         LastStateScores
096     );
097     EFSMStateType OldState = FSMComp->GetCurrentStateType();
098     if (NewState == EFSMStateType::Retreat && OldState == EFSMStateType::Idle)
099     {
100         float Now = GetWorld()->GetTimeSeconds();
101         if (Now - LastRetreatExitTime < RetreatHysteresisDuration)
102         {
103             UE_LOG(LogAdaptiveBehavior, Verbose, TEXT("[AdaptiveBehavior] Blocking Retreat
transition due to hysteresis.));
104             NewState = EFSMStateType::Idle;
105         }
106     }
107     if (NewState != OldState)
108     {
109         if (OldState == EFSMStateType::Retreat) { LastRetreatExitTime = GetWorld()-
>GetTimeSeconds(); }
110         FSMComp->TransitionTo(NewState);
111         OnAdaptiveDecisionMade.Broadcast(OldState, NewState);
112     }
113     if (AAdaptiveEnemy* AdaptiveOwner = Cast<AAdaptiveEnemy>(Owner))
114     {
115         const float MeleeR = Owner->MeleeAttackRange;
116         const float RangedR = Owner->RangedAttackRange;
117         const float Mid = (MeleeR + RangedR) * 0.5f;
118         const float Hyst = (RangedR - MeleeR) * 0.10f;
119         float SwitchUp = Mid + Hyst;
120         float SwitchDown = Mid - Hyst;
121         switch (CombatStyle)
122         {
123             case EEnemyCombatStyle::MeleeOriented:
124                 SwitchUp = Mid + Hyst * 3.f;
125                 SwitchDown = Mid - Hyst;
126                 break;
127             case EEnemyCombatStyle::Balanced:
128                 SwitchUp = Mid + Hyst;
129                 SwitchDown = Mid - Hyst;
130                 break;
131             case EEnemyCombatStyle::RangedOriented:
132                 SwitchUp = Mid - Hyst;
133                 SwitchDown = MeleeR * 1.1f;
134                 break;
135         }
136         const bool bWasRanged = AdaptiveOwner->bIsRangedMode;
137         const float Dist = LastContext.DistanceToPlayer;
138         bool bShouldBeRanged;
139         if (bWasRanged) { bShouldBeRanged = (Dist > SwitchDown); }
140         else { bShouldBeRanged = (Dist > SwitchUp); }
141         if (NewState == EFSMStateType::Flank || NewState == EFSMStateType::Retreat) {

```

```

        bShouldBeRanged = true; }
142     else if (NewState == EFSMStateType::Attack && Dist <= MeleeR) { bShouldBeRanged =
false; }
143     AdaptiveOwner->bIsRangedMode = bShouldBeRanged;
144 }
145 FString Pattern = PatternRecog->GetDominantPattern();
146 if (!Pattern.IsEmpty()) { OnPatternRecognized.Broadcast(Pattern); }
147 if (UE_LOG_ACTIVE(LogAdaptiveBehavior, Verbose))
148 {
149     FString ScoresLog;
150     for (const FStateScore& S : LastStateScores) { ScoresLog +=
FString::Printf(TEXT("%s=%.2f "), *UFSMComponent::GetStateName(S.State), S.FinalScore); }
151     UE_LOG(LogAdaptiveBehavior, Verbose,
152         TEXT("[AdaptiveBehavior] Enemy=%s | T=%.3f C=%.3f | %s | → %s"),
153         *Owner->GetName(),
154         LastThreatAssessment.ThreatFinal,
155         LastThreatAssessment.Confidence,
156         *ScoresLog,
157         *UFSMComponent::GetStateName(NewState));
158 }
159 }
160 void UAdaptiveBehaviorComponent::OnDamageDealt() {
161     if (WeightManager && Config) { WeightManager->UpdateWeights(1.0f, LastContext, Config);
}
162 }
163 void UAdaptiveBehaviorComponent::OnDamageTaken() {
164     if (WeightManager && Config) { WeightManager->UpdateWeights(-1.0f, LastContext, Config);
}
165 }
166 void UAdaptiveBehaviorComponent::HandlePlayerAction(
167     const FPlayerActionEvent &ActionEvent) {
168     if (Memory) {
169         FMemoryEvent Ev;
170         Ev.ActionType = ActionEvent.ActionType;
171         Ev.Timestamp = ActionEvent.Timestamp;
172         Ev.Location = ActionEvent.Location;
173         Ev.Intensity = ActionEvent.Intensity;
174         Memory->RecordEvent(Ev);
175     }
176     if (PatternRecog) { PatternRecog->AddAction(ActionEvent.ActionType); }
177 }
178 float UAdaptiveBehaviorComponent::GetThreatFinal() const { return
LastThreatAssessment.ThreatFinal; }
179 float UAdaptiveBehaviorComponent::GetConfidence() const { return
LastThreatAssessment.Confidence; }
180 TArray<float> UAdaptiveBehaviorComponent::GetCurrentWeights() const { return WeightManager
? WeightManager->GetWeights() : TArray<float>(); }
181 FString UAdaptiveBehaviorComponent::GetRecognizedPattern() const { return PatternRecog ?
PatternRecog->GetDominantPattern() : TEXT(""); }
182 TArray<FStateScore> UAdaptiveBehaviorComponent::GetLastStateScores() const { return
LastStateScores; }
183 FThreatAssessment UAdaptiveBehaviorComponent::GetLastThreatAssessment() const { return
LastThreatAssessment; }
184 FContextData UAdaptiveBehaviorComponent::GetLastContext() const { return LastContext; }

ThreatCalculator.cpp
001 #include "ThreatCalculator.h"
002 #include "AdaptiveConfig.h"
003 #include "DynamicWeightManager.h"
004 DEFINE_LOG_CATEGORY_STATIC(LogThreatCalculator, Log, All);
005 FThreatAssessment UThreatCalculator::CalculateThreat(
006     const FContextData& Context,
007     const UDynamicWeightManager* WeightsMgr,
008     const UAdaptiveConfig* Config)
009 {
010     FThreatAssessment Result;
011     if (!Config) { LastAssessment = Result; return Result; }
012     const TArray<float>& W = WeightsMgr ? WeightsMgr->GetWeights() : DefaultWeights;
013     Result.ThreatBase = ComputeTBase(Context, W);
014     Result.ThreatRaw = Result.ThreatBase;
015     Result.ThreatFinal = FMath::Clamp(Result.ThreatRaw, 0.f, 1.f);
016     Result.ThreatSmoothed = Result.ThreatFinal;
017     Result.Confidence = 1.f;
018     UE_LOG(LogThreatCalculator, Verbose,
019         TEXT("[Threat] T_base=%.3f T_final=%.3f"),
020         Result.ThreatBase, Result.ThreatFinal);
021     LastAssessment = Result;
022     return Result;
023 }
024 float UThreatCalculator::ComputeTBase(const FContextData& Ctx, const TArray<float>& W)

```

```

025     const
026     {
027         if (W.Num() < 6) return 0.f;
028         const float fD = Ctx.DistanceNorm;
029         const float fW = Ctx.WeaponThreatNorm;
030         const float fH = Ctx.EnemyHPRatioNorm;
031         const float fA = 1.f - FMath::Sqrt(FMath::Clamp(Ctx.AllyCountNorm, 0.f, 1.f));
032         const float fR = Ctx.RoomDensityNorm;
033         const float fM = FMath::Clamp(Ctx.MemoryAggressiveness, 0.f, 1.f);
034         return W[0]*fD + W[1]*fW + W[2]*fH + W[3]*fA + W[4]*fR + W[5]*fM;
035     }
036 }
037
038 StateTransitionResolver.cpp
039 #include "StateTransitionResolver.h"
040 #include "AdaptiveConfig.h"
041 #include "PatternRecognizer.h"
042 #include "TransitionCostMatrix.h"
043 #include "UtilityCurves.h"
044
045 EFSMStateType UStateTransitionResolver::ResolveNextState(
046     EFSMStateType CurrentState,
047     const FThreatAssessment& Threat,
048     const FContextData& Context,
049     float TimeInCurrentState,
050     const UUtilityCurves* UtilCurves,
051     const UTransitionCostMatrix* CostMatrix,
052     const UPatternRecognizer* PatternRecog,
053     const UAdaptiveConfig* Config,
054     TArray<FStateScore>& OutScores) const
055 {
056     OutScores.Reset();
057     if (!UtilCurves || !CostMatrix || !PatternRecog) return CurrentState;
058     const TArray<EFSMStateType> StatesToEvaluate = {
059         EFSMStateType::Idle,
060         EFSMStateType::Chase,
061         EFSMStateType::Attack,
062         EFSMStateType::Retreat,
063         EFSMStateType::Flank
064     };
065     EFSMStateType BestState = CurrentState;
066     float BestScore = -1e9f;
067     for (EFSMStateType Candidate : StatesToEvaluate)
068     {
069         FStateScore Score;
070         Score.State = Candidate;
071         Score.UtilityValue = UtilCurves->EvaluateUtility(Candidate, Threat, Context,
072             Config);
073         Score.TransitionCost = CostMatrix->GetCost(CurrentState, Candidate);
074         Score.InertiaBonus = CostMatrix->CalculateInertia(CurrentState, Candidate,
075             TimeInCurrentState, Config);
076         Score.PatternModifier = PatternRecog->GetPatternModifier(Candidate);
077         Score.FinalScore = ScoreState(Candidate, CurrentState,
078             Score.UtilityValue, Score.TransitionCost, Score.InertiaBonus,
079             Score.PatternModifier);
080         OutScores.Add(Score);
081         if (Score.FinalScore > BestScore)
082         {
083             BestScore = Score.FinalScore;
084             BestState = Candidate;
085         }
086     }
087     return BestState;
088 }
089
090 float UStateTransitionResolver::ScoreState(
091     EFSMStateType Candidate,
092     EFSMStateType Current,
093     float Utility, float Cost, float Inertia, float Pattern) const { return Utility -
094     Cost + Inertia + Pattern; }
095
096 AdaptiveMemory.cpp
097 #include "AdaptiveMemory.h"
098 #include "AdaptiveConfig.h"
099 void UAdaptiveMemory::Initialize(const UAdaptiveConfig* Config)
100 {
101     if (Config) { MemoryWindowSeconds = Config->MemoryWindowSeconds; }
102     MemoryBuffer.Reserve(MaxBufferSize);
103 }
104 void UAdaptiveMemory::RecordEvent(const FMemoryEvent& Event)
105 {
106     if (MemoryBuffer.Num() >= MaxBufferSize)
107     {
108

```

```

012         MemoryBuffer[0] = MemoryBuffer.Last();
013         MemoryBuffer.Pop(EAllowShrinking::No);
014     }
015     MemoryBuffer.Add(Event);
016 }
017 float UAdaptiveMemory::GetDecayedAggressiveness(float CurrentTime, float ) const
018 {
019     return CountWindowMetric(CurrentTime, [](EPlayerActionType T)
020     {
021         return T == EPlayerActionType::Shot || T == EPlayerActionType::MeleeAttack;
022     });
023 }
024 float UAdaptiveMemory::GetDecayedMobility(float CurrentTime, float ) const
025 {
026     return CountWindowMetric(CurrentTime, [](EPlayerActionType T)
027     {
028         return T == EPlayerActionType::Dash;
029     });
030 }
031 float UAdaptiveMemory::GetDecayedCaution(float CurrentTime, float ) const
032 {
033     return CountWindowMetric(CurrentTime, [](EPlayerActionType T)
034     {
035         return T == EPlayerActionType::Heal;
036     });
037 }
038 void UAdaptiveMemory::CleanupOldEvents(float CurrentTime, float )
039 {
040     MemoryBuffer.RemoveAll([CurrentTime, this](const FMemoryEvent& Ev)
041     {
042         return (CurrentTime - Ev.Timestamp) > MemoryWindowSeconds;
043     });
044 }
045 float UAdaptiveMemory::CountWindowMetric(float CurrentTime,
046     TFunctionRef<bool(EPlayerActionType)> Filter) const
047 {
048     if (MemoryWindowSeconds <= 0.f) return 0.f;
049     int32 Count = 0;
050     for (const FMemoryEvent& Ev : MemoryBuffer)
051     {
052         if ((CurrentTime - Ev.Timestamp) <= MemoryWindowSeconds &&
053             Filter(Ev.ActionType)) { ++Count; }
054     }
055     return FMath::Clamp(static_cast<float>(Count) / MemoryWindowSeconds, 0.f, 1.f);
056 }

```

PatternRecognizer.cpp

```

001 #include "PatternRecognizer.h"
002 #include "FSM/FSMTypes.h"
003 DEFINE_LOG_CATEGORY_STATIC(LogPatternRecognizer, Log, All);
004 void UPatternRecognizer::AddAction(EPlayerActionType ActionType)
005 {
006     if (ActionWindow.Num() >= WindowSize) { ActionWindow.RemoveAt(0); }
007     ActionWindow.Add(ActionType);
008     RebuildNgrams();
009 }
010 FString UPatternRecognizer::GetDominantPattern() const
011 {
012     FString BestPattern = TEXT("None");
013     int32 BestScore = 0;
014     for (const auto& Pair : Unigrams)
015     {
016         if (Pair.Value > BestScore)
017         {
018             BestScore = Pair.Value;
019             BestPattern = Pair.Key;
020         }
021     }
022     for (const auto& Pair : Bigrams)
023     {
024         if (Pair.Value * 2 > BestScore)
025         {
026             BestScore = Pair.Value * 2;
027             BestPattern = Pair.Key;
028         }
029     }
030     for (const auto& Pair : Trigrams)
031     {
032         if (Pair.Value * 3 > BestScore)
033     {

```

```

034         BestScore = Pair.Value * 3;
035         BestPattern = Pair.Key;
036     }
037 }
038     return BestPattern;
039 }
040 float UPatternRecognizer::GetPatternModifier(EFSMStateType CandidateState) const
041 {
042     const FString Dominant = GetDominantPattern();
043     static const TArray<FPatternRule> Rules =
044     {
045         { TEXT("Shot+Shot+Shot"), EFSMStateType::Flank, +0.65f },
046         { TEXT("Shot+Shot+Shot"), EFSMStateType::Chase, -0.20f },
047         { TEXT("Shot+Shot+Shot"), EFSMStateType::Attack, -0.25f },
048         { TEXT("Shot+Shot+Dash"), EFSMStateType::Flank, +0.45f },
049         { TEXT("Shot+Shot+Dash"), EFSMStateType::Chase, +0.10f },
050         { TEXT("Melee+Dash"), EFSMStateType::Retreat, +0.25f },
051         { TEXT("Melee+Dash"), EFSMStateType::Flank, +0.15f },
052         { TEXT("Dash+Melee"), EFSMStateType::Retreat, +0.25f },
053         { TEXT("Dash+Melee"), EFSMStateType::Flank, +0.15f },
054         { TEXT("Dash+Dash"), EFSMStateType::Attack, -0.15f },
055         { TEXT("Dash+Dash"), EFSMStateType::Chase, +0.20f },
056         { TEXT("Shot+Shot"), EFSMStateType::Flank, +0.65f },
057         { TEXT("Shot+Shot"), EFSMStateType::Chase, -0.10f },
058         { TEXT("Shot+Shot"), EFSMStateType::Retreat, +0.10f },
059         { TEXT("Shot+Shot"), EFSMStateType::Attack, -0.20f },
060         { TEXT("Melee"), EFSMStateType::Flank, +0.30f },
061         { TEXT("Melee"), EFSMStateType::Retreat, +0.10f },
062         { TEXT("Shot"), EFSMStateType::Flank, +0.50f },
063         { TEXT("Shot"), EFSMStateType::Chase, +0.10f },
064     };
065     for (const FPatternRule& Rule : Rules)
066     {
067         if (Rule.Pattern == Dominant && Rule.State == CandidateState) { return
Rule.Modifier; }
068     }
069     return 0.f;
070 }
071 void UPatternRecognizer::Reset()
072 {
073     ActionWindow.Reset();
074     Unigrams.Reset();
075     Bigrams.Reset();
076     Trigrams.Reset();
077 }
078 void UPatternRecognizer::RebuildNgrams()
079 {
080     Unigrams.Reset();
081     Bigrams.Reset();
082     Trigrams.Reset();
083     const int32 N = ActionWindow.Num();
084     for (int32 i = 0; i < N; ++i) {
085         Unigrams.FindOrAdd(ActionToString(ActionWindow[i]))++; }
086     for (int32 i = 0; i + 1 < N; ++i)
087     {
088         FString Key = ActionToString(ActionWindow[i]) + TEXT("+") +
ActionToString(ActionWindow[i + 1]);
089         Bigrams.FindOrAdd(Key)++;
090     }
091     for (int32 i = 0; i + 2 < N; ++i)
092     {
093         FString Key = ActionToString(ActionWindow[i])
+ TEXT("+") + ActionToString(ActionWindow[i + 1])
094         + TEXT("+") + ActionToString(ActionWindow[i + 2]);
095         Trigrams.FindOrAdd(Key)++;
096     }
097 }
098 FString UPatternRecognizer::ActionToString(EPlayerActionType T) const
099 {
100     switch (T)
101     {
102     case EPlayerActionType::Shot: return TEXT("Shot");
103     case EPlayerActionType::Dash: return TEXT("Dash");
104     case EPlayerActionType::MeleeAttack: return TEXT("Melee");
105     case EPlayerActionType::Heal: return TEXT("Heal");
106     case EPlayerActionType::SpecialAbility: return TEXT("Special");
107     default: return TEXT("Unknown");
108     }
109 }

```



```

DynamicWeightManager.cpp
001  #include "DynamicWeightManager.h"
002  #include "AdaptiveConfig.h"
003  DEFINE_LOG_CATEGORY_STATIC(LogDynamicWeights, Log, All);
004  void UDynamicWeightManager::ResetToDefaults(const UAdaptiveConfig* Config)
005  {
006      Weights.SetNum(6);
007      if (!Config) { Weights.Init(1.f / 6.f, 6); return; }
008      Weights[0] = Config->WeightDistance;
009      Weights[1] = Config->WeightWeaponThreat;
010      Weights[2] = Config->WeightHealth;
011      Weights[3] = Config->WeightAllies;
012      Weights[4] = Config->WeightRoomDensity;
013      Weights[5] = Config->WeightMemory;
014  }
015  void UDynamicWeightManager::UpdateWeights(float Reward, const FContextData& Context, const
UAdaptiveConfig* Config)
016  {
017      if (!Config || Weights.Num() < 6) return;
018      const float FactorValues[6] =
019      {
020          Context.DistanceNorm,
021          Context.WeaponThreatNorm,
022          Context.EnemyHPRatioNorm,
023          1.f - FMath::Sqrt(FMath::Clamp(Context.AllyCountNorm, 0.f, 1.f)),
024          Context.RoomDensityNorm,
025          FMath::Clamp(Context.MemoryAggressiveness, 0.f, 1.f),
026      };
027      float SumFactors = 0.f;
028      for (int32 i = 0; i < 6; ++i) SumFactors += FactorValues[i];
029      if (SumFactors < KINDA_SMALL_NUMBER) return;
030      const float Eta = Config->WeightLearningRate;
031      const float WMin = Config->WeightMin;
032      const float WMax = Config->WeightMax;
033      for (int32 i = 0; i < 6; ++i)
034      {
035          const float Contribution = FactorValues[i] / SumFactors;
036          Weights[i] = FMath::Clamp(Weights[i] + Eta * Reward * Contribution, WMin,
WMax);
037      }
038      float Sum = 0.f;
039      for (int32 i = 0; i < 6; ++i) Sum += Weights[i];
040      if (Sum > KINDA_SMALL_NUMBER)
041      {
042          const float InvSum = 1.f / Sum;
043          for (int32 i = 0; i < 6; ++i) Weights[i] *= InvSum;
044      }
045      UE_LOG(LogDynamicWeights, Verbose,
046          TEXT("[Weights] reward=%.1f | w0=%.3f w1=%.3f w2=%.3f w3=%.3f w4=%.3f
w5=%.3f"),
047          Reward, Weights[0], Weights[1], Weights[2], Weights[3], Weights[4],
Weights[5]);
048  }

ContextEvaluator.cpp
001  #include "ContextEvaluator.h"
002  #include "Enemy/BaseEnemy.h"
003  #include "Enemy/EnemyHealthComponent.h"
004  #include "Player/DepthrunCharacter.h"
005  #include "Player/PlayerCombatComponent.h"
006  #include "Combat/BaseWeapon.h"
007  #include "AdaptiveBehavior/AdaptiveConfig.h"
008  #include "AdaptiveBehavior/AdaptiveMemory.h"
009  #include "Kismet/KismetSystemLibrary.h"
010  #include "Engine/World.h"
011  FContextData UContextEvaluator::EvaluateContext(
012      const ABaseEnemy* Owner,
013      const ADepthrunCharacter* Player,
014      const UAdaptiveConfig* Config) const
015  {
016      FContextData Data;
017      if (!Owner || !Player || !Config) return Data;
018      float Dist = FVector::Dist2D(Player->GetActorLocation(), Owner->
GetActorLocation());
019      Data.DistanceToPlayer = Dist;
020      Data.DistanceNorm = 1.f - FMath::Clamp(Dist / Config->MaxEngagementRange, 0.f, 1.f);
021      if (Player->CombatComponent && Player->CombatComponent->CurrentWeapon) {
022          Data.PlayerWeaponType = Player->CombatComponent->CurrentWeapon->
GetWeaponType();
023      } else { Data.PlayerWeaponType = EWeaponType::Melee; }

```

```

024     Data.WeaponThreatNorm = GetWeaponThreatNorm(Data.PlayerWeaponType);
025     UEnemyHealthComponent* HC = Owner->FindComponentByClass<UEnemyHealthComponent>();
026     if (HC && HC->GetMaxHP() > 0.f) {
027         float Ratio = HC->GetCurrentHP() / HC->GetMaxHP();
028         Data.EnemyHPRatioNorm = NormalizeHealth(Ratio, Config->HealthPowerExponent);
029     }
030     Data.NearbyAllyCount = CountNearbyAllies(Owner, Config->AllyCheckRadius);
031     Data.AllyCountNorm = FMath::Clamp(static_cast<float>(Data.NearbyAllyCount) /
Config->MaxAllyThreshold, 0.f, 1.f);
032     Data.RoomDensityNorm = EvaluateRoomDensity(Owner, Config->MaxRoomCapacity);
033     return Data;
034 }
035 FContextData UContextEvaluator::EvaluateContextWithMemory(
036     const ABaseEnemy* Owner,
037     const ADepthrunCharacter* Player,
038     const UAdaptiveMemory* Memory,
039     const UAdaptiveConfig* Config) const
040 {
041     FContextData Data = EvaluateContext(Owner, Player, Config);
042     if (Memory && Owner)
043     {
044         float Now = Owner->GetWorld()->GetTimeSeconds();
045         Data.MemoryAggressiveness = Memory->GetDecayedAggressiveness(Now, 0.f);
046         Data.MemoryMobility = Memory->GetDecayedMobility(Now, 0.f);
047     }
048     return Data;
049 }
050 float UContextEvaluator::NormalizeDistance(float RawDistance, float MaxRange) const
051 {
052     if (MaxRange <= 0.f) return 0.f;
053     return 1.f - FMath::Clamp(RawDistance / MaxRange, 0.f, 1.f);
054 }
055 float UContextEvaluator::NormalizeHealth(float HPRatio, float Exponent) const
056 {
057     float InverseRatio = FMath::Clamp(1.f - HPRatio, 0.f, 1.f);
058     return FMath::Pow(InverseRatio, Exponent);
059 }
060 float UContextEvaluator::EvaluateRoomDensity(const ABaseEnemy* Owner, float MaxCapacity)
const
061 {
062     if (!Owner || MaxCapacity <= 0.f) return 0.f;
063     TArray<TEnumAsByte<EObjectTypeQuery>> ObjectTypes;
064     ObjectTypes.Add(UEngineTypes::ConvertToObjectType(ECollisionChannel::ECC_Pawn));
065     TArray<AActor*> IgnoreActors;
066     IgnoreActors.Add(const_cast<ABaseEnemy*>(Owner));
067     TArray<AActor*> OutActors;
068     UKismetSystemLibrary::SphereOverlapActors(
069         Owner, Owner->GetActorLocation(), 600.f, ObjectTypes, APawn::StaticClass(),
IgnoreActors, OutActors);
070     return FMath::Clamp(OutActors.Num() / MaxCapacity, 0.f, 1.f);
071 }
072 int32 UContextEvaluator::CountNearbyAllies(const ABaseEnemy* Owner, float Radius) const
073 {
074     if (!Owner) return 0;
075     TArray<TEnumAsByte<EObjectTypeQuery>> ObjectTypes;
076     ObjectTypes.Add(UEngineTypes::ConvertToObjectType(ECollisionChannel::ECC_Pawn));
077     TArray<AActor*> IgnoreActors;
078     IgnoreActors.Add(const_cast<ABaseEnemy*>(Owner));
079     TArray<AActor*> OutActors;
080     UKismetSystemLibrary::SphereOverlapActors(
081         Owner, Owner->GetActorLocation(), Radius, ObjectTypes,
ABaseEnemy::StaticClass(), IgnoreActors, OutActors);
082     return OutActors.Num();
083 }
084 float UContextEvaluator::GetWeaponThreatNorm(EWeaponType WeaponType) const
085 {
086     switch (WeaponType) {
087         case EWeaponType::Melee: return 0.6f;
088         case EWeaponType::Ranged: return 0.8f;
089         default: return 0.5f;
090     }
091 }

```

TransitionCostMatrix.cpp

```

001 #include "TransitionCostMatrix.h"
002 #include "AdaptiveConfig.h"
003 void UTransitionCostMatrix::InitializeFromConfig(const UAdaptiveConfig* Config)
004 {
005     for (int i = 0; i < 6; ++i) Matrix[0][i] = 1.0f;
006     for (int i = 0; i < 6; ++i) Matrix[i][0] = 1.0f;

```

```

007         const float Defaults[5][5] = {
008             { 0.00f, 0.05f, 0.10f, 0.05f, 0.15f },
009             { 0.10f, 0.00f, 0.05f, 0.15f, 0.10f },
010             { 0.15f, 0.10f, 0.00f, 0.20f, 0.15f },
011             { 0.10f, 0.15f, 0.20f, 0.00f, 0.25f },
012             { 0.15f, 0.10f, 0.10f, 0.15f, 0.00f },
013         };
014         for (int32 Row = 0; Row < 5; ++Row)
015         {
016             for (int32 Col = 0; Col < 5; ++Col) { Matrix[Row + 1][Col + 1] =
Defaults[Row][Col]; }
017         }
018         if (Config && Config->TransitionCostMatrix.Num() == 36)
019         {
020             for (int32 Row = 0; Row < 6; ++Row)
021             {
022                 for (int32 Col = 0; Col < 6; ++Col) { Matrix[Row][Col] = Config-
>TransitionCostMatrix[Row * 6 + Col]; }
023             }
024         }
025         bInitialized = true;
026     }
027     float UTransitionCostMatrix::GetCost(EFSMStateType From, EFSMStateType To) const
028     {
029         if (!bInitialized) return 0.f;
030         const int32 F = static_cast<int32>(From);
031         const int32 T = static_cast<int32>(To);
032         if (F < 0 || F >= 6 || T < 0 || T >= 6) return 0.f;
033         return Matrix[F][T];
034     }
035     float UTransitionCostMatrix::CalculateInertia(EFSMStateType Current, EFSMStateType
Candidate,
036         float TimeInState, const UAdaptiveConfig* Config) const
037     {
038         if (Current != Candidate) return 0.f;
039         const float Rate = Config ? Config->InertiaGrowthRate : 0.02f;
040         const float Max = Config ? Config->InertiaMax : 0.2f;
041         return FMath::Min(Max, Rate * TimeInState);
042     }
043     int32 UTransitionCostMatrix::StateToIndex(EFSMStateType State) const { return
static_cast<int32>(State); }

```

UtilityCurves.cpp

```

001     #include "UtilityCurves.h"
002     #include "AdaptiveConfig.h"
003     #include "Utils/MathUtils.h"
004     float UUtilityCurves::EvaluateUtility(
005         EFSMStateType State, const FThreatAssessment& Threat, const FContextData& Context,
const UAdaptiveConfig* Cfg) const
006     {
007         const float T = Threat.ThreatFinal;
008         switch (State)
009         {
010             case EFSMStateType::Idle: return EvaluateIdle(T);
011             case EFSMStateType::Chase: return EvaluateChase(T, Context, Cfg);
012             case EFSMStateType::Attack: return EvaluateAttack(T, Cfg);
013             case EFSMStateType::Flank: return EvaluateFlank(T, Context, Cfg);
014             case EFSMStateType::Retreat: return EvaluateRetreat(T, Context, Cfg);
015             default: return 0.f;
016         }
017     }
018     float UUtilityCurves::EvaluateIdle(float T) const { return FMath::Max(0.f, 1.f - 6.f * T *
T); }
019     float UUtilityCurves::EvaluateChase(float T, const FContextData& Context, const
UAdaptiveConfig* Cfg) const
020     {
021         const float Center = Cfg ? Cfg->ChaseBellCenter : 0.3f;
022         const float Width = Cfg ? Cfg->ChaseBellWidth : 0.2f;
023         float Utility = DepthrunMath::BellCurve(T, Center, Width);
024         float PushFactor = 0.4f;
025         if (Context.BraveryLevel == EEnemyBravery::Coward) PushFactor = 0.05f;
026         else if (Context.BraveryLevel == EEnemyBravery::Heroic) PushFactor = 0.7f;
027         if (Context.CombatStyle == EEnemyCombatStyle::RangedOriented) PushFactor *= 0.4f;
028         if (Context.MemoryAggressiveness > 0.4f) { Utility += Context.MemoryAggressiveness *
PushFactor; }
029         return FMath::Clamp(Utility, 0.f, 1.2f);
030     }
031     float UUtilityCurves::EvaluateAttack(float T, const UAdaptiveConfig* Cfg) const
032     {
033         const float Center = Cfg ? Cfg->AttackBellCenter : 0.5f;

```

```

034         const float Width = Cfg ? Cfg->AttackBellWidth : 0.18f;
035         return DepthrunMath::BellCurve(T, Center, Width);
036     }
037     float UUtilityCurves::EvaluateFlank(float T, const FContextData& Context, const
UAdaptiveConfig* Cfg) const
038     {
039         const float Center = Cfg ? Cfg->FlankBellCenter : 0.6f;
040         const float Width = Cfg ? Cfg->FlankBellWidth : 0.2f;
041         const float SoloBase = Cfg ? Cfg->FlankSoloBase : 0.75f;
042         const float AllyFactor = SoloBase + ((1.f - SoloBase) * Context.AllyCountNorm);
043         float Utility = DepthrunMath::BellCurve(T, Center, Width) * AllyFactor;
044         if (Context.DistanceToPlayer < 40.f) { Utility *= 0.4f; }
045         return Utility;
046     }
047     float UUtilityCurves::EvaluateRetreat(float T, const FContextData& Context, const
UAdaptiveConfig* Cfg) const
048     {
049         const float Center = Cfg ? Cfg->RetreatSigmoidCenter : 0.75f;
050         const float K = Cfg ? Cfg->RetreatSigmoidSteepness : 12.f;
051         float Utility = DepthrunMath::Sigmoid(T, K, Center);
052         if (Context.BraveryLevel == EEnemyBravery::Coward) Utility *= 1.4f;
053         float SafeZoneDist = 150.f;
054         if (Context.BraveryLevel == EEnemyBravery::Coward) SafeZoneDist = 350.f;
055         if (Context.DistanceToPlayer > SafeZoneDist) { Utility *= 0.4f; }
056         return Utility;
057     }

```

AdaptiveConfig.cpp

```

001     #include "AdaptiveConfig.h"
002     UAdaptiveConfig::UAdaptiveConfig()
003     {
004         TransitionCostMatrix.SetNumUninitialized(36);
005         for(int i=0; i<6; ++i) TransitionCostMatrix[0 * 6 + i] = 1.0f;
006         for(int i=0; i<6; ++i) TransitionCostMatrix[i * 6 + 0] = 1.0f;
007         TransitionCostMatrix[1 * 6 + 1] = 0.00f;
008         TransitionCostMatrix[1 * 6 + 2] = 0.05f;
009         TransitionCostMatrix[1 * 6 + 3] = 0.10f;
010         TransitionCostMatrix[1 * 6 + 4] = 0.05f;
011         TransitionCostMatrix[1 * 6 + 5] = 0.15f;
012         TransitionCostMatrix[2 * 6 + 1] = 0.10f;
013         TransitionCostMatrix[2 * 6 + 2] = 0.00f;
014         TransitionCostMatrix[2 * 6 + 3] = 0.05f;
015         TransitionCostMatrix[2 * 6 + 4] = 0.15f;
016         TransitionCostMatrix[2 * 6 + 5] = 0.10f;
017         TransitionCostMatrix[3 * 6 + 1] = 0.15f;
018         TransitionCostMatrix[3 * 6 + 2] = 0.10f;
019         TransitionCostMatrix[3 * 6 + 3] = 0.00f;
020         TransitionCostMatrix[3 * 6 + 4] = 0.20f;
021         TransitionCostMatrix[3 * 6 + 5] = 0.15f;
022         TransitionCostMatrix[4 * 6 + 1] = 0.10f;
023         TransitionCostMatrix[4 * 6 + 2] = 0.15f;
024         TransitionCostMatrix[4 * 6 + 3] = 0.20f;
025         TransitionCostMatrix[4 * 6 + 4] = 0.00f;
026         TransitionCostMatrix[4 * 6 + 5] = 0.25f;
027         TransitionCostMatrix[5 * 6 + 1] = 0.15f;
028         TransitionCostMatrix[5 * 6 + 2] = 0.10f;
029         TransitionCostMatrix[5 * 6 + 3] = 0.10f;
030         TransitionCostMatrix[5 * 6 + 4] = 0.15f;
031         TransitionCostMatrix[5 * 6 + 5] = 0.00f;
032     }

```

Source\Depthrun\FSM

FSMComponent.cpp

```

001     #include "FSMComponent.h"
002     #include "FSMState.h"
003     #include "Enemy/BaseEnemy.h"
004     #include "DepthrunLogChannels.h"
005     UFSMComponent::UFSMComponent() { PrimaryComponentTick.bCanEverTick = true; }
006     void UFSMComponent::BeginPlay()
007     {
008         Super::BeginPlay();
009         OwnerEnemy = Cast<ABaseEnemy>(GetOwner());
010     }
011     void UFSMComponent::TickComponent(float DeltaTime, ELevelTick TickType,
FActorComponentTickFunction* ThisTickFunction)
012     {
013         Super::TickComponent(DeltaTime, TickType, ThisTickFunction);
014         if (CurrentState && OwnerEnemy) { CurrentState->TickState(OwnerEnemy, DeltaTime); }
015     }
016     void UFSMComponent::RegisterState(EFSMStateType StateType, UFSMState* StateObject)

```

```

018     {
019         if (!StateObject)
020         {
021             UE_LOG(LogFSM, Warning, TEXT("UFSMComponent::RegisterState - null state for
type %d"), (int32)StateType);
022             return;
023         }
024         States.Add(StateType, StateObject);
025         UE_LOG(LogFSM, Verbose, TEXT("UFSMComponent: registered state %d"),
(int32)StateType);
026     }
027 void UFSMComponent::TransitionTo(EFSMStateType NewState)
028 {
029     if (NewState == CurrentStateType) return;
030     const EFSMStateType OldState = CurrentStateType;
031     if (CurrentState && OwnerEnemy) { CurrentState->ExitState(OwnerEnemy); }
032     auto Found = States.Find(NewState);
033     if (!Found || !(*Found))
034     {
035         UE_LOG(LogFSM, Warning, TEXT("UFSMComponent::TransitionTo - state %d not
registered!"), (int32)NewState);
036         return;
037     }
038     CurrentState = *Found;
039     CurrentStateType = NewState;
040     if (CurrentState && OwnerEnemy) { CurrentState->EnterState(OwnerEnemy); }
041     const FString OldName = GetStateName(OldState);
042     const FString NewName = GetStateName(NewState);
043     UE_LOG(LogFSM, Log, TEXT("FSM[%s]: %s → %s"), *GetOwner()->GetName(), *OldName,
*NewName);
044     OnStateChanged.Broadcast(OldState, NewState);
045 }
046 float UFSMComponent::GetTimeInCurrentState() const { return CurrentState ? CurrentState-
>GetTimeInState() : 0.f; }
047 FString UFSMComponent::GetStateName(EFSMStateType State)
048 {
049     const UEnum* EnumPtr = StaticEnum<EFSMStateType>();
050     if (!EnumPtr) return TEXT("Unknown");
051     return EnumPtr->GetDisplayNameTextByValue((int64)State).ToString();
052 }

```

FSMState.cpp

```

001 #include "FSMState.h"
002 void UFSMState::EnterState(ABaseEnemy* Owner) { TimeInState = 0.f; }
003 void UFSMState::TickState(ABaseEnemy* Owner, float DeltaTime) { TimeInState += DeltaTime;
}
004 void UFSMState::ExitState(ABaseEnemy* Owner) { }

```

Source\Depthrun\Enemy

AdaptiveEnemy.cpp

```

001 #include "AdaptiveEnemy.h"
002 #include "Enemy/BaseEnemy.h"
003 #include "Combat/BaseProjectile.h"
004 #include "Combat/RangedWeapon.h"
005 #include "Components/SphereComponent.h"
006 #include "AdaptiveBehavior/AdaptiveBehaviorComponent.h"
007 #include "Core/DepthrunLogChannels.h"
008 #include "Enemy/EnemyHealthComponent.h"
009 #include "FSM/FSMComponent.h"
010 #include "Kismet/GameplayStatics.h"
011 #include "NiagaraFunctionLibrary.h"
012 #include "NiagaraSystem.h"
013 #include "Player/DepthrunCharacter.h"
014 #include "Player/PlayerActionTracker.h"
015 #include "PaperFlipbookComponent.h"
016 AAdaptiveEnemy::AAdaptiveEnemy() {
017     EnemyType = EEnemyType::Adaptive;
018     AdaptiveComp = CreateDefaultSubobject<UAdaptiveBehaviorComponent>(
TEXT("AdaptiveBehaviorComponent"));
019 }
020 void AAdaptiveEnemy::BeginPlay() {
021     Super::BeginPlay();
022     ADepthrunCharacter *Player = Cast<ADepthrunCharacter>(
UGameplayStatics::GetPlayerCharacter(GetWorld(), 0));
023     if (Player && Player->GetActionTracker() && AdaptiveComp) {
024         Player->GetActionTracker()->OnPlayerAction.AddDynamic(
AdaptiveComp, &UAdaptiveBehaviorComponent::HandlePlayerAction);
025     }
026     if (HealthComponent && AdaptiveComp) {
027         HealthComponent->OnHealthChanged.AddDynamic(

```

```

031         this, &AAdaptiveEnemy::HandleHealthChanged);
032     }
033     UE_LOG(LogAdaptiveBehavior, Log, TEXT("[AdaptiveEnemy] Initialized: %s"),
034         *GetName());
035 }
036 void AAdaptiveEnemy::HandleHealthChanged(float OldHP, float NewHP,
037     float MaxHP) {
038     if (NewHP < OldHP) {
039         if (AdaptiveComp) { AdaptiveComp->OnDamageTaken(); }
040         UNiagaraSystem* HitFX = bIsRangedMode ? NS_HitRanged : NS_HitMelee;
041         if (HitFX) {
042             UNiagaraFunctionLibrary::SpawnSystemAtLocation(
043                 GetWorld(), HitFX, GetActorLocation(),
044                 FRotator::ZeroRotator, FVector(1.f), true, true,
045                 ENCPoolMethod::AutoRelease);
046         }
047     }
048 }
049 void AAdaptiveEnemy::PerformMeleeAttack() {
050     if (bIsRangedMode) {
051         if (!ProjectileClass)
052         {
053             UE_LOG(LogAdaptiveBehavior, Warning,
054                 TEXT("[AdaptiveEnemy] %s bIsRangedMode=true but ProjectileClass is NULL -
055 assign it in BP!"),
056                 *GetName());
057             return;
058         }
059         UE_LOG(LogAdaptiveBehavior, Log, TEXT("[AdaptiveEnemy] %s firing ranged projectile
060 (ShotDelay=%.2f)"),
061             *GetName(), ShotDelay);
062         if (ShotDelay > 0.01f) {
063             FTimerHandle Timer;
064             GetWorldTimerManager().SetTimer(Timer, this, &AAdaptiveEnemy::ActuallyFire,
065                 ShotDelay, false);
066         } else { ActuallyFire(); }
067     } else { Super::PerformMeleeAttack(); }
068     if (AdaptiveComp) { AdaptiveComp->OnDamageDealt(); }
069 }
070 void AAdaptiveEnemy::ActuallyFire() {
071     ACharacter *Player = UGameplayStatics::GetPlayerCharacter(this, 0);
072     if (!IsValid(Player))
073     {
074         return;
075     }
076     const FVector EnemyLoc = GetActorLocation();
077     const FVector PlayerLoc = Player->GetActorLocation();
078     const FVector FireDir = (PlayerLoc - EnemyLoc).GetSafeNormal2D();
079     const FVector SpawnLoc =
080         EnemyLoc + (FireDir * MuzzleOffset) + FVector(0.f, 0.f, 0.f);
081     const FTransform SpawnTransform = FTransform(FireDir.Rotation(), SpawnLoc);
082     UWorld *World = GetWorld();
083     if (!World)
084     {
085         return;
086     }
087     ABaseProjectile *Projectile = World->SpawnActorDeferred<ABaseProjectile>(
088         ProjectileClass, SpawnTransform, this, Cast<APawn>(this),
089         ESpawnActorCollisionHandlingMethod::AlwaysSpawn);
090     if (!IsValid(Projectile))
091     {
092         return;
093     }
094     Projectile->CollisionSphere->IgnoreActorWhenMoving(this, true);
095     Projectile->InitProjectile(FireDir, AttackDamage, this, ProjectileSpeed);
096     Projectile->FinishSpawning(SpawnTransform);
097 }
098 void AAdaptiveEnemy::OnKilled() {
099     if (bIsRangedMode && FB_Death_Ranged) { FB_Death = FB_Death_Ranged; }
100     Super::OnKilled();
101     UE_LOG(LogAdaptiveBehavior, Log, TEXT("[AdaptiveEnemy] Killed: cleaning up
102 evaluation."));
103 }
104 void AAdaptiveEnemy::UpdateAnimation() {
105     if (!GetSprite() || bIsDead)
106     {
107         return;
108     }
109     UPaperFlipbook *DesiredFB = nullptr;
110     const bool bAttacking = FSMComponent && FSMComponent->GetCurrentStateType() ==
111 EFSMStateType::Attack;
112     bool bRetreatShooting = false;
113     if (FSMComponent && FSMComponent->GetCurrentStateType() == EFSMStateType::Retreat &&
114 bIsRangedMode) { }
115     if (bIsHitAnimationActive) {
116         if (bIsRangedMode) {
117             DesiredFB = FB_Hit_Ranged ? FB_Hit_Ranged : FB_Hit;
118         } else { DesiredFB = FB_Hit; }
119     }

```

```

106     } else if (bIsRangedMode) {
107         if (bAttacking) {
108             DesiredFB = FB_Attack_Ranged ? FB_Attack_Ranged : FB_Idle_Ranged;
109         } else if (GetVelocity().SizeSquared() > 10.f) {
110             DesiredFB = FB_Walk_Ranged ? FB_Walk_Ranged : FB_Idle_Ranged;
111         } else { DesiredFB = FB_Idle_Ranged; }
112     } else {
113         if (bAttacking) {
114             DesiredFB = FB_Attack ? FB_Attack : FB_Idle;
115         } else if (GetVelocity().SizeSquared() > 10.f) {
116             DesiredFB = FB_Walk ? FB_Walk : FB_Idle;
117         } else { DesiredFB = FB_Idle; }
118     }
119     if (DesiredFB && GetSprite()->GetFlipbook() != DesiredFB) { GetSprite()-
>SetFlipbook(DesiredFB); }
120     float FacingY = 0.f;
121     const FVector V = GetVelocity();
122     if (FMath::Abs(V.Y) > 0.1f) { FacingY = V.Y; }
123     else if (ACharacter* Player = UGameplayStatics::GetPlayerCharacter(this, 0))
124     {
125         const FVector ToPlayer = Player->GetActorLocation() - GetActorLocation();
126         if (FMath::Abs(ToPlayer.Y) > 1.f) { FacingY = ToPlayer.Y; }
127     }
128     if (FMath::Abs(FacingY) > 0.05f)
129     {
130         FVector Scale = GetSprite()->GetRelativeScale3D();
131         const float XAbs = FMath::Max(FMath::Abs(Scale.X), 1.0f);
132         Scale.X = (FacingY < 0.f) ? -XAbs : XAbs;
133         GetSprite()->SetRelativeScale3D(Scale);
134     }
135 }

```

Source\Depthrun\Player

DepthrunCharacter.cpp

```

001 #include "DepthrunCharacter.h"
002 #include "Combat/BaseWeapon.h"
003 #include "Combat/MeleeWeapon.h"
004 #include "Combat/RangedWeapon.h"
005 #include "Core/DepthrunLogChannels.h"
006 #include "Data/DepthrunSaveSubsystem.h"
007 #include "Data/SQLiteManager.h"
008 #include "Audio/CombatMusicTrigger.h"
009 #include "Data/HubUpgradeTypes.h"
010 #include "Items/RunItemInventory.h"
011 #include "Items/RunItemCollection.h"
012 #include "RoomGeneration/RoomGeneratorSubsystem.h"
013 #include "PlayerActionTracker.h"
014 #include "PlayerCombatComponent.h"
015 #include "PlayerEconomy.h"
016 #include "PlayerMovementConfig.h"
017 #include "Camera/CameraComponent.h"
018 #include "Components/CapsuleComponent.h"
019 #include "EnhancedInputComponent.h"
020 #include "EnhancedInputSubsystems.h"
021 #include "GameFramework/CharacterMovementComponent.h"
022 #include "GameFramework/SpringArmComponent.h"
023 #include "PaperFlipbook.h"
024 #include "PaperFlipbookComponent.h"
025 #include "UI/DepthrunHUD.h"
026 #include "UI/HUDOverlayWidget.h"
027 #include "UI/PauseMenuWidget.h"
028 #include "UI/DeathScreenWidget.h"
029 #include "UI/VictoryScreenWidget.h"
030 #include "Blueprint/UserWidget.h"
031 #include "Kismet/GameplayStatics.h"
032 namespace { constexpr float DashStopDelay = 0.12f; }
033 ADepthrunCharacter::ADepthrunCharacter() {
034     PrimaryActorTick.bCanEverTick = true;
035     SpringArm = CreateDefaultSubobject<USpringArmComponent>(TEXT("SpringArm"));
036     SpringArm->SetupAttachment(RootComponent);
037     SpringArm->TargetArmLength = 700.f;
038     SpringArm->SetRelativeRotation(
039         FRotator(-90.f, 0.f, 0.f));
040     SpringArm->bDoCollisionTest = false;
041     SpringArm->bInheritPitch = false;
042     SpringArm->bInheritYaw = false;
043     SpringArm->bInheritRoll = false;
044     FollowCamera = CreateDefaultSubobject<UCameraComponent>(TEXT("FollowCamera"));
045     FollowCamera->SetupAttachment(SpringArm, USpringArmComponent::SocketName);
046     FollowCamera->SetProjectionMode(ECameraProjectionMode::Orthographic);

```

```

047 FollowCamera->OrthoWidth = 1024.f;
048 FollowCamera->PostProcessSettings.bOverride_MotionBlurAmount = true;
049 FollowCamera->PostProcessSettings.MotionBlurAmount = 0.f;
050 FollowCamera->PostProcessSettings.bOverride_DepthOfFieldFstop = true;
051 FollowCamera->PostProcessSettings.DepthOfFieldFstop = 32.f;
052 FollowCamera->PostProcessSettings.bOverride_DynamicGlobalIlluminationMethod =
053     true;
054 FollowCamera->PostProcessSettings.DynamicGlobalIlluminationMethod =
055     EDynamicGlobalIlluminationMethod::None;
056 FollowCamera->PostProcessSettings.bOverride_ReflectionMethod = true;
057 FollowCamera->PostProcessSettings.ReflectionMethod = EReflectionMethod::None;
058 FollowCamera->PostProcessSettings.bOverride_BloomIntensity = true;
059 FollowCamera->PostProcessSettings.BloomIntensity = 0.f;
060 FollowCamera->PostProcessSettings.bOverride_AmbientOcclusionIntensity = true;
061 FollowCamera->PostProcessSettings.AmbientOcclusionIntensity = 0.f;
062 CombatComponent =
063     CreateDefaultSubobject<UPlayerCombatComponent>(TEXT("CombatComponent"));
064 ActionTracker =
065     CreateDefaultSubobject<UPlayerActionTracker>(TEXT("ActionTracker"));
066 ItemInventory =
067     CreateDefaultSubobject<URunItemInventory>(TEXT("ItemInventory"));
068 CombatMusicTrigger =
069     CreateDefaultSubobject<UCombatMusicTrigger>(TEXT("CombatMusicTrigger"));
070 PlayerEconomy =
071     CreateDefaultSubobject<UPlayerEconomy>(TEXT("PlayerEconomy"));
072 GetCharacterMovement()->bOrientRotationToMovement = false;
073 GetCharacterMovement()->GravityScale = 0.f;
074 GetCharacterMovement()->MaxFlySpeed = 450.f;
075 GetCharacterMovement()->MaxWalkSpeed = 450.f;
076 GetCharacterMovement()->MaxAcceleration = 20480.f;
077 GetCharacterMovement()->BrakingDecelerationFlying =
078     20480.f;
079 GetCharacterMovement()->BrakingDecelerationWalking = 20480.f;
080 GetCharacterMovement()->BrakingDecelerationFalling =
081     20480.f;
082 GetCharacterMovement()->GroundFriction = 12.f;
083 GetCharacterMovement()->BrakingFrictionFactor = 1.f;
084 GetCharacterMovement()->bUseSeparateBrakingFriction = true;
085 GetCharacterMovement()->BrakingFriction = 10.f;
086 GetCharacterMovement()->SetMovementMode(MOVE_Flying);
087 GetCharacterMovement()->bConstrainToPlane = true;
088 GetCharacterMovement()->bSnapToPlaneAtStart = true;
089 GetCharacterMovement()->SetPlaneConstraintNormal(FVector(0.f, 0.f, 1.f));
090 GetCapsuleComponent()->SetCapsuleHalfHeight(4.f);
091 GetCapsuleComponent()->SetCapsuleRadius(12.f);
092 if (GetSprite()) { GetSprite()->SetRelativeRotation(SpriteRotationOffset); }
093 GetCapsuleComponent()->SetCapsuleRadius(12.f);
094 bUseControllerRotationPitch = false;
095 bUseControllerRotationYaw = false;
096 bUseControllerRotationRoll = false;
097 }
098 void ADepthrunCharacter::PostInitializeComponents()
099 {
100     Super::PostInitializeComponents();
101     ApplyProfileUpgrades();
102 }
103 void ADepthrunCharacter::BeginPlay() {
104     Super::BeginPlay();
105     CurrentHP = MaxHP;
106     bCanDash = true;
107     RunStartTime = GetWorld() ? GetWorld()->GetTimeSeconds() : 0.f;
108     if (MovementConfig) {
109         UCharacterMovementComponent *MoveComp = GetCharacterMovement();
110         MoveComp->MaxFlySpeed = MovementConfig->MaxWalkSpeed;
111         MoveComp->MaxWalkSpeed = MovementConfig->MaxWalkSpeed;
112         MoveComp->MaxAcceleration = MovementConfig->MaxAcceleration;
113         MoveComp->BrakingDecelerationFlying =
114             MovementConfig->BrakingDecelerationWalking;
115         MoveComp->BrakingDecelerationWalking =
116             MovementConfig->BrakingDecelerationWalking;
117         MoveComp->BrakingDecelerationFalling =
118             MovementConfig->BrakingDecelerationWalking;
119         MoveComp->GroundFriction = MovementConfig->GroundFriction;
120         MoveComp->BrakingFrictionFactor = MovementConfig->BrakingFrictionFactor;
121         MoveComp->bUseSeparateBrakingFriction =
122             MovementConfig->bUseSeparateBrakingFriction;
123         MoveComp->BrakingFriction = MovementConfig->BrakingFriction;
124         DashImpulse = MovementConfig->DashImpulse;
125         DashCooldown = MovementConfig->DashCooldown;
126     }

```



```

127     if (APlayerController *PC = Cast<APlayerController>(GetController())) {
128         if (UEnhancedInputLocalPlayerSubsystem *Subsystem =
129             ULocalPlayer::GetSubsystem<UEnhancedInputLocalPlayerSubsystem>(
130                 PC->GetLocalPlayer())) {
131             if (DefaultMappingContext) { Subsystem->AddMappingContext(DefaultMappingContext, 0);
132         }
133     }
134     auto SpawnWeapon = [this](TSubclassOf<ABaseWeapon> WClass) -> ABaseWeapon * {
135         if (!WClass)
136             return nullptr;
137         FActorSpawnParameters P;
138         P.Owner = this;
139         P.SpawnCollisionHandlingOverride =
140             ESpawnActorCollisionHandlingMethod::AlwaysSpawn;
141         return GetWorld()->SpawnActor<ABaseWeapon>(WClass, GetActorTransform(), P);
142     };
143     SpawnedWeapon1 = SpawnWeapon(WeaponSlotClass1);
144     if (SpawnedWeapon1)
145         SpawnedWeapon1->AttachToComponent(
146             GetRootComponent(),
147             FAttachmentTransformRules::SnapToTargetIncludingScale);
148     SpawnedWeapon2 = SpawnWeapon(WeaponSlotClass2);
149     if (SpawnedWeapon2)
150         SpawnedWeapon2->AttachToComponent(
151             GetRootComponent(),
152             FAttachmentTransformRules::SnapToTargetIncludingScale);
153     auto DisableWeapon = [](ABaseWeapon *W) {
154         if (W) {
155             W->SetActorHiddenInGame(true);
156             W->SetActorEnableCollision(false);
157         }
158     };
159     DisableWeapon(SpawnedWeapon1);
160     DisableWeapon(SpawnedWeapon2);
161     SwitchToWeaponSlot(1);
162     ApplyWeaponProfileUpgrades();
163     if (!SpawnedWeapon1 && !SpawnedWeapon2) {
164         UE_LOG(
165             LogDepthrun, Warning,
166             TEXT("[Character] No weapon slots assigned in BP_DepthrunCharacter!"));
167     }
168     UE_LOG(LogDepthrun, Log,
169         TEXT("ADepthrunCharacter::BeginPlay - player ready, HP=%.0f"), MaxHP);
170     if (APlayerController* PC = Cast<APlayerController>(GetController()))
171     {
172         if (ADepthrunHUD* HUD = Cast<ADepthrunHUD>(PC->GetHUD())) { HUD->UpdatePlayerHP(CurrentHP, MaxHP); }
173     }
174     SetActorHiddenInGame(false);
175     if (UPaperFlipbookComponent* PlayerSprite = GetSprite()) {
176         PlayerSprite->SetHiddenInGame(false);
177         PlayerSprite->SetVisibility(true, true);
178         PlayerSprite->MarkRenderStateDirty();
179     }
180     UpdateAnimation();
181 }
182 void ADepthrunCharacter::SetupPlayerInputComponent(
183     UInputComponent *PlayerInputComponent) {
184     Super::SetupPlayerInputComponent(PlayerInputComponent);
185     if (UEnhancedInputComponent *EIC =
186         Cast<UEnhancedInputComponent>(PlayerInputComponent)) {
187         if (IA_Move) {
188             EIC->BindAction(IA_Move, ETriggerEvent::Triggered, this,
189                 &ADepthrunCharacter::HandleMove);
190             EIC->BindAction(IA_Move, ETriggerEvent::Completed, this,
191                 &ADepthrunCharacter::HandleMove);
192         }
193         if (IA_Attack)
194             EIC->BindAction(IA_Attack, ETriggerEvent::Started, this,
195                 &ADepthrunCharacter::HandleAttack);
196         if (IA_Dash)
197             EIC->BindAction(IA_Dash, ETriggerEvent::Started, this,
198                 &ADepthrunCharacter::HandleDash);
199         if (IA_SwitchSlot1)
200             EIC->BindAction(IA_SwitchSlot1, ETriggerEvent::Started, this,
201                 &ADepthrunCharacter::HandleSwitchSlot1);
202         if (IA_SwitchSlot2)
203             EIC->BindAction(IA_SwitchSlot2, ETriggerEvent::Started, this,
204                 &ADepthrunCharacter::HandleSwitchSlot2);

```

```

205     if (IA_UsePotion)
206         EIC->BindAction(IA_UsePotion, ETriggerEvent::Started, this,
207             &ADepthrunCharacter::HandleUsePotion);
208     if (IA_Pause)
209         EIC->BindAction(IA_Pause, ETriggerEvent::Started, this,
210             &ADepthrunCharacter::HandlePause);
211 }
212 PlayerInputComponent->BindKey(EKeys::Y, IE_Pressed, this,
213     &ADepthrunCharacter::ToggleGodMode);
214 PlayerInputComponent->BindKey(EKeys::U, IE_Pressed, this,
215     &ADepthrunCharacter::ToggleSuperAttack);
216 }
217 void ADepthrunCharacter::HandleMove(const FInputActionValue &Value) {
218     if (bIsDead) return;
219     const FVector2D Input = Value.Get<FVector2D>();
220     bIsMoving = !Input.IsNearlyZero();
221     if (bIsMoving) {
222         UpdateFacingDirection(Input);
223         AddMovementInput(FVector(Input.Y, Input.X, 0.f));
224     }
225     UpdateAnimation();
226 }
227 void ADepthrunCharacter::HandleAttack(const FInputActionValue &Value) {
228     if (bIsDead) return;
229     if (!CombatComponent || !IsValid(CombatComponent->CurrentWeapon)) {
230         UE_LOG(LogTemp, Warning,
231             TEXT("[HandleAttack] Failed: No weapon equipped. Assign "
232                 "DefaultWeaponClass in BP_DepthrunCharacter!"));
233         return;
234     }
235     float OriginalDamage = 0.f;
236     float OriginalRange = 1.0f;
237     float OriginalWidth = 1.0f;
238     float OriginalThickness = 1.0f;
239     bool bOriginalOmniSwing = false;
240     AMeleeWeapon *MeleeW = Cast<AMeleeWeapon>(CombatComponent->CurrentWeapon);
241     if (bSuperAttack && CombatComponent->CurrentWeapon) {
242         OriginalDamage = CombatComponent->CurrentWeapon->BaseDamage;
243         CombatComponent->CurrentWeapon->BaseDamage = 9999.f;
244         if (MeleeW) {
245             OriginalRange = MeleeW->GetRangeMultiplier();
246             OriginalWidth = MeleeW->GetWidthMultiplier();
247             OriginalThickness = MeleeW->GetThicknessMultiplier();
248             bOriginalOmniSwing = MeleeW->IsOmniSwing();
249             MeleeW->SetRangeMultiplier(5.0f);
250             MeleeW->SetWidthMultiplier(4.0f);
251             MeleeW->SetThicknessMultiplier(5.0f);
252             MeleeW->SetOmniSwing(true);
253         }
254     }
255     CombatComponent->CurrentWeapon->SetFireDirection(GetFireDirection());
256     CombatComponent->Attack();
257     if (bSuperAttack && CombatComponent->CurrentWeapon) {
258         CombatComponent->CurrentWeapon->BaseDamage = OriginalDamage;
259         if (MeleeW) {
260             MeleeW->SetRangeMultiplier(OriginalRange);
261             MeleeW->SetWidthMultiplier(OriginalWidth);
262             MeleeW->SetThicknessMultiplier(OriginalThickness);
263             MeleeW->SetOmniSwing(bOriginalOmniSwing);
264         }
265     }
266     bIsAttacking = true;
267     UpdateAnimation();
268     const float CooldownDur =
269         CombatComponent->CurrentWeapon->AttackCooldown > 0.05f
270         ? CombatComponent->CurrentWeapon->AttackCooldown
271         : 0.3f;
272     GetWorldTimerManager().SetTimer(
273         AttackAnimTimer,
274         [this]() {
275             bIsAttacking = false;
276             UpdateAnimation();
277         },
278         CooldownDur, false);
279     const EPlayerActionType ActionType =
280         (CombatComponent->CurrentWeapon->GetWeaponType() == EWeaponType::Ranged)
281         ? EPlayerActionType::Shot
282         : EPlayerActionType::MeleeAttack;
283     ActionTracker->RecordAction(ActionType, GetActorLocation());
284 }

```

```

285 void ADepthrunCharacter::HandleDash(const FInputActionValue &Value) {
286     if (bIsDead) return;
287     Dash();
288 }
289 void ADepthrunCharacter::HandleSwitchSlot1(const FInputActionValue &) {
    SwitchToWeaponSlot(1); }
290 void ADepthrunCharacter::HandleSwitchSlot2(const FInputActionValue &) {
    SwitchToWeaponSlot(2); }
291 void ADepthrunCharacter::HandleUsePotion(const FInputActionValue &) {
292     if (PlayerEconomy) { PlayerEconomy->UsePotion(); }
293 }
294 void ADepthrunCharacter::HandlePause(const FInputActionValue &) { TogglePause(); }
295 void ADepthrunCharacter::TogglePause() {
296     if (bIsDead) return;
297     APlayerController* PC = Cast<APlayerController>(GetController());
298     if (!PC) return;
299     const bool bPaused = UGameplayStatics::IsGamePaused(GetWorld());
300     if (!bPaused) {
301         if (PauseMenuWidgetClass && !IsValid(ActivePauseWidget)) {
302             ActivePauseWidget = CreateWidget<UPauseMenuWidget>(PC, PauseMenuWidgetClass);
303             if (ActivePauseWidget) {
304                 ActivePauseWidget->OnPauseClosed.AddLambda([this, PC]()
305                 {
306                     ActivePauseWidget = nullptr;
307                     if (PC)
308                     {
309                         PC->SetInputMode(FInputModeGameOnly());
310                         PC->bShowMouseCursor = false;
311                     }
312                 });
313                 ActivePauseWidget->AddToViewport(10);
314                 ActivePauseWidget->Show();
315             }
316         }
317         UGameplayStatics::SetGamePaused(GetWorld(), true);
318         PC->SetInputMode(FInputModeUIOnly());
319         PC->bShowMouseCursor = true;
320     } else {
321         if (IsValid(ActivePauseWidget)) {
322             ActivePauseWidget->RemoveFromParent();
323             ActivePauseWidget = nullptr;
324         }
325         UGameplayStatics::SetGamePaused(GetWorld(), false);
326         PC->SetInputMode(FInputModeGameOnly());
327         PC->bShowMouseCursor = false;
328     }
329 }
330 void ADepthrunCharacter::ShowVictoryScreen() {
331     APlayerController* PC = Cast<APlayerController>(GetController());
332     if (!PC || !VictoryScreenWidgetClass) return;
333     const float RunTime = GetWorld() ? (GetWorld()->GetTimeSeconds() - RunStartTime) : 0.f;
334     int32 EarnedDiamonds = PlayerEconomy ? PlayerEconomy->RunDiamonds : 0;
335     if (PlayerEconomy) PlayerEconomy->OnRunExitToHub();
336     int32 TotalDiamonds = 0;
337     int32 ClearedRooms = 0;
338     int32 TotalRooms = 0;
339     if (UDepthrunSaveSubsystem* Save = GetGameInstance() ? GetGameInstance()-
>GetSubsystem<UDepthrunSaveSubsystem>() : nullptr) {
340         if (URoomGeneratorSubsystem* RoomGen = GetWorld()-
>GetSubsystem<URoomGeneratorSubsystem>()) {
341             TotalRooms = RoomGen->GetTotalRooms();
342             ClearedRooms = RoomGen->GetClearedRoomsCount();
343             Save->SaveRunResult(TotalRooms, FMath::RoundToInt(RunTime), true);
344         }
345         TotalDiamonds = Save->GetTotalDiamonds();
346     }
347     UVictoryScreenWidget* Widget = CreateWidget<UVictoryScreenWidget>(PC,
VictoryScreenWidgetClass);
348     if (Widget) {
349         Widget->AddToViewport(20);
350         Widget->Show(RunTime, ClearedRooms, TotalRooms, EarnedDiamonds, TotalDiamonds);
351     }
352     DisableInput(PC);
353     PC->SetInputMode(FInputModeUIOnly());
354     PC->bShowMouseCursor = true;
355 }
356 void ADepthrunCharacter::SwitchToWeaponSlot(int32 SlotIndex) {
357     auto UpdateWeaponState = [](ABaseWeapon *W, bool bActive) {
358         if (W) {
359             W->SetActorHiddenInGame(!bActive);

```

```

360         W->SetActorEnableCollision(bActive);
361     }
362 };
363 if (SlotIndex == 1 && SpawnedWeapon1) {
364     UpdateWeaponState(SpawnedWeapon1, true);
365     UpdateWeaponState(SpawnedWeapon2, false);
366     if (CombatComponent)
367         CombatComponent->EquipWeapon(SpawnedWeapon1);
368     if (ItemInventory)
369         ItemInventory->ApplyToWeapon(SpawnedWeapon1);
370     ActiveWeaponSlot = 1;
371     UE_LOG(LogDepthrun, Log, TEXT("[Weapon] Switched to Slot 1 (Sword)"));
372 } else if (SlotIndex == 2 && SpawnedWeapon2) {
373     UpdateWeaponState(SpawnedWeapon1, false);
374     UpdateWeaponState(SpawnedWeapon2, true);
375     if (CombatComponent)
376         CombatComponent->EquipWeapon(SpawnedWeapon2);
377     if (ItemInventory)
378         ItemInventory->ApplyToWeapon(SpawnedWeapon2);
379     ActiveWeaponSlot = 2;
380     UE_LOG(LogDepthrun, Log, TEXT("[Weapon] Switched to Slot 2 (Bow)"));
381 }
382 UpdateAnimation();
383 if (APlayerController* PC = Cast<APlayerController>(GetController()))
384     if (ADepthrunHUD* HUD = Cast<ADepthrunHUD>(PC->GetHUD()))
385         if (UHUDOverlayWidget* Overlay = HUD->GetHUDOverlay())
386             Overlay->SetActiveWeaponSlot(ActiveWeaponSlot - 1);
387 }
388 void ADepthrunCharacter::Dash() {
389     if (!bCanDash || bIsDead)
390         return;
391     bCanDash = false;
392     const FVector DashDir = GetVelocity().IsNearlyZero()
393         ? GetFireDirection()
394         : GetVelocity().GetSafeNormal();
395     UCharacterMovementComponent *MoveComp = GetCharacterMovement();
396     const float Impulse =
397         MovementConfig ? MovementConfig->DashImpulse : DashImpulse;
398     const float Duration = MovementConfig ? MovementConfig->DashDuration : 0.15f;
399     const float CancelFactor =
400         MovementConfig ? MovementConfig->PostDashVelocityCancelFactor : 0.1f;
401     MoveComp->BrakingDecelerationFlying = 0.f;
402     MoveComp->BrakingFriction = 0.f;
403     MoveComp->GroundFriction = 0.f;
404     MoveComp->MaxFlySpeed = 9999.f;
405     MoveComp->Velocity = DashDir * Impulse;
406     GetWorldTimerManager().SetTimer(
407         DashStopTimer,
408         [this, CancelFactor]() {
409             UCharacterMovementComponent *MC = GetCharacterMovement();
410             if (!MC)
411                 return;
412             const float BrakeDec = MovementConfig
413                 ? MovementConfig->BrakingDecelerationWalking
414                 : 20480.f;
415             const float BrakeFric =
416                 MovementConfig ? MovementConfig->BrakingFriction : 10.f;
417             const float GndFric =
418                 MovementConfig ? MovementConfig->GroundFriction : 12.f;
419             const float FlySpeed =
420                 MovementConfig ? MovementConfig->MaxWalkSpeed : 450.f;
421             MC->BrakingDecelerationFlying = BrakeDec;
422             MC->BrakingFriction = BrakeFric;
423             MC->GroundFriction = GndFric;
424             MC->MaxFlySpeed = FlySpeed;
425             MC->Velocity *= CancelFactor;
426             if (CancelFactor <= 0.05f) { MC->StopMovementImmediately(); }
427         },
428         Duration, false);
429     ActionTracker->RecordAction(EPlayerActionType::Dash, GetActorLocation());
430     UE_LOG(LogDepthrun, Verbose,
431         TEXT("ADepthrunCharacter::Dash dir=(%.1f,%.1f,%.1f) impulse=%.0f "
432             "dur=%.2fs"),
433         DashDir.X, DashDir.Y, DashDir.Z, Impulse, Duration);
434     const float Cooldown =
435         MovementConfig ? MovementConfig->DashCooldown : DashCooldown;
436     GetWorldTimerManager().SetTimer(DashCooldownTimer, this,
437         &ADepthrunCharacter::OnDashCooldownEnd,
438         Cooldown, false);
439 }

```

```

440 void ADepthrunCharacter::OnDashCooldownEnd() { bCanDash = true; }
441 void ADepthrunCharacter::Tick(float DeltaSeconds)
442 {
443     Super::Tick(DeltaSeconds);
444     UpdateChromaticAberration(DeltaSeconds);
445 }
446 void ADepthrunCharacter::TriggerChromaticAberration()
447 {
448     CA_HoldTimeRemaining = HitFX_HoldDuration;
449     bCA_FadingOut = false;
450 }
451 void ADepthrunCharacter::UpdateChromaticAberration(float DeltaSeconds)
452 {
453     if (!FollowCamera) return;
454     if (CA_CurrentIntensity <= 0.f && bCA_FadingOut) return;
455     if (!bCA_FadingOut)
456     {
457         CA_CurrentIntensity = FMath::Min(
458             CA_CurrentIntensity + HitFX_FadeInSpeed * DeltaSeconds,
459             HitFX_Intensity);
460         if (CA_CurrentIntensity >= HitFX_Intensity)
461         {
462             CA_HoldTimeRemaining -= DeltaSeconds;
463             if (CA_HoldTimeRemaining <= 0.f)
464                 bCA_FadingOut = true;
465         }
466     }
467     else
468     {
469         const float FadeSpeed = (HitFX_FadeOutDuration > 0.f)
470             ? (HitFX_Intensity / HitFX_FadeOutDuration)
471             : 1.f;
472         CA_CurrentIntensity = FMath::Max(
473             CA_CurrentIntensity - FadeSpeed * DeltaSeconds, 0.f);
474     }
475     FollowCamera->PostProcessSettings.bOverride_SceneFringeIntensity = true;
476     FollowCamera->PostProcessSettings.SceneFringeIntensity = CA_CurrentIntensity;
477     FollowCamera->PostProcessSettings.bOverride_ChromaticAberrationStartOffset = true;
478     FollowCamera->PostProcessSettings.ChromaticAberrationStartOffset = 0.f;
479 }
480 float ADepthrunCharacter::TakeDamage(float DamageAmount,
481                                     FDamageEvent const &DamageEvent,
482                                     AController *EventInstigator,
483                                     AActor *DamageCauser) {
484     if (DamageAmount <= 0.f || bIsDead || bGodMode)
485         return 0.f;
486     const float ActualDamage = Super::TakeDamage(DamageAmount, DamageEvent,
487                                                 EventInstigator, DamageCauser);
488     CurrentHP -= ActualDamage;
489     if (ActualDamage > 0.f) {
490         bIsHitAnimationActive = true;
491         GetWorldTimerManager().SetTimer(HitAnimTimer, this,
492                                         &ADepthrunCharacter::ResetHitAnimation,
493                                         0.2f, false);
494         TriggerChromaticAberration();
495     }
496     UE_LOG(LogDepthrun, Log,
497         TEXT("[Player] Took %.1f damage from %s. Remaining HP: %.1f"), ActualDamage,
498         DamageCauser ? *DamageCauser->GetName() : TEXT("Unknown"), CurrentHP);
499     if (CurrentHP <= 0.f) { Die(); }
500     if (APlayerController* PC = Cast<APlayerController>(GetController()))
501     {
502         if (ADepthrunHUD* HUD = Cast<ADepthrunHUD>(PC->GetHUD())) { HUD-
503 >UpdatePlayerHP(CurrentHP, MaxHP); }
504     }
505     UpdateAnimation();
506     return ActualDamage;
507 }
508 void ADepthrunCharacter::Die() {
509     if (bIsDead)
510         return;
511     bIsDead = true;
512     UE_LOG(LogDepthrun, Error, TEXT("[Player] DIED!"));
513     OnPlayerDeath.Broadcast();
514     if (UDepthrunSaveSubsystem* Save = GetGameInstance() ? GetGameInstance()-
515 >GetSubsystem<UDepthrunSaveSubsystem>() : nullptr)
516     {
517         const float Duration = GetWorld() ? (GetWorld()->GetTimeSeconds() - RunStartTime) :
518         0.f;
519         int32 ClearedRooms = 0;

```

```

517         if (URoomGeneratorSubsystem* RoomGen = GetWorld()-
>GetSubsystem<URoomGeneratorSubsystem>()) { ClearedRooms = RoomGen->GetClearedRoomsCount();
}
518         Save->SaveRunResult(ClearedRooms, FMath::RoundToInt(Duration), false);
519     }
520     const int32 EarnedDiamondsForUI = PlayerEconomy ? PlayerEconomy->RunDiamonds : 0;
521     if (PlayerEconomy) { PlayerEconomy->OnPlayerDeath(); }
522     if (UCharacterMovementComponent* MoveComp = GetCharacterMovement())
523     {
524         MoveComp->StopMovementImmediately();
525         MoveComp->DisableMovement();
526     }
527     ConsumeMovementInputVector();
528     SetActorEnableCollision(false);
529     bIsMoving = false;
530     if (APlayerController* PC = Cast<APlayerController>(GetController()))
531     {
532         DisableInput(PC);
533         if (UEnhancedInputLocalPlayerSubsystem* EIS =
534             ULocalPlayer::GetSubsystem<UEnhancedInputLocalPlayerSubsystem>(
535                 PC->GetLocalPlayer())) { EIS->ClearAllMappings(); }
536         UE_LOG(LogDepthrun, Log, TEXT("[Player] Input disabled after death"));
537     }
538     if (FB_Death && GetSprite()) {
539         GetSprite()->SetFlipbook(FB_Death);
540         GetSprite()->SetLooping(false);
541         GetSprite()->Play();
542     }
543     if (APlayerController* PC2 = Cast<APlayerController>(GetController())) {
544         if (DeathScreenWidgetClass) {
545             const float RunTime = GetWorld() ? (GetWorld()->GetTimeSeconds() - RunStartTime) :
0.f;
546             int32 EarnedDiamonds = EarnedDiamondsForUI;
547             int32 TotalDiamonds = 0;
548             int32 TotalRooms = 0;
549             int32 ClearedRooms = 0;
550             if (URoomGeneratorSubsystem* RoomGen = GetWorld()-
>GetSubsystem<URoomGeneratorSubsystem>()) {
551                 ClearedRooms = RoomGen->GetClearedRoomsCount();
552                 TotalRooms = RoomGen->GetTotalRooms();
553             }
554             if (UDepthrunSaveSubsystem* Save = GetGameInstance() ? GetGameInstance()-
>GetSubsystem<UDepthrunSaveSubsystem>() : nullptr)
555                 TotalDiamonds = Save->GetTotalDiamonds();
556             UDeathScreenWidget* DeathWidget = CreateWidget<UDeathScreenWidget>(PC2,
DeathScreenWidgetClass);
557             if (DeathWidget) {
558                 DeathWidget->AddToViewport(20);
559                 DeathWidget->Show(RunTime, ClearedRooms, TotalRooms, EarnedDiamonds,
TotalDiamonds);
560             }
561         }
562     }
563 }
564 float ADepthrunCharacter::Heal(float Amount) {
565     if (Amount <= 0.0f || bIsDead || CurrentHP >= MaxHP) { return 0.0f; }
566     float OldHP = CurrentHP;
567     CurrentHP = FMath::Clamp(CurrentHP + Amount, 0.0f, MaxHP);
568     float ActualHealed = CurrentHP - OldHP;
569     UE_LOG(LogDepthrun, Log, TEXT("[Player] Healed: %.0f → %.0f (+%.0f)", OldHP, CurrentHP,
ActualHealed));
570     if (APlayerController* PC = Cast<APlayerController>(GetController()))
571         if (ADepthrunHUD* HUD = Cast<ADepthrunHUD>(PC->GetHUD()))
572             HUD->UpdatePlayerHP(CurrentHP, MaxHP);
573     return ActualHealed;
574 }
575 void ADepthrunCharacter::ApplyProfileUpgrades()
576 {
577     UDepthrunSaveSubsystem* Save = GetGameInstance()
? GetGameInstance()->GetSubsystem<UDepthrunSaveSubsystem>()
: nullptr;
578     if (!Save)
579     {
580         UE_LOG(LogDepthrun, Warning, TEXT("[Player] ApplyProfileUpgrades: no
SaveSubsystem"));
581     }
582     return;
583 }
584 int32 DamageLvl = Save->GetUpgradeLevel(EHubUpgrade::Damage);
585 int32 RangeLvl = Save->GetUpgradeLevel(EHubUpgrade::Range);
586 int32 ArrowCountLvl = Save->GetUpgradeLevel(EHubUpgrade::ArrowCount);

```

```

588         int32 MaxHPLvl      = Save->GetUpgradeLevel(EHubUpgrade::MaxHP);
589         DamageMultiplier    = HubUpgradeConfig::GetDamageMultiplier(DamageLvl);
590         MeleeRangeMultiplier = HubUpgradeConfig::GetMeleeRangeMultiplier(RangeLvl);
591         BaseProjectileCount  = HubUpgradeConfig::GetBaseProjectileCount(ArrowCountLvl);
592         float HPBonus = HubUpgradeConfig::GetMaxHPBonus(MaxHPLvl);
593         MaxHP = 500.f + HPBonus;
594         CurrentHP = FMath::Min(CurrentHP, MaxHP);
595         UE_LOG(LogDepthrunSave, Log, TEXT("[Player] Profile upgrades applied: Damage x%.2f,
Range x%.2f, Arrows %d, MaxHP %.0f"),
                    DamageMultiplier, MeleeRangeMultiplier, BaseProjectileCount, MaxHP);
596     }
597 }
598 void ADepthrunCharacter::ApplyWeaponProfileUpgrades()
599 {
600     if (ARangedWeapon* RangedWeapon = Cast<ARangedWeapon>(GetWeaponSlot(2)))
601     {
602         RangedWeapon->SetBaseShotsPerFire(BaseProjectileCount);
603         RangedWeapon->SetShotsPerFire(BaseProjectileCount);
604         UE_LOG(LogDepthrunSave, Log, TEXT("[Player] ApplyWeaponProfileUpgrades:
BaseShotsPerFire=%d"), BaseProjectileCount);
605     }
606     else { UE_LOG(LogDepthrunSave, Warning, TEXT("[Player] ApplyWeaponProfileUpgrades:
no ranged weapon in slot 2")); }
607 }
608 void ADepthrunCharacter::AddRunDiamonds(int32 Amount)
609 {
610     if (PlayerEconomy && Amount > 0)
611     {
612         PlayerEconomy->AddDiamonds(Amount);
613         UE_LOG(LogDepthrunEconomy, Log, TEXT("[Console] Added %d run diamonds"),
Amount);
614     }
615 }
616 void ADepthrunCharacter::AddProfileDiamonds(int32 Amount)
617 {
618     if (UDepthrunSaveSubsystem* Save = GetGameInstance() ? GetGameInstance()-
>GetSubsystem<UDepthrunSaveSubsystem>() : nullptr)
619     {
620         Save->AddDiamondsToProfile(Amount);
621         UE_LOG(LogDepthrunEconomy, Log, TEXT("[Console] Added %d profile diamonds"),
Amount);
622     }
623     else { UE_LOG(LogDepthrunEconomy, Error, TEXT("[Console] SaveSubsystem not
available")); }
624 }
625 void ADepthrunCharacter::BuyUpgradeCmd(const FString& UpgradeType)
626 {
627     if (UDepthrunSaveSubsystem* Save = GetGameInstance() ? GetGameInstance()-
>GetSubsystem<UDepthrunSaveSubsystem>() : nullptr)
628     {
629         EHubUpgrade Type = EHubUpgrade::Damage;
630         if (UpgradeType.Equals(TEXT("Damage"), ESearchCase::IgnoreCase)) Type =
EHubUpgrade::Damage;
631         else if (UpgradeType.Equals(TEXT("Range"), ESearchCase::IgnoreCase)) Type =
EHubUpgrade::Range;
632         else if (UpgradeType.Equals(TEXT("ArrowCount"), ESearchCase::IgnoreCase) ||
UpgradeType.Equals(TEXT("Arrows"), ESearchCase::IgnoreCase)) Type =
EHubUpgrade::ArrowCount;
633         else if (UpgradeType.Equals(TEXT("MaxHP"), ESearchCase::IgnoreCase) ||
UpgradeType.Equals(TEXT("HP"), ESearchCase::IgnoreCase)) Type = EHubUpgrade::MaxHP;
634         else
635         {
636             UE_LOG(LogDepthrunEconomy, Error, TEXT("[Console] Unknown upgrade
type: %s. Use: Damage, Range, ArrowCount, MaxHP"), *UpgradeType);
637             return;
638         }
639         if (Save->BuyUpgrade(Type))
640         {
641             UE_LOG(LogDepthrunEconomy, Log, TEXT("[Console] Successfully bought
%s upgrade"), *UpgradeType);
642             ApplyProfileUpgrades();
643         }
644         else { UE_LOG(LogDepthrunEconomy, Warning, TEXT("[Console] Failed to buy %s
upgrade (max level or insufficient diamonds)"), *UpgradeType); }
645     }
646     else { UE_LOG(LogDepthrunEconomy, Error, TEXT("[Console] SaveSubsystem not
available")); }
647 }
648 void ADepthrunCharacter::ShowProfile()
649 {
650     if (UDepthrunSaveSubsystem* Save = GetGameInstance() ? GetGameInstance()-

```

```

>GetSubsystem<UDepthrunSaveSubsystem>() : nullptr)
651     {
652         int32 Diamonds = Save->GetTotalDiamonds();
653         int32 DmgLvl = Save->GetUpgradeLevel(EHubUpgrade::Damage);
654         int32 RangeLvl = Save->GetUpgradeLevel(EHubUpgrade::Range);
655         int32 ArrowLvl = Save->GetUpgradeLevel(EHubUpgrade::ArrowCount);
656         int32 MaxHPLvl = Save->GetUpgradeLevel(EHubUpgrade::MaxHP);
657         UE_LOG(LogDepthrunEconomy, Log, TEXT("[Console] ===== PROFILE ====="));
658         UE_LOG(LogDepthrunEconomy, Log, TEXT("[Console] Total Diamonds: %d"),
Diamonds);
659         UE_LOG(LogDepthrunEconomy, Log, TEXT("[Console] Damage Lvl: %d (Cost: %d)",
DmgLvl, Save->GetUpgradeCost(EHubUpgrade::Damage));
660         UE_LOG(LogDepthrunEconomy, Log, TEXT("[Console] Range Lvl: %d (Cost: %d)",
RangeLvl, Save->GetUpgradeCost(EHubUpgrade::Range));
661         UE_LOG(LogDepthrunEconomy, Log, TEXT("[Console] ArrowCount Lvl: %d (Cost:
%d)", ArrowLvl, Save->GetUpgradeCost(EHubUpgrade::ArrowCount));
662         UE_LOG(LogDepthrunEconomy, Log, TEXT("[Console] MaxHP Lvl: %d (Cost: %d)",
MaxHPLvl, Save->GetUpgradeCost(EHubUpgrade::MaxHP));
663         UE_LOG(LogDepthrunEconomy, Log, TEXT("[Console] ====="));
664     }
665     else { UE_LOG(LogDepthrunEconomy, Error, TEXT("[Console] SaveSubsystem not
available")); }
666 }
667 void ADepthrunCharacter::ResetProfileCmd()
668 {
669     if (UDepthrunSaveSubsystem* Save = GetGameInstance() ? GetGameInstance()-
>GetSubsystem<UDepthrunSaveSubsystem>() : nullptr)
670     {
671         Save->ResetProfile();
672         UE_LOG(LogDepthrunEconomy, Log, TEXT("[Console] Profile reset to
defaults"));
673         ApplyProfileUpgrades();
674     }
675     else { UE_LOG(LogDepthrunEconomy, Error, TEXT("[Console] SaveSubsystem not
available")); }
676 }
677 void ADepthrunCharacter::GiveItem(const FString& ItemName)
678 {
679     if (!ItemCollection)
680     {
681         UE_LOG(LogDepthrun, Error, TEXT("[Console] GiveItem: ItemCollection not
assigned in BP_DepthrunCharacter!"));
682         return;
683     }
684     if (!ItemInventory)
685     {
686         UE_LOG(LogDepthrun, Error, TEXT("[Console] GiveItem: ItemInventory component
missing"));
687         return;
688     }
689     const FRunItemData* Found = ItemCollection->FindByName(ItemName);
690     if (!Found)
691     {
692         UE_LOG(LogDepthrun, Warning, TEXT("[Console] GiveItem: '%s' not found. Use
ListItems to see available items."), *ItemName);
693         return;
694     }
695     const bool bAdded = ItemInventory->AddItem(*Found);
696     if (bAdded)
697     {
698         if (SpawnedWeapon1) ItemInventory->ApplyToWeapon(SpawnedWeapon1);
699         if (SpawnedWeapon2) ItemInventory->ApplyToWeapon(SpawnedWeapon2);
700         ItemInventory->ApplyToCharacter(this);
701         UE_LOG(LogDepthrun, Log, TEXT("[Console] GiveItem: gave '%s', applied"),
*Found->ItemName);
702     }
703     else { UE_LOG(LogDepthrun, Warning, TEXT("[Console] GiveItem: could not add '%s'
(inventory full)", *Found->ItemName); }
704 }
705 void ADepthrunCharacter::ListItems()
706 {
707     if (!ItemCollection)
708     {
709         UE_LOG(LogDepthrun, Error, TEXT("[Console] ListItems: ItemCollection not
assigned in BP_DepthrunCharacter!"));
710         return;
711     }
712     UE_LOG(LogDepthrun, Log, TEXT("[Console] === Available Items (%d) ==="),
ItemCollection->Items.Num());
713     for (int32 i = 0; i < ItemCollection->Items.Num(); ++i)

```



```

714         {
715             const FRunItemData& Item = ItemCollection->Items[i];
716             UE_LOG(LogDepthrun, Log, TEXT("[Console] [%d] %s - %s"),
717                 i, *Item.ItemName, *UEnum::GetValueAsString(Item.Effect));
718         }
719         UE_LOG(LogDepthrun, Log, TEXT("[Console] Usage: GiveItem <name>"));
720     }
721     void ADepthrunCharacter::ClearRunItems()
722     {
723         if (!ItemInventory)
724         {
725             UE_LOG(LogDepthrun, Error, TEXT("[Console] ClearRunItems: ItemInventory
missing"));
726             return;
727         }
728         ItemInventory->ClearItems();
729         ApplyProfileUpgrades();
730         ApplyWeaponProfileUpgrades();
731         if (SpawnedWeapon1) ItemInventory->ApplyToWeapon(SpawnedWeapon1);
732         if (SpawnedWeapon2) ItemInventory->ApplyToWeapon(SpawnedWeapon2);
733         UE_LOG(LogDepthrun, Log, TEXT("[Console] ClearRunItems: all items cleared, base
stats restored"));
734     }
735     void ADepthrunCharacter::DBCheck()
736     {
737         UDepthrunSaveSubsystem* Save = GetGameInstance() ? GetGameInstance()-
>GetSubsystem<UDepthrunSaveSubsystem>() : nullptr;
738         USQLiteManager* DB = Save ? Save->GetDB() : nullptr;
739         if (!DB)
740         {
741             UE_LOG(LogDepthrunSave, Error, TEXT("[Console] DBCheck: DB not open"));
742             return;
743         }
744         const TArray<TMap<FString,FString>> Profile = DB-
>SelectRows(TEXT("player_profile"), TEXT("id=1"));
745         if (Profile.Num() > 0)
746         {
747             const TMap<FString,FString>& Row = Profile[0];
748             UE_LOG(LogDepthrunSave, Log, TEXT("[DBCheck] player_profile: Diamonds=%s
Dmg=%s Range=%s Arrows=%s HP=%s"),
749                 Row.Contains(TEXT("TotalDiamonds")) ? *Row[TEXT("TotalDiamonds")] :
TEXT("?"),
750                 Row.Contains(TEXT("Damage_Lvl")) ? *Row[TEXT("Damage_Lvl")] :
TEXT("?"),
751                 Row.Contains(TEXT("Range_Lvl")) ? *Row[TEXT("Range_Lvl")] :
TEXT("?"),
752                 Row.Contains(TEXT("ArrowCount_Lvl")) ? *Row[TEXT("ArrowCount_Lvl")] :
TEXT("?"),
753                 Row.Contains(TEXT("MaxHP_Lvl")) ? *Row[TEXT("MaxHP_Lvl")] :
TEXT("?"));
754         }
755         else { UE_LOG(LogDepthrunSave, Warning, TEXT("[DBCheck] player_profile: no rows"));
756         }
757         const TArray<TMap<FString,FString>> Runs = DB->SelectRows(TEXT("run_history"),
TEXT("1=1"));
758         UE_LOG(LogDepthrunSave, Log, TEXT("[DBCheck] run_history: %d rows"), Runs.Num());
759         for (const TMap<FString,FString>& Row : Runs)
760         {
761             UE_LOG(LogDepthrunSave, Log, TEXT("[DBCheck] id=%s Rooms=%s Won=%s
Duration=%ss Timestamp=%s"),
762                 Row.Contains(TEXT("id")) ? *Row[TEXT("id")] :
TEXT("?"),
763                 Row.Contains(TEXT("Rooms")) ? *Row[TEXT("Rooms")] :
TEXT("?"),
764                 Row.Contains(TEXT("Won")) ? *Row[TEXT("Won")] :
TEXT("?"),
765                 Row.Contains(TEXT("RunDuration")) ? *Row[TEXT("RunDuration")] :
TEXT("?"),
766                 Row.Contains(TEXT("Timestamp")) ? *Row[TEXT("Timestamp")] :
TEXT("?"));
767         }
768     }
769     void ADepthrunCharacter::UpdateFacingDirection(const FVector2D &Input) {
770         if (FMath::Abs(Input.X) >= FMath::Abs(Input.Y)) {
771             FacingDirection = Input.X > 0.f ? EPlayerFacingDirection::Right
: EPlayerFacingDirection::Left;
772         } else {
773             FacingDirection = Input.Y > 0.f ? EPlayerFacingDirection::Up
: EPlayerFacingDirection::Down;
774         }
775     }

```

```

776     }
777     FVector ADepthrunCharacter::GetFireDirection() const {
778         switch (FacingDirection) {
779             case EPlayerFacingDirection::Right:
780                 return FVector(0.f, 1.f, 0.f);
781             case EPlayerFacingDirection::Left:
782                 return FVector(0.f, -1.f, 0.f);
783             case EPlayerFacingDirection::Up:
784                 return FVector(1.f, 0.f, 0.f);
785             case EPlayerFacingDirection::Down:
786                 return FVector(-1.f, 0.f, 0.f);
787             default:
788                 return FVector(1.f, 0.f, 0.f);
789         }
790     }
791     void ADepthrunCharacter::UpdateAnimation() {
792         if (!GetSprite() || bIsDead)
793             return;
794         UPaperFlipbook *Desired = nullptr;
795         const bool bShouldFlip = (FacingDirection == EPlayerFacingDirection::Left);
796         FVector TargetScale = GetSprite()->GetRelativeScale3D();
797         TargetScale.X = bShouldFlip ? -1.f : 1.f;
798         if (GetSprite()->GetRelativeScale3D().X != TargetScale.X) { GetSprite()-
>SetRelativeScale3D(TargetScale); }
799         if (bIsHitAnimationActive && FB_Hit) {
800             Desired = FB_Hit;
801         } else if (bIsAttacking && CombatComponent &&
802             IsValid(CombatComponent->CurrentWeapon)) {
803             const bool bIsMelee =
804                 CombatComponent->CurrentWeapon->GetWeaponType() == EWeaponType::Melee;
805             switch (FacingDirection) {
806                 case EPlayerFacingDirection::Right:
807                 case EPlayerFacingDirection::Left:
808                     Desired = bIsMelee ? FB_MeleeAttackRight : FB_RangedAttackRight;
809                     break;
810                 case EPlayerFacingDirection::Up:
811                     Desired = bIsMelee ? FB_MeleeAttackUp : FB_RangedAttackUp;
812                     break;
813                 case EPlayerFacingDirection::Down:
814                     Desired = bIsMelee ? FB_MeleeAttackDown : FB_RangedAttackDown;
815                     break;
816             }
817         } else if (bIsMoving) {
818             switch (FacingDirection) {
819                 case EPlayerFacingDirection::Right:
820                 case EPlayerFacingDirection::Left:
821                     Desired = FB_WalkRight;
822                     break;
823                 case EPlayerFacingDirection::Up:
824                     Desired = FB_WalkUp;
825                     break;
826                 case EPlayerFacingDirection::Down:
827                     Desired = FB_WalkDown;
828                     break;
829             }
830         } else {
831             switch (FacingDirection) {
832                 case EPlayerFacingDirection::Right:
833                 case EPlayerFacingDirection::Left:
834                     Desired = FB_IdleRight;
835                     break;
836                 case EPlayerFacingDirection::Up:
837                     Desired = FB_IdleUp;
838                     break;
839                 case EPlayerFacingDirection::Down:
840                     Desired = FB_IdleDown;
841                     break;
842             }
843         }
844         if (!Desired) {
845             switch (FacingDirection) {
846                 case EPlayerFacingDirection::Right:
847                 case EPlayerFacingDirection::Left:
848                     Desired = FB_WalkRight ? FB_WalkRight : FB_IdleRight;
849                     break;
850                 case EPlayerFacingDirection::Up:
851                     Desired = FB_WalkUp ? FB_WalkUp : FB_IdleUp;
852                     break;
853                 case EPlayerFacingDirection::Down:
854                     Desired = FB_WalkDown ? FB_WalkDown : FB_IdleDown;

```

```

855         break;
856     }
857 }
858 if (Desired && GetSprite()->GetFlipbook() != Desired) { GetSprite()-
>SetFlipbook(Desired); }
859 }
860 FText ADepthrunCharacter::GetGodStatusText() const { return bGodMode ?
FText::FromString(TEXT("GOD MODE: ON")) : FText::GetEmpty(); }
861 FText ADepthrunCharacter::GetSuperAttackStatusText() const {
862     return bSuperAttack ? FText::FromString(TEXT("SUPER ATTACK: ON"))
863         : FText::GetEmpty();
864 }

```

Source\Depthrun\RoomGeneration

RoomGeneratorSubsystem.cpp

```

001 #include "RoomGeneratorSubsystem.h"
002 #include "RoomBase.h"
003 #include "RoomTemplate.h"
004 #include "DungeonTypes.h"
005 #include "Engine/World.h"
006 #include "Kismet/GameplayStatics.h"
007 #include "GameFramework/Character.h"
008 #include "PaperCharacter.h"
009 #include "PaperFlipbookComponent.h"
010 #include "Player/DepthrunCharacter.h"
011 #include "UI/DepthrunHUD.h"
012 #include "UI/HUDOverlayWidget.h"
013 #include "GameFramework/PlayerController.h"
014 namespace RoomGeneratorLocal
015 {
016 constexpr float RoomGenBaseTileSize = 16.0f;
017 float ResolveRoomGenWorldScale(const URoomTemplate* Template)
018 {
019     if (!Template || Template->WorldScale <= 0.01f) { return 2.6f; }
020     return Template->WorldScale;
021 }
022 FVector ResolveRoomGenSpawnOffset(const URoomTemplate* Template)
023 {
024     if (!Template) { return FVector::ZeroVector; }
025     return Template->SpawnOffset;
026 }
027 }
028 void URoomGeneratorSubsystem::Initialize(FSubsystemCollectionBase& Collection) {
Super::Initialize(Collection); }
029 void URoomGeneratorSubsystem::Deinitialize() { Super::Deinitialize(); }
030 void URoomGeneratorSubsystem::GenerateRooms(int32 RoomCount)
031 {
032     if (RoomCount <= 0) return;
033     for (ARoomBase* Room : GeneratedRooms) { if (Room) Room->Destroy(); }
034     GeneratedRooms.Empty();
035     CurrentRoomIndex = 0;
036     TSet<FIntPoint> Occupied;
037     TArray<FIntPoint> Coords;
038     FIntPoint Pos(0, 0);
039     Coords.Add(Pos);
040     Occupied.Add(Pos);
041     TArray<FIntPoint> Dirs = { {1,0}, {-1,0}, {0,1}, {0,-1} };
042     int32 LastIdx = 0;
043     for (int32 i = 1; i < RoomCount; ++i)
044     {
045         bool bFound = false;
046         for (int32 At = 0; At < 40; ++At)
047         {
048             FIntPoint Parent;
049             if (FMath::FRand() < 0.7f)
050                 Parent = Coords[LastIdx];
051             else
052                 Parent = Coords[FMath::RandRange(0, Coords.Num() - 1)];
053             TArray<FIntPoint> ShuffledDirs = Dirs;
054             for (int32 d = ShuffledDirs.Num() - 1; d > 0; --d)
055             {
056                 int32 j = FMath::RandRange(0, d);
057                 ShuffledDirs.Swap(d, j);
058             }
059             for (const FIntPoint& Dir : ShuffledDirs)
060             {
061                 FIntPoint Candidate = Parent + Dir;
062                 if (Occupied.Contains(Candidate)) continue;
063                 int32 NeighborCount = 0;
064                 for (const FIntPoint& D2 : Dirs)

```

```

065         if (Occupied.Contains(Candidate + D2)) ++NeighborCount;
066         if (NeighborCount > 1) continue;
067         Coords.Add(Candidate);
068         Occupied.Add(Candidate);
069         LastIdx = Coords.Num() - 1;
070         bFound = true;
071         break;
072     }
073     if (bFound) break;
074 }
075 if (!bFound)
076 {
077     for (int32 At2 = 0; At2 < 20 && !bFound; ++At2)
078     {
079         FIntPoint Parent = Coords[FMATH::RandRange(0, Coords.Num() - 1)];
080         FIntPoint Candidate = Parent + Dirs[FMATH::RandRange(0, 3)];
081         if (!Occupied.Contains(Candidate))
082         {
083             Coords.Add(Candidate);
084             Occupied.Add(Candidate);
085             LastIdx = Coords.Num() - 1;
086             bFound = true;
087         }
088     }
089 }
090 if (!bFound) break;
091 }
092 if (Coords.Num() > 2)
093 {
094     int32 BossIdx = -1;
095     int32 MaxDist = 0;
096     for (int32 k = 1; k < Coords.Num(); ++k)
097     {
098         int32 NeighborCount = 0;
099         for (const FIntPoint& D : Dirs)
100             if (Occupied.Contains(Coords[k] + D)) ++NeighborCount;
101         if (NeighborCount == 1)
102         {
103             int32 Dist = FMATH::Abs(Coords[k].X) + FMATH::Abs(Coords[k].Y);
104             if (Dist > MaxDist)
105             {
106                 MaxDist = Dist;
107                 BossIdx = k;
108             }
109         }
110     }
111     if (BossIdx == -1)
112     {
113         for (int32 k = 1; k < Coords.Num(); ++k)
114         {
115             int32 Dist = FMATH::Abs(Coords[k].X) + FMATH::Abs(Coords[k].Y);
116             if (Dist > MaxDist) { MaxDist = Dist; BossIdx = k; }
117         }
118     }
119     if (BossIdx != -1 && BossIdx != Coords.Num() - 1)
120         Coords.Swap(BossIdx, Coords.Num() - 1);
121     UE_LOG(LogTemp, Log, TEXT("[DungeonGen] Boss Room at grid (%d,%d), dist=%d from
122 start"),
123           Coords.Last().X, Coords.Last().Y, MaxDist);
124 }
125 const float TileSize = RoomGeneratorLocal::RoomGenBaseTileSize *
126   RoomGeneratorLocal::ResolveRoomGenWorldScale(StartTemplate);
127 const float RoomSizeX = 6.0f * TileSize;
128 const float RoomSizeY = 8.0f * TileSize;
129 for (int32 i = 0; i < Coords.Num(); ++i)
130 {
131     FIntPoint C = Coords[i];
132     FVector Loc = FVector(C.Y * RoomSizeX, C.X * RoomSizeY, 0.f);
133     URoomTemplate* T = nullptr;
134     if (i == 0) T = StartTemplate;
135     else if (i == Coords.Num() - 1) T = BossTemplate;
136     else
137     {
138         float Rand = FMATH::FRand();
139         if (Rand < 0.15f && TreasurePool.Num() > 0) T = TreasurePool[FMATH::RandRange(0,
140 TreasurePool.Num() - 1)];
141         else if (Rand < 0.25f && RestPool.Num() > 0) T = RestPool[FMATH::RandRange(0,
142 RestPool.Num() - 1)];
143         else if (CombatPool.Num() > 0) T = CombatPool[FMATH::RandRange(0,
144 CombatPool.Num() - 1)];

```

```

141         if (!T && CombatPool.Num() > 0) T = CombatPool[0];
142     }
143     if (T)
144     {
145         FActorSpawnParameters Params;
146         Params.SpawnCollisionHandlingOverride =
ESpawnActorCollisionHandlingMethod::AlwaysSpawn;
147         ARoomBase* NewRoom = GetWorld()-
>SpawnActor<ARoomBase>(ARoomBase::StaticClass(), Loc, FRotator::ZeroRotator, Params);
148         if (NewRoom)
149         {
150             NewRoom->SetupRoom(T,
151                 Occupied.Contains(C + FIntPoint(0, 1)),
152                 Occupied.Contains(C + FIntPoint(0, -1)),
153                 Occupied.Contains(C + FIntPoint(-1, 0)),
154                 Occupied.Contains(C + FIntPoint(1, 0)));
155             GeneratedRooms.Add(NewRoom);
156         }
157     }
158 }
159 if (GeneratedRooms.Num() > 0)
160 {
161     ActivateRoom(0);
162     if (ACharacter* Player = UGameplayStatics::GetPlayerCharacter(GetWorld(), 0))
163     {
164         FVector SpawnPos = GeneratedRooms[0]->GetActorLocation();
165         SpawnPos.Z = StartTemplate ? StartTemplate->PlayerLockedZ : 1.0f;
166         Player->SetActorLocation(SpawnPos);
167         Player->SetActorRotation(FRotator::ZeroRotator);
168         Player->SetActorHiddenInGame(false);
169         Player->SetActorEnableCollision(true);
170         if (ADepthrunCharacter* DepthrunPlayer = Cast<ADepthrunCharacter>(Player))
171         {
172             const float LockedZ = StartTemplate ? StartTemplate->PlayerLockedZ : 4.0f;
173             DepthrunPlayer->SetLockedZ(LockedZ);
174             UE_LOG(LogTemp, Log, TEXT("[RoomGen] Player Z locked to %.1f (from
template)"), LockedZ);
175         }
176         if (APaperCharacter* PaperPlayer = Cast<APaperCharacter>(Player))
177         {
178             if (UPaperFlipbookComponent* Sprite = PaperPlayer->GetSprite())
179             {
180                 Sprite->SetHiddenInGame(false);
181                 Sprite->SetVisibility(true, true);
182                 Sprite->SetTranslucentSortPriority(100);
183                 Sprite->MarkRenderStateDirty();
184             }
185         }
186     }
187 }
188 UE_LOG(LogTemp, Log, TEXT("[DungeonGen] Successfully generated %d rooms"),
GeneratedRooms.Num());
189 }
190 void URoomGeneratorSubsystem::ActivateRoom(int32 Index)
191 {
192     if (!GeneratedRooms.IsValidIndex(Index)) return;
193     CurrentRoomIndex = Index;
194     if (Index == 0)
195     {
196         GeneratedRooms[Index]->SetIsActive(true);
197         UE_LOG(LogTemp, Log, TEXT("[DungeonGen] Activating Start Room %d (doors stay
open)"), Index);
198     }
199     else
200     {
201         GeneratedRooms[Index]->ActivateRoom();
202         UE_LOG(LogTemp, Log, TEXT("[DungeonGen] Activating Combat Room %d"), Index);
203     }
204 }
205 void URoomGeneratorSubsystem::OnPlayerEnteredRoom(ARoomBase* Room)
206 {
207     if (!Room) return;
208     int32 Index = GeneratedRooms.IndexOfByKey(Room);
209     if (Index != INDEX_NONE && Index != CurrentRoomIndex) { ActivateRoom(Index); }
210     if (APlayerController* PC = UGameplayStatics::GetPlayerController(GetWorld(), 0))
211     {
212         if (ADepthrunHUD* HUD = Cast<ADepthrunHUD>(PC->GetHUD()))
213             if (UHUDOverlayWidget* Overlay = HUD->GetHUDOverlay())
214                 Overlay->SetRoomInfo(GetClearedRoomsCount(), GeneratedRooms.Num() - 1);
215     }
216 void URoomGeneratorSubsystem::SetTemplates(URoomTemplate* Start, URoomTemplate* Boss, const

```

```

TArray<URoomTemplate*>& Combat, const TArray<URoomTemplate*>& Treasure, const
TArray<URoomTemplate*>& Rest)
216 {
217     StartTemplate = Start;
218     BossTemplate = Boss;
219     CombatPool = Combat;
220     TreasurePool = Treasure;
221     RestPool = Rest;
222 }
223 ARoomBase* URoomGeneratorSubsystem::GetCurrentActiveRoom() const
224 {
225     if (GeneratedRooms.IsValidIndex(CurrentRoomIndex)) { return
GeneratedRooms[CurrentRoomIndex].Get(); }
226     return nullptr;
227 }
228 int32 URoomGeneratorSubsystem::GetClearedRoomsCount() const
229 {
230     int32 Count = 0;
231     for (int32 i = 1; i < GeneratedRooms.Num(); ++i) { if (GeneratedRooms[i] &&
GeneratedRooms[i]->IsCleared()) ++Count; }
232     return Count;
233 }
234 void URoomGeneratorSubsystem::NotifyRoomCleared()
235 {
236     if (APlayerController* PC = UGameplayStatics::GetPlayerController(GetWorld(), 0))
237         if (ADepthrunHUD* HUD = Cast<ADepthrunHUD>(PC->GetHUD()))
238             if (UHUDOverlayWidget* Overlay = HUD->GetHUDOverlay())
239                 Overlay->SetRoomInfo(GetClearedRoomsCount(), GeneratedRooms.Num() - 1);
240 }
241 void URoomGeneratorSubsystem::OnPlayerEnteredTransition(ARoomBase* FromRoom, int32
ExitIndex)
242 {
243     UE_LOG(LogTemp, Log, TEXT("[DungeonGen] Player entered transition from room %s"),
FromRoom ? *FromRoom->GetName() : TEXT("NULL"));
244     if (FromRoom && !FromRoom->IsCleared())
245     {
246         UE_LOG(LogTemp, Log, TEXT("[DungeonGen] Opening doors in room %s (player leaving)",
*FromRoom->GetName());
247         FromRoom->DeactivateRoom();
248     }
249 }

```

Source\Depthrun\Data

DepthrunSaveSubsystem.cpp

```

001 #include "DepthrunSaveSubsystem.h"
002 #include "SQLiteManager.h"
003 #include "DepthrunLogChannels.h"
004 #include "Misc/Paths.h"
005 void UDepthrunSaveSubsystem::Initialize(FSubsystemCollectionBase& Collection)
006 {
007     Super::Initialize(Collection);
008     UE_LOG(LogDepthrunSave, Log, TEXT("[Save] Initializing SaveSubsystem..."));
009     DB = NewObject<USQLiteManager>(this);
010     if (!DB)
011     {
012         UE_LOG(LogDepthrunSave, Error, TEXT("[Save] Failed to create"));
013         return;
014     }
015     const FString DBPath = FPaths::ProjectSavedDir() / TEXT("Depthrun.db");
016     UE_LOG(LogDepthrunSave, Log, TEXT("[Save] Database path: %s"), *DBPath);
017     if (DB->OpenDatabase(DBPath))
018     {
019         UE_LOG(LogDepthrunSave, Log, TEXT("[Save] Database opened successfully"));
020         InitializeSchema();
021     }
022     else { UE_LOG(LogDepthrunSave, Error, TEXT("[Save] Failed to open database at %s"),
*DBPath); }
023 }
024 void UDepthrunSaveSubsystem::Deinitialize()
025 {
026     if (DB) { DB->CloseDatabase(); }
027     Super::Deinitialize();
028 }
029 void UDepthrunSaveSubsystem::InitializeSchema()
030 {
031     if (!DB)
032     {
033         UE_LOG(LogDepthrunSave, Error, TEXT("[Save] InitializeSchema: DB is null,
aborting"));
034         return;

```

```

035     }
036     UE_LOG(LogDepthrunSave, Log, TEXT("[Save] InitializeSchema: step 1 - CREATE TABLE
player_profile"));
037     const FString CreateProfileTable = TEXT(
038         "CREATE TABLE IF NOT EXISTS player_profile ("
039         "id INTEGER PRIMARY KEY CHECK (id = 1),"
040         "TotalDiamonds INTEGER DEFAULT 0,"
041         "Damage_Lvl INTEGER DEFAULT 0,"
042         "Range_Lvl INTEGER DEFAULT 0,"
043         "ArrowCount_Lvl INTEGER DEFAULT 0,"
044         "MaxHP_Lvl INTEGER DEFAULT 0"
045         ");"
046     );
047     const bool bTableCreated = DB->ExecuteQuery(CreateProfileTable);
048     UE_LOG(LogDepthrunSave, Log, TEXT("[Save] CREATE TABLE ExecuteQuery result: %s"),
049         bTableCreated ? TEXT("returned true") : TEXT("returned false - check SQL
error above"));
050     const TArray<TMap<FString, FString>> TableCheck =
051         DB->SelectRows(TEXT("sqlite_master"), TEXT("type='table' AND
name='player_profile'"));
052     const bool bTablePhysicallyExists = TableCheck.Num() > 0;
053     UE_LOG(LogDepthrunSave, Log, TEXT("[Save] sqlite_master check - player_profile
physically exists: %s"),
054         bTablePhysicallyExists ? TEXT("YES") : TEXT("NO - table was NOT created!"));
055     if (!bTablePhysicallyExists)
056     {
057         return;
058     }
059     UE_LOG(LogDepthrunSave, Log, TEXT("[Save] InitializeSchema: step 2 - INSERT OR
IGNORE default row"));
060     const FString InsertDefault = TEXT(
061         "INSERT OR IGNORE INTO player_profile (id, TotalDiamonds, Damage_Lvl,
Range_Lvl, ArrowCount_Lvl, MaxHP_Lvl) "
062         "VALUES (1, 0, 0, 0, 0, 0);"
063     );
064     const bool bRowInserted = DB->ExecuteQuery(InsertDefault);
065     UE_LOG(LogDepthrunSave, Log, TEXT("[Save] INSERT OR IGNORE default row: %s"),
066         bRowInserted ? TEXT("OK") : TEXT("FAILED"));
067     const TArray<TMap<FString, FString>> RowCheck =
068         DB->SelectRows(TEXT("player_profile"), TEXT("id = 1"));
069     UE_LOG(LogDepthrunSave, Log, TEXT("[Save] Row verification - rows in player_profile:
%d"), RowCheck.Num());
070     if (RowCheck.Num() > 0) { UE_LOG(LogDepthrunSave, Log, TEXT("[Save] Schema OK -
player_profile table and default row confirmed")); }
071     else { UE_LOG(LogDepthrunSave, Error, TEXT("[Save] Schema FAILED - table exists but
default row missing")); }
072     UE_LOG(LogDepthrunSave, Log, TEXT("[Save] InitializeSchema: step 3 - CREATE TABLE
run_history"));
073     const FString CreateRunHistory = TEXT(
074         "CREATE TABLE IF NOT EXISTS run_history ("
075         "id INTEGER PRIMARY KEY,"
076         "Rooms INTEGER DEFAULT 0,"
077         "Won INTEGER DEFAULT 0,"
078         "RunDuration REAL DEFAULT 0.0,"
079         "Timestamp TEXT DEFAULT (datetime('now'))"
080         ");"
081     );
082     const bool bRunHistoryCreated = DB->ExecuteQuery(CreateRunHistory);
083     UE_LOG(LogDepthrunSave, Log, TEXT("[Save] CREATE TABLE run_history: %s"),
084         bRunHistoryCreated ? TEXT("OK") : TEXT("FAILED"));
085 }
086 void UDepthrunSaveSubsystem::SaveRunResult(int32 Rooms, int32 DurationSeconds, bool bWon)
087 {
088     if (!DB) { return; }
089     const FString Query = FString::Printf(
090         TEXT("INSERT INTO run_history (Rooms, Won, RunDuration) VALUES (%d, %d,
%.2f);"),
091         Rooms, bWon ? 1 : 0, (float)DurationSeconds
092     );
093     const bool bOk = DB->ExecuteQuery(Query);
094     UE_LOG(LogDepthrunSave, Log, TEXT("[Save] SaveRunResult: Rooms=%d Won=%s
Duration=%ds - %s"),
095         Rooms, bWon ? TEXT("YES") : TEXT("NO"), DurationSeconds, bOk ? TEXT("OK") :
TEXT("FAILED"));
096 }
097 bool UDepthrunSaveSubsystem::LoadProgress(int32& OutBestFloor, int32& OutTotalRuns)
098 {
099     OutTotalRuns = 0;
100     return false;
101 }

```

```

102 void UDepthrunSaveSubsystem::SaveAdaptiveWeights(const TArray<float>& Weights, int32
RunNumber) { }
103 int32 UDepthrunSaveSubsystem::GetTotalDiamonds() const
104 {
105     if (!DB) { return 0; }
106     TArray<TMap<FString, FString>> Rows = DB->SelectRows(TEXT("player_profile"),
TEXT("id = 1"));
107     if (Rows.Num() > 0 && Rows[0].Contains(TEXT("TotalDiamonds"))) { return
FCString::Atoi(*Rows[0][TEXT("TotalDiamonds")]); }
108     return 0;
109 }
110 void UDepthrunSaveSubsystem::AddDiamondsToProfile(int32 Amount)
111 {
112     if (!DB || Amount <= 0) { return; }
113     int32 Current = GetTotalDiamonds();
114     int32 NewTotal = Current + Amount;
115     FString Query = FString::Printf(
116         TEXT("UPDATE player_profile SET TotalDiamonds = %d WHERE id = 1;"),
117         NewTotal
118     );
119     DB->ExecuteQuery(Query);
120     UE_LOG(LogDepthrunSave, Log, TEXT("[Save] Diamonds: %d → %d (+%d)", Current,
NewTotal, Amount);
121 }
122 int32 UDepthrunSaveSubsystem::GetUpgradeLevel(EHubUpgrade Type) const
123 {
124     if (!DB) { return 0; }
125     const TCHAR* ColumnName = TEXT("");
126     switch (Type)
127     {
128         case EHubUpgrade::Damage: ColumnName = TEXT("Damage_Lvl"); break;
129         case EHubUpgrade::Range: ColumnName = TEXT("Range_Lvl"); break;
130         case EHubUpgrade::ArrowCount: ColumnName = TEXT("ArrowCount_Lvl"); break;
131         case EHubUpgrade::MaxHP: ColumnName = TEXT("MaxHP_Lvl"); break;
132     }
133     TArray<TMap<FString, FString>> Rows = DB->SelectRows(TEXT("player_profile"),
TEXT("id = 1"));
134     if (Rows.Num() > 0 && Rows[0].Contains(ColumnName)) { return
FCString::Atoi(*Rows[0][ColumnName]); }
135     return 0;
136 }
137 int32 UDepthrunSaveSubsystem::GetUpgradeCost(EHubUpgrade Type) const
138 bool UDepthrunSaveSubsystem::BuyUpgrade(EHubUpgrade Type)
139 {
140     if (!DB) { return false; }
141     int32 CurrentLevel = GetUpgradeLevel(Type);
142     if (CurrentLevel >= HubUpgradeConfig::GetMaxLevel(Type))
143     {
144         UE_LOG(LogDepthrunSave, Warning, TEXT("[Save] BuyUpgrade failed: %s already
at max level %d"),
145             *UEEnum::GetValueAsString(Type), CurrentLevel);
146         return false;
147     }
148     int32 Cost = HubUpgradeConfig::GetUpgradeCostForType(Type, CurrentLevel);
149     int32 CurrentDiamonds = GetTotalDiamonds();
150     if (CurrentDiamonds < Cost)
151     {
152         UE_LOG(LogDepthrunSave, Log, TEXT("[Save] BuyUpgrade failed: need %d
diamonds, have %d"),
153             Cost, CurrentDiamonds);
154         return false;
155     }
156     int32 NewDiamonds = CurrentDiamonds - Cost;
157     int32 NewLevel = CurrentLevel + 1;
158     const TCHAR* ColumnName = TEXT("");
159     switch (Type)
160     {
161         case EHubUpgrade::Damage: ColumnName = TEXT("Damage_Lvl"); break;
162         case EHubUpgrade::Range: ColumnName = TEXT("Range_Lvl"); break;
163         case EHubUpgrade::ArrowCount: ColumnName = TEXT("ArrowCount_Lvl"); break;
164         case EHubUpgrade::MaxHP: ColumnName = TEXT("MaxHP_Lvl"); break;
165     }
166     FString Query = FString::Printf(
167         TEXT("UPDATE player_profile SET TotalDiamonds = %d, %s = %d WHERE id = 1;"),
168         NewDiamonds, ColumnName, NewLevel
169     );
170     DB->ExecuteQuery(Query);
171     UE_LOG(LogDepthrunSave, Log, TEXT("[Save] Upgrade %s: level %d → %d, diamonds %d"),
return true;
172 }
173 }

```



**ПРИЛОЖЕНИЕ Б**  
(обязательное)

**Спецификация**

## ПРИЛОЖЕНИЕ В (обязательное)

### Отчет о проверке на заимствования

Отчет с результатом проверки на сайте «Text.ru» приведен на рисунке В.1.

The screenshot shows the Text.ru interface for document plagiarism checking. The top navigation bar includes the Text.ru logo, account status (PRO-аккаунт), and various service statistics. The main heading is "Проверка документа на плагиат". Below this, there's a section for uploading documents with a button "Загрузите документы" and a note about the maximum file size (10 MB). A table displays the check results for a file named "ПЗ\_А4\_Снитко.docx". The table has columns for file name, text, number of symbols, number of words, date added, and uniqueness percentage. The uniqueness percentage is 97.75%.

| Имя файла         | Текст                           | Число символов | Число слов | Дата добавления | Уникальность |
|-------------------|---------------------------------|----------------|------------|-----------------|--------------|
| ПЗ_А4_Снитко.docx | СОДЕРЖАНИЕ ТОС\o "1-3" \h \z... | 225460         | 25458      | Tue.May.26      | 97.75%       |

Рисунок В.1 – Результат проверки дипломного проекта

**ПРИЛОЖЕНИЕ Г**  
(обязательное)

**Ведомость документов**